Data science notes

Libero Biagi

June 23, 2026

Contents

1 Data Mining

1.1 Introduction

Fernando Becao presents himself and the tutor.
Fernando Becao asks us why we weren't at the party.
We study about supervised and unsupervised learning.
Exam + project.
Exam is 55% of the final grade
Project is 45% and is divided into

- Deliverable 1 -> 30% by November 4th
- Deliverable 2 -> 60% by January 3rd
- Discussion -> 10%

Both exam and project will expire after the year

1.2 Context

Random debate about things
We need to find relevant data to reduce computation and improve the quality of works

Big Data -> we simplify datasets that are too big for normal processing. From big data into mean datasets
Big data characteristics:

- **Volume:** big data are big
- **Velocity:** Big data technology allows databases to process and analyze data while it is being generated
- **Variety:** Big data can be a mixture of structured, unstructured and semi-structured data. To solve this problem Big Data is flexible

Some information about AI, ML and Data Science.

AI: Making things that show human intelligence. Automatize human tasks.

Machine Learning: Approach AI using systems that can find patterns from data and examples. ML systems learn by themselves. We can see them as a sort of subset of AI or a way towards AI

Data Science: the study of where information comes from and how to turn it into a valuable resource.

Data Science vs Data Mining

Data science is a set of principle that guide the extraction of information from data. We try to view problems from a data perspective.

Data mining on the other hand is the extraction of knowledge from data via the use of algorithms.

Find/build attributes is very important. Basically the properties of the things that we are studying have to be selected or preprocessed.

After the construction of the features a ML model can learn how to divide the instances on an hyperplane. This can be used to make predictions. Recall that a model will predict only based on the class that it was trained on.

Is very important to use the relevant and appropriate features. More features can mean more possibility of discriminating between the classes.

Typically we use labeled examples, enough data and clear cut definitions.

If our models try to attribute a label we have supervised learning. Future examples will get a prediction.

To build features we use data warehouse and ETL (extract, transform and load). Recall that we can transform every relational schema into a table (at least with SQL).

Minimum information to identify a costumer:

- Transaction number
- Date and time of transaction
- Item purchased
- Price
- Quantity purchased
- A table that matches product code to its name, subgroup code to name, product group code to group name.
- Product taxonomy to link product code to subgroup code and product subgroup code to product group code.
- Card ID

Consistent behaviors are easier to analyze. To found the level of consistency i can use the standard deviation and I can remove the outliers.

We can also look at relevant variables like:

- Recency
- Frequency
- Monetary value
- Average purchase
- Most frequent store
- Average time between transactions
- Standard deviation of transactional interval
- Costumer stability index
- Relative spend on each product

1.3 Canonical tasks in data mining

If we want to classify new data from a decision criterion previously learned we talk about **Supervised learning**

If we want to summarize a data set we talk about **Unsupervised learning**

Supervised learning:

- Classification
- Regression

Unsupervised learning:

- Clustering
- Visualization
- Association

In our datasets the features are the columns while the rows are the instances

Clustering -> We plot the instances on a hyperplane and we try to group them by distance, we can have different plots

Association rules -> based on our data we can use the transactions to find some rule to infer costumer routine. We use confidence, support, lift and so on.

Visualization -> n-D data can be difficult to visualize so we can flatten them into 2-D ones. Examples are histograms, bubbles and goggle boxes. We can also do some dimensionality reduction using PCA or other algorithm.

1.4 The data mining process

We can have different methodologies to acquire insights from data.

KDD

1. Data
2. Selection
3. Preprocessing
4. Transformation
5. Data mining
6. Interpretation/Evaluation
7. Knowledge

CRISP-DM

1. Business understanding
2. Data understanding
3. Data preparation
4. Modeling
5. Evaluation
6. Deployment

We use ML algorithms instead of fixed formulas because the events are too complex for a simple formula. We don't understand the problems but we try to approximate solution. Black Box ML approach basically.

If the model is not good enough we can either improve the model or gather more data. The second one is quicker. Dumb algorithms with a lot of data will learn better than smart ones with not a lot of them.

Traditional statistics might be described as being characterized by data sets which are small and clean, which are static, which were sampled in an iid manner, which were often collected to answer the particular problem being addressed, and which are solely numeric.

Dimension	Primary data	Secondary data
Definition	Data you collect yourself for a specific, current purpose.	Data collected by others for a different (often past) purpose.
Typical sources	Surveys, experiments, interviews, field measurements, sensors.	Government statistics, research papers, data portals, company databases, syndicated datasets, web-scraped corpora.
Control over design	Full control (sampling, instruments, definitions, timing).	Little/no control; must accept others' design choices.
Fit to your question	High: tailored to your problem and target population.	Varies: often indirect or requires redefinition/derivations.
Cost	Usually higher (money, time, staff, tooling).	Usually lower or free; licensing may apply.
Time to obtain	Longer (planning → collection → cleaning).	Faster (download/access + cleaning/understanding).
Timeliness/recency	Up-to-date by design.	May be outdated; release lags common.
Granularity	Exactly what you need (variables, frequency, detail).	Fixed by source; may lack key variables or be too aggregated.

Table 1: Comparison between primary and secondary data.

Size of data set is also useful. If we have big datasets even tiny effects exists. They can be useless. Now we should ask if the effect is important or not.

From statistical significance to substantive significance.

Since the datasets are very big we try to compute them in a adaptive or sequential way. Also we might have multiple and interrelated files

We have two main ways to process data

- Incremental
- Batch

Other characteristics of problems in data mining are

- Nonstationarity and population drift
- Selection bias
- Spurious relationships

Input variables should be causally related to the output. If we have a small number of observations we will have high correlation.

Confounding variables will correlate both with dependent and independent variables. The confounding factor will estimate incorrectly the relation. A **Spurious relationship** happens when we perceive a relationship between two variables that actually doesn't exist. We ar not accounting for the confounding factor.

Input variables must be causally related to the output to be meaningful.

We have to discriminate between **causality** and **correlation**:

- Correlation -> two things co-occurs, changing one of them will not change the output
- Causality -> a change on the input will cause a change on the output

Correlation is a pattern, causation a consequence

Input Space -> is the input feature vector, the algorithm will look for a solution

Curse of dimensionality -> bad effects can be caused by redundant features, bigger dimensionality means bigger and more sparse input. Clustering can be harder. Generalization is exponentially harder. Feature selection can be an answer.

Input space coverage -> representative training examples will improve our model quality. Test data outside the training input space will be bad for the performance.

Interpolation -> predictions in the range of data that we have
Extrapolation predictions outside the range of data that we have, Time series
Separation -> if we plot our data on a 2-D space we can be able to draw by hand the regions where the data are. ML models will try to do it mathematically. If we can use a linear hyperplane we have **Linearly separable data** if not they are not linearly separable. With separable classes we can get 0 errors. We want to minimize the error (finding the Bayes error). Simple algorithm like perceptron will solve problems with separables classes.

Kind of variables

- Nominal
- Ordinal
- Discrete
- Continuous
- Interval
- Ratio

Metadata -> Information that provides information about data

- Descriptive metadata
- Structural metadata
- Administrative metadata

1.5 Exploration

Today visualization Strong points of good visualization

- Can show processes
- Show comparisons between numbers
- Can show differences and changes
- Can use geography to show local data
- Can show relationships
- Can show density

What not to do

- Don't clog the graph
- No 3-D
- Don't use unfaithful charts

Guidelines

- Reduce chartjunk
- Increase data-ink ration
- Use similar structures in the same charts

Tufte lie factor -> Measure of distortion in a graph

$$\text{Lie Factor} = \frac{\text{Size of effect in graph}}{\text{Size of effect in data}} \tag{1}$$

As rule of thumb we should have $0.95 < \text{Lie Factor} < 1.05$
Suggestions

- White background
- Don't use colors if not necessary, Charts should be like man suits (Prof, 2025)
- Order the items in a smart way
- Use a scale
- Crisp borders
- No smooth line
- Simple is better
- No 3-D
- Discriminate clearly the clusters with shapes and colors
- Spaghetti chart should highlight interesting patterns
- Clutterplot should highlight interesting points
- Use shadows to highlights interesting points

Charts that can be useful

- Bar charts, vertical and horizontal

- Line charts
- Mix of the previous
- Stacked bar charts
- Scatterplot, also with tendency lines, grids, bubbles
- Pie charts, can mix them with histograms, we can label them, compare with others (same as paired column chart). They are like stacked bar charts. We also have part to whole mini pie charts. We can plot them on a graph.
- Radar plot, easy to compare more of them

Visualization for analysis

- Histogram, can be stacked and combined with scatterplots
- Boxplot, can be combined with scatterplots, histograms

Correlation matrices

- Can be done with distributions, scatter plots and matrices

Parallel coordinate

- Show patterns that can be compared easily

Small Multiples

- Can show many graphs together, easy for comparisons
- Basically we can have a lot of variables in a 2-D graph

Heat maps

- Can show quantities in the plane that we are visualizing

Tree maps

- We can see quantities with sizes and dividing them points based on certain characteristics

Geo-visualization

- We can aggregate data from different geographical point of view, we can use normal maps, bubbles and plot quantities of them. Cartograms are useful for quantities on and densities.

Linked Views

- We can put different vies of the dataset together. We can select different subsets to compare them

1.6 Preprocessing

Today data preparation and pre-processing

With data pre-processing and preparation we want to make our data useful for the model. All the datasets are made of signal and noise. We want to maximize the signal and reduce as much as possible the noise.

Real data usually are:

- Incomplete
- Noisy
- Inconsistent

Preparation:

1. Missing values
2. Outliers
3. Discretization and encoding
4. Imbalanced datasets

Pre processing

1. Feature selection →Relevance analysis and redundancy removale
2. Feature engineering
3. Feature scaling and normalization

Treating missing data

Missing values are values that are not available, very common.
How to deal:

- Delete records with missing data, biased
- Delete columns with too many missing data, loose information
- Manually insert them
- Imputing with central tendency metrics, for general and for subsets
- Derive them in an objective way
- Use a predictive model, like KNN
- Use similarity measures

Usually our go to is to use the quickest and simplest option, after we analyze the performance of the model. If the error is significantly higher on the subset with imputed values than on the one with non imputed we look for other options.

Today treatment of missing data. Again.

Outliers →Data-points that is distant from the other observations. It can be from variability, experimental errors or extreme cases based on different causes. They can come from

1. Unusual but correct situations
2. Incorrecto measurements
3. Errors in data collection
4. Lack of code for missing data

Outliers can have an effect on our training, **Leverage effect**.
To deal with outliers:

- Detect them
    1. Automatic limitation using a threshold
    2. If we have a standard distribution we do 3σ + and - Average
    3. Boxplots
    4. Scatterplots for 2-D cases
    5. Multi-D outliers →KNN distance, isolation forest, cluster methods, self.organizing maps, dimensionality reduction
- Treatment

1. Capping/Winsoring
2. Transformation methods →Log distribution

Discretization →We pass from continuous values to discrete ones.

Basically we bin our data, we can do supervised and unsupervised discretization.

- Equal-width binning, unsupervised
- Equal-depth binning, unsupervised

Choose the number of bins

- Square root rule
- Rice rule
- Business common sense

For supervised discretization

- Entropy

Encoding →To be able to work with categorical variables we need to transform them into numerical ones.

- 1-Hot encoding
- Binary encoding
- Ordinal encoding

Imbalanced Learning →if our dataset is imbalanced, we have much more instances for a certain class than for another.

In an imbalanced dataset usually we have **majority** and **minority** classes. We use the **IR** imbalance ratio to consider it. We can't use standard learning ethods, they will introduce a bias in favor of the majority class during training (the minority class will contribute less to the maximisation of the objective function). Accuracy in this setting becomes almost useless. The problems arise when most algorithms assume a balanced class distribution and uniformity of misclassification costs.

The usual solutions are

- Undersampling
- Oversampling
- Hybrid approaches

One of the most used solution is **SMOTE** (Synthetic Minority Oversampling TEchnique).

1. Randomly selecting a minority class instance
2. Define the set of k-nearest neighbors
3. Randomly select another minority class sample in that KNN set
4. Generate new samples using linear Interpolation

It's important to remember that we should always use real data instead of synthetic ones. The generated ones should be as realsitica as possible and they will always introduce some noise.

Now we start with **Data Preprocessing**, we want to reduce the noise and increase the signal. The idea is to reduce the size of the dataset in terms of the number of features. We want to beat the curse of dimensionality since in high dimensions distance metrics will lose meaning.

To decide what to keep and what to remove we check two principles

- Redundancy →features that give us the same information about the dataset

- Irrelevance →do not affect the target

To reduce the input space we can

- Feature engineering →Derive new features from the existent ones
- Heuristic feature selection
- Correlation matrices
- PCA

Since we care about distances we should also aim at having standardized data. The main ways are

- Normalization
- Standardization

1.7 Segmentation

Cluster analysis is basically classify different objects in a finite and smaller subspace, basically binning with extra-steps. We want to use their differences and similarities to group or separate them. The objective is to minimize the intraclusters distances while maximising the distances between clusters.

The difference from classification is that we do not know to which cluster each example is part of, we want to derive those clusters and analyze them.
In clustering we will have a solution, we define a fixed number of clusters. It's different from A Priori grouping since in the latter we define only the rules.

An example of A Priori grouping is Cohort analysis →group observations based on certain characteristics.

Cell based segmentation is based on the quantiles and we try to segment our observations on those. We can do that using just one variable using the **Migration matrix**.

To analyze our data we can use different approaches. One of them is the **RFM**.

- Recency →Customers that have purchased more recently will purchase Again
- Frequebcy →Customers that have purchased more will buy again
- Monetary →Customers that have spent more will spend again

We can use RFM for customers files that contain purchase history. We have two methods

1. Exact Quintiles
    - Sort the database according to recency and divide into 5 quintiles
    - Do the same for frequency and monetary
    - We have 125 cells of equal size
2. Hard coding
    - We divide the categories into exact values
    - More expensive to program
    - We have many quantities from cell to cell

The popularity of RFM comes from its semplicity and capacity to segment customers.

On the more general case of cluster analysis we need to understand which are the stages of it

1. **Decide the set of variable that we want to use** →To decide which variable we want to use we have to kniw which is the objective of the segmentation. Since the type of problem will determine the variables that we will use. If we want to group objects we want to maximise discrimination ability. The quality of the cluster analysis is stricly based on the variables used. The choice should replicate a theoretical context and use variables that we know will be good discriminators.

2. **Decide the similarity or dissimilarity criterion** →The usual choice is calculating the difference between the objects using metrics based on euclidean spaces. The differences are based on how much instances differe on numerical values. A measure of distance should obey three axioms: simmetry, distances are always positive and that we have the triangular inequality. The usual choice is the euclidean distance, but we also have the Minkowski, the weighted euclidean and correlation.

3. **Decide our clustering algorithm**

4. **Do profilization with the analysis and validation of the resulting solution**

As said before we have to firstly differentiate between A priori grouping and Clustering.

To cluster our data we can have different approaches, based on different algorithms and ideas.

- Hierarchical methods
- Partition methods
- DBScan
- Shift means
- Self-organizing maps

1.8 Clustering techniques

1.8.1 Hierarchical Clustering

With **Hierarchical clustering** we start from a feature matrix and we calculate a dissimilarity matrix to see the differences. This last matrix is based on the distances between the points. To aggregate we can use different aggregation rules like single linkage or complete linkage (connect the 2 closest of farthest points) or average linkage or group linkage. We can obtain also the dendrogram of the data and visualize differences with it. Hierarchical clustering can have many disadvantages

- We can't go back after an operation
- Even though is not computational costly we can't correct wrong decisions
- Doesn't works really well with large datasets

To improve the performance of Hierarchical methods we can analize the links after each partition (CURE or chamaleon) or integrate iterative optimization (BIRCH).

1.8.2 Partition methods

With **Partition methods** (like k-means algorithm) we try to divide the data into k (decided at the start) different clusters. We will find for each cluster a centroid. Each group must contain at least one point and each point must belong to exactly one cluster. The algorithm works like this

1. We create a random partition based on k
2. We relocate iteratively moving objects between groups
3. We decide if the partitioning is good based on the similarities between groups

More specifically

- Choose the seed
- Associate each individual to a seed
- Calculate the centroids
- Go back to 2
- End when we don't move anymore the centroids

With k-means we want to minimize intra-group variance (the SSE, sum of squared errors)

$$\text{SSE} = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2 \tag{2}$$

We can reduce the SSE by increasing k but good clustering with small k can have a lower SSE than a poor one with higher k.

K-means is simple, efficient and linear but is sensitive to outliers, the initial random position and not good enough for non spherical clusters. One improvement to the problem if initialization is to use K-means++ that is made via a 4 step initialization process.

1. Choose the first centroid randomly
2. Calculate the distance from the centroids for each point
3. Choose the next centroid probabilistically
4. Repeat untill we get k centroids

This way can find better centroids and converges faster.

1.8.3 DBScan

Density Based Clustering is a clustering algorithm meant to find outliers. The previously seen methods are based on several assumptions that if they are not true will give us suboptimal results. Here with DBSCAN we don't assume any specific shape for our clusters, basically we want to find regions with an high desity of points. We use mainly two parameters:

1. $\epsilon \rightarrow$radius around a point
2. MinPts $\rightarrow$minimum number of points required to form a dense region

Steps

1. Pick a point
2. Check the neighborhood to find core and border points
3. Expand the cluster to find if the neighbors of a core point are also core point. We apply this step to all found core points.
4. Stop when we can't expand our cluster

The choice of the hyperparameters is based most on case to case

A different approach is HDBSCAN where we can try different values. It adapts at different levels of density both wit ϵ and MinPts. Basically we want to explore with different density levels and we build a dendrogram. The parameters are min_cluster_size and min_samples. The steps are

- Transform distances
- Build a graph and hierarchy
- Condense and select clusters
- Output

1.8.4 Shift mean

Mean shift is another algorithm that is density based and mode seeking. It treats the data as a sample from an unknown probability density and looks for the local maxima (nodes) of this density, which will become the cluser centers. It uses a silding window (kernel) of radius h that is moved to the mean of the points inside the windows looking for the highest density. The windows are moved untill they converge, if they are really near they are put together inside the same cluster.

Algorithm:

- Create the kernel
- At every iteration move the window towards the most dense region within itself
- Repeat untill the movements are negligible
- Do this for every points untill we have clear saparations

The main characteristics are that all the data points will converge to the same cluster every time and that we have an attraction basin (a region where all the starting points will converge to a certain cluster).

Strengths

- Can identify different shapes
- Can leave outliers as noise
- We dont't have to choose the number of clusters
- Good for small and medium datasets
- Versatile even though it can struggle in high dimensions

Weaknesses

- Struggles with varying densities
- Sensitive to radius choice
- The radius is critical and difficult to set
- Expensive for large datasets
- Suffers from the curse of dimensionality

1.8.5 Self-organizing maps

With SOMs we want to train a unsupervised neural network where the inputs are connected to a matrix of neurons. Each neuron is a vector in the input space that during the training is pulled in the position of the input data dragging with them their neighbours in the output space. The map aims to pass between the data patterns. Input patterns are then compared with the neurons and the closest is considered the winner. The winner is updated and so its neighbors. The difference between data and neurons is called quantization error.

Algorithm

- **Step 0:** Randomly initialize the weights w_{ij}.
- Set the neighborhood topological parameters.
- Set the learning rate α.
- **Step 1:** While stop condition is false:
  - **Step 2:** For each input vector $\mathbf{x}$:
    - **Step 3:** For each unit j, compute

$$D(j) = \sum_i (w_{ij} - x_i)^2$$

    - **Step 4:** Find the unit that minimizes $D(j)$ (Best Matching Unit).
    - **Step 5:** For each unit j in the neighborhood and for all i:

$$w_{ij}^{new} = w_{ij}^{old} + \alpha \left(x_i - w_{ij}^{old} \right)$$

  - **Step 6:** Update the learning rate α.
  - **Step 7:** Reduce the radius of the topological neighborhood.

From our SOM we get as output

- U-matrices
- Component planes
- Hit plots

1.9 Data Clustering

Another problem is understanding how many clusters we want. One way is to create many cases with different K and choose the best. The choice should be done via

1. Elbow curve method to lower intra cluster variability, we use a plot of the objective function value and the number of clusters
2. Silhouette analysis, how similar are the points inside the clusters
3. Evaluation of the profile of the cluster
4. Operational considerations, based on business environment

As said the shape of the clusters can be tricky for k-means, since is based on round shaped clusters. We can solve this by using k-medoids.

Another method to evaluate the clustering quality is the DBI. It focus on how compact the clusters are internally and how separated are from each other. We aim at lower values. DBI is made by 3 key components: Scatter S_i (average distance for the points in the cluster from the centroid), Separation M_{ij} (distance between the centroids of 2 clusters) and similarity ratio R_{ij} (how similar two clusters are, the lower the better).
To calculate the DBI

1. Compute the centroid of each cluster
2. Calculate S_i for each cluster
3. Calculate M_{ij} for each pair of clusters
4. Compute R_{ij} for each cluster pair and select the maximum R_{ij}.
5. DBI is given by the average of the R_{ij}

We want to use elbow, Silhouette and DBI together and maximise them as much as possible to find the optimal k. If they do not converge we need to choose based on the domain knowledge and Interpretability of our analysis.
It's very easy to interpret the elbow curve since we want to minimize the intra-cluster variability. The other 2 provides us for additional information (Silhouette how similar each point is to it's own cluster (higher is better) and DBI the average similarity ratio of each cluster with it's most similar one (lower is betetr))
The evaluation of the cluster profile is subjective dependent on the analyst.
Operational considerations are based on the business, usually we want a k small enough for developing a specific strategy, a number if individuals large enough to be worth it to develop a specific startegy. Usually we can have an high initial k and grouping the clusters

Other important characteristics of the clusters are the shape and the density of the points. As said k-means can have problems with clusters of different size and density. Also in this setting we are assuming that either an individual is part or not of a cluster, yes or no.
One variant is the k-medoids algorithm where each cluster is represented by one of the (real) points located near the center of the cluster.

1.10 Association rules

Another way to find patterns is the use of association rules, we want to extract compact patterns that describe subsets of data. Based on events that occur together to find rules on the form if X then Y. We get information about things that happen toegther.

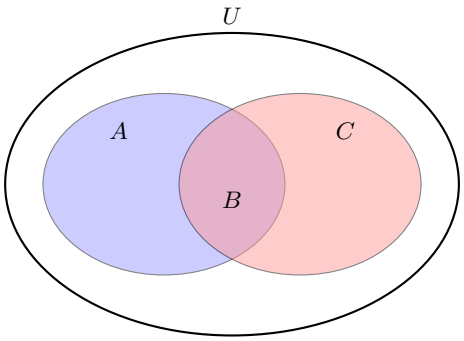
We can start with a table of purchased items to create a table of co-occurences with the number of times that any pair of products were purchased together.
Usually association rules are expressed in the form of

"If the item A is part of an event, then the item B will also be part of the event X percent of time"

The rules are not to be considered as causation but as association. All rules have an antecedent and a consequent →if a customer buys shoes, then 10% of the time will also buy socks.

To evaluate rules we usually use

- Confidence →How strong is the implication, the conditional probability that C occurs given the occurence of A
- Consequent support →Consequent transactions over the total number of transactions
- Support →How frequently the combination occurs in the database
- Lift →how much more often A and B occur together than would be expected if they were statistically independent, how strong the correlation is beyond random chance



- Confidence $\rightarrow \frac{B}{A} = P(C \cap A)$
- Consequent support $\rightarrow \frac{C}{U}$
- Support $\rightarrow \frac{B}{U} = P(A \cap C)$
- Lift $\rightarrow \frac{\text{Confidence}}{\text{Consequent support}} = \frac{P(C \cap A)}{P(C)}$

How to read

1. A lift of 2 means that two events will occur together twice as often as would be expected if their occurrences were independent
2. A lift greater than 1 means a positive association
3. A lift smaller than 1 means a negative association
4. A lift equal to 1 means that that the two events are uncorrelated
5. A credible rule should have good confidence, high support and a lift higher than 1
6. High confidence and low support must be taken with caution

Apriori algorithm for association rules

1. Frequent itemset generation

2. Create candidate itemsets
3. Pruning infrequent itemsets
4. Repeat the process
5. Association rule generation
6. Pruning rules

Types of rules

- Trivial
- Inexplicable
- Actionable
- Rules obtained by promotions

We can also find product hierarchies. Remember that we need a lot of data about products and the customers.

1.11 Dimensionality reduction

Dimensionality reduction is the process of reducing the number of dimensions (features) in a dataset while retaining as much meaningful information as possible. It is used for visualization, computational efficiency and noise reduction. High-dimensional data often suffer from the curse of dimensionality.

Dimensionality reduction can be achieved through:

- **Feature selection** → selecting a subset of relevant features
- **Feature extraction** → transforming data into a lower-dimensional representation

The goal is to preserve the structure of the original dataset when projecting it into a lower-dimensional space. Linear methods like PCA aim to preserve variance, while non-linear methods like t-SNE and UMAP focus on preserving neighborhood relations or the manifold structure.

High-dimensional data often lie on or near a lower-dimensional manifold. This motivates dimensionality reduction.

To recap:

- Data redundancy
- Low intrinsic dimensionality
- Noise in high dimensions
- Manifold hypothesis

Dimensionality reduction can be obtained with:

- Linear methods → PCA
- Non-linear methods → t-SNE or UMAP

PCA (Principal Component Analysis) is a linear method that identifies orthogonal directions (principal components) that maximize the projected variance. We keep those that explain most of the total variance.

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a non-linear technique mainly used for visualization of high-dimensional clusters. It preserves local neighborhoods rather than global distances.

UMAP (Uniform Manifold Approximation and Projection) is also non-linear. It attempts to preserve both local and some global structure based on manifold and topological assumptions.

Dimensionality reduction is widely used in text mining to obtain dense vector representations. Classical representations include one-hot encoding, frequency vectors and TF-IDF (related to Zipf's law). More advanced embeddings include Word2Vec (CBOW or Skip-gram), which produce semantic clusters in vector space.

1.11.1 PCA

Principal Component Analysis aims at using an orthogonal transformation to convert a set of correlated variables into linearly uncorrelated principal components.

If we have k features, PCA can produce up to k components (kernel PCA as well: it cannot exceed the rank of the data, which is $\min(n, k)$).

Since PCA components are linear combinations of the original variables, PCA captures only linear relationships.

PCA steps:

1. Standardize the data
2. Compute the covariance matrix
3. Perform eigendecomposition
4. Sort eigenvectors by descending eigenvalues
5. Select the principal components
6. Create the projection matrix
7. Transform the data

1.11.2 t-SNE

t-SNE uses probabilistic similarity measures to map high-dimensional data to a lower-dimensional space (typically 2D or 3D). It preserves local neighborhood structure.

- Build conditional probabilities $p_{j|i}$ over pairs of points in high-dimensional space
- Build similar probabilities q_{ij} in low dimension using a Student-t distribution
- Minimize the KL divergence between the two distributions

$$D_{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right)$$

The Student-t distribution in low dimension helps alleviate the crowding problem.

The main parameter is **perplexity**, which controls the effective number of neighbors considered (typically between 5 and 50). It strongly influences the visualization.

1.11.3 UMAP

UMAP is a non-linear dimensionality reduction technique used for visualization and manifold learning, based on three assumptions:

- Data is uniformly distributed on a manifold
- The manifold is locally connected
- The Riemannian metric is locally constant

With these assumptions, UMAP constructs a fuzzy topological representation of the data. The low-dimensional embedding is obtained by optimizing a layout that preserves this fuzzy structure.

- Build the fuzzy representation of the manifold
- Optimize the low-dimensional embedding

The objective function is:

$$L_{UMAP} = \sum_{i,j} [p_{ij} \log(q_{ij}) + (1 - p_{ij}) \log(1 - q_{ij})]$$

The first term represents **attractive forces**, the second **repulsive forces**.
Important parameters:

- Number of nearest neighbors
- Minimum distance in the embedding

2 Programming

2.1 Introduction to Python and computers

Printing →A string is returned and shown on the screen

```
    print("Hello World")
```

print() syntax →print(object(s), separator=separator, end=end, file=file, flush=flush)

Errors →They happen when there is a problem in the code

- **Runtime Errors** →Something wrong, the code won't run
- **Semantic Errors** →The output is not what we expected

We have many more of them, to solve we have to **debug the code**.

Using Jupyter notebooks we can use some special commands that are preceded by the % character

- **%timeit** →determines the execution time of a single-line statement. Performs multiple runs to achieve robust results.
- **%%timeit** →same as %timeit but for the entire cell
- **%time**→determines the execution time of a single-line statement. Performs a single run!
- **%%time**→same as %time but for the entire cell
- **%run**→Run the named file inside IPython as a program
- **%history**→displays the command history. Use %history -n to display last n-commands with line numbers.
- **%recall<line_no>**→re-executes command at line_no. You can also specify range of line numbers
- **%who**→Shows list of all variables defined within the current notebook
- **%Ismagic**→Shows a list of magic commands
- **%magic**→Quick and simple list of all available magic functions with detailed descriptions
- **%quickref**→List of common magic commands and their descriptions

And many more

2.2 Semantics, datatypes, flow controls and structures

Semantics →in Python tabs and spaces are used to structure the code. If you use a colon you have to indent the code in the right way. Semicolons instead can be used for multiple statements on the same line

Objects →everything in Python is an object with it's own methods, functions and characteristics

Comments →# denotes comments

To represent information we can use different kind of data types and structures

Variables →a variable can take whatever value we want

```
    a = 2 #variable declaration
    print(a)
>>> 2
    type(a)
>>>int
```

Other possible types are

- float
- string
- complex
- boolean

If we want to save a collection of objects we can use

```
List = [1, 3, "a", 7]
Tuple = (1, "a", 8, 7)
Dictionary = {"a":1, "b":4}
Set = {1, "a", 4, 8}
```

Data Structure	Mutable / Immutable	Ordered / Unordered	Indexed / History
List	Mutable	Ordered	Indexed
Dictionary	Mutable	Unordered	History of addition
Tuple	Immutable	Ordered	Indexed
Set	Mutable	Unordered	No indexing (unique elements)

Table 2: Comparison of Python collection data structures.

Every data structure has it's own methods.
When we are using indexed data structures we have to remember that the first index is 0.

Since we want to work with data and modify them we need a way to do it. Operators can take variables and make computations if those are compatible

Operator	Name	Example
`+`	Addition	`x + y`
`-`	Subtraction	`x - y`
`*`	Multiplication	`x * y`
`/`	Division	`x / y`
`%`	Modulus	`x % y`
`**`	Exponentiation	`x ** y`
`//`	Floor division	`x // y`

Table 3: Python arithmetic operators with names and examples.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

Table 4: Python assignment operators with examples.

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Table 5: Python comparison operators.

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverses the result, returns False if the result is True	not(x < 5 and x < 10)

Table 6: Python logical operators.

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

Table 7: Python bitwise operators.

Operator	Description
`is`	Returns True if both variables are the same object
`is not`	Returns True if both variables are not the same object

Table 8: Python identity operators.

Flow control →to check that our program is doing what we want we can apply different statements.
if →we define a condition under which a certain action is done if that condition is reached

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Table 9: Python comparison operators with examples for conditional statements.

elif →it means else if, we specify a new condition
else →to be put as last, if neither if or elif are met we use else

```
if name == "Libero":
    print("Hello")
elif name == "Jose":
    print("Forza Milan")
else:
    print("None")
```

Loops →for now we just cared about one time operations, we can do those on multiple times with loops. We have two main loops

- **for loops** →we specify an interval and the action is performed that number of times.
- **while loops** →the operation is made as long as a certain condition is true

```
    for x in range (10):
        print(x)

    a = 0
    while a < 25:
        print(a)
        a +=1
```

Some problems can arise if the condition of the while loop is never met.

Comprehension →for faster performances we can use this particular structure. It works for all the mutable data structure

```
    squares = [x**2 for x in range(10)]
    print(squares)

    even_squares = [x**2 for x in range(10) if x % 2 == 0]
    print(even_squares)


    squares_dict = {x: x**2 for x in range(5)}
    print(squares_dict)

    squares_set = {x**2 for x in range(5)}
    print(squares_set)
```

With this general structure

```
    {key_expression: value_expression for item in iterable if condition}

    {expression for item in iterable if condition}
```

Slices →if we want a subset of the data structure.

General structure

```
    list[start:stop:step]
```

Negative indexes mean to start from the end

typecast →we can transform a variable with a certain structure into another with another structure

```
    # Typecasting examples in one snippet

# String to int
x_str = "10"
x_int = int(x_str)
print(x_int, type(x_int))  # 10 <class 'int'>

# String to float
y_str = "3.14"
y_float = float(y_str)
print(y_float, type(y_float))  # 3.14 <class 'float'>

# Int to string
z_int = 100
z_str = str(z_int)
print(z_str, type(z_str))  # '100' <class 'str'>

# Tuple to list
tup = (1, 2, 3)
lst = list(tup)
print(lst, type(lst))  # [1, 2, 3] <class 'list'>

# List to tuple
lst2 = [4, 5, 6]
tup2 = tuple(lst2)
print(tup2, type(tup2))  # (4, 5, 6) <class 'tuple'>

# Int to float
num = 7
num_float = float(num)
print(num_float, type(num_float))  # 7.0 <class 'float'>

# Float to int
num2 = 9.8
num2_int = int(num2)  # truncates decimal part
print(num2_int, type(num2_int))  # 9 <class 'int'>
```

2.3 Functions, imports and namespace

Functions →like a mathematical function a python one will take some inputs and will apply the same operations to get a certain result. We define them to avoid writing the same code over and over.

```
    def function(input): #to define
        input operation
        return output

    function(other_input) #to call
```

A function can also work with the elements of a list or other collections of elements using a for loop.

If we want to have a flexible number of inputs we can use *args and **kwargs

- ***args** →the function will take a list as input and will do the computations on each element of that list
- ****kwargs** →same as args but with a list of keys and values, the function will return a dictionary

```
    def multiply(*args):
        output = 1
        for n in args:
            output *= n
        return output

    print(multiply(2, 3, 4))
```

```
    def my_function(**kwargs):
        for key, value in kwargs.items():
            print(key, value)

    my_function(name="Alice", age=25, city="Paris")

    ###Output
name Alice
age 25
city Paris
```

Link for a video if it's still confusing

You can add infos about your function with the ___doc___

```
    def greets(*args):
        """This function says Hello"""
        for name in args:
            print("Hello", name)
    greet.__doc__
```

If we want a fast singular use function we can use **lambda functions**.

```
    lambda arguments: expression

    add = lambda x, y : x + y
    add(2, 3)
```

Map →we want to transform certain values into others in a fast way

```
    list(map(lambda x : x * 2, [1, 2, 3, 4]))
```

Filter →if we want to select a subset of our data

```
    from random import sample

    X = sample(range(-25, 25), 25)
    f = filter(lambda x: x > 0, X)
```

When we want to call a certain function multiple times we can use two main ways:

- **Iteration** →We define a number of times and the function will do its job for that number.
- **Recursion** →The function will call itself until it reaches a base case. It's important to define base, edge and general cases. Useful when we work with trees or graphs

```
    def factorial(n):
    if n == 1:
        return 1    #base case
    elif n == 0:
        return 1    #edge case
    else:
        return n * factorial (n-1) #general case
```

We should differentiate between:

- **Functions** →defined by def or lambda, they can be applied to almost everything if well defined.
- **Methods** →associated with certain objects
- **Attributes** →variables associated to certain objects

Remember kids, if you want to code something probably someone has already done it. So why bother? We can import the work done by other people

```
    import module as mod
    from module import method1, method2
```

sys.path will give us all the directories that our interpreter is watching
Useful modules can be:

- csv
- datetime
- io
- json
- math
- os
- random
- sqlite3
- xml
- zipfile
- zlib

Namespace A namespace is a collection of names that reference objects.
- **Built-in Names**: predefined names available in every Python interpreter. Examples: `list`, `dict`, `map`, `tuple`.

```
import builtins
print(dir(builtins))   # list built-in names
```

- **Global Names**: user-defined names created in the main program body (variables, functions, classes).

```
x = 42   # global variable
def foo():
    return x
print(globals())   # list global names
```

- **Local Names**: names defined inside a function, valid only inside it.

```
def bar():
    y = 10  # local variable
    print(locals())   # list local names
bar()
# print(y)  -> Error: y does not exist in the global scope
```

Scope Scope defines where a variable can be accessed.
- **Global Scope**: variable accessible throughout the whole program. - **Local Scope**: variable accessible only inside the function where it was declared.

```
animal = "dog"  # global variable

def test_scope():
    # local variable with the same name
    animal = "cat"
    print("Local:", animal)

test_scope()
print("Global:", animal)
```

Output:

```
Local: cat
Global: dog
```

Using the keyword global It allows modifying a global variable inside a function.

```
count = 0

def increment():
    global count
    count += 1

increment()
print(count)  # 1
```

2.4 Pandas, Series, Dataframes and selection

Today we start with Pandas, fav library for data.

```
    import pandas a pd #always import as pd
```

Series →a Pandas series is 1-D object that can hold any data type. Is made of 2 arrays, one for the index and the other one with the actual data.

```
    obj = pd.Series([11, 28,72, , 5, 8])

    obj.array #information on the array
    obj.index #information on the indices
```

The left column is the index one, always. We can use custom index lists

```
    fruits = ['apples', 'oranges', 'cherries', 'pears']
    quantities = [20, 33, 52, 10]
    S = pd.Series(quantities, index=fruits)
    #the index list is fruits

    S2 = pd.Series([17, 13, 31, 32], index=fruits)
    print(S + S2) #will sum the elements on the same position
    sum(S) #will tell us the total of the nubers in the array

    fruits2 = ['raspberries', 'oranges', 'cherries', 'pears']
    S2 = pd.Series([17, 13, 31, 32], index=fruits2)
    print(S + S2) #the operations are aligned by index, elements that don't appear in
        both lists will be NaN, like a Joint in relational algebra

```

```
    print(S["apples]) #we can access like a dictionary, output will be 20
    S["apples"] = 25 #to modify the element

    print(S + 2) #to add 2 to every element
    print(np.sin(S)) #to apply the sin function

    S.apply(lambda x: x if x > 25 else -1 * x) #apply will make a function to all the
        elements of the series, Map() will work element wise

    print(S[S > 25]) #to filter using booleans

    "apples" in S #to see if a key exists

    cities = {"London": 8615246,
    "Sesto San Giovanni": 79121,
    "Montevideo": 1405798}
    city_series = pd.Series(cities) #from dictionary to series

    print(cities.isnull()) #to see which element is NaN
    print(cities.notnull()) #to see which element is not Nan
    print(cities.fillna(0)) #to change Nan to 0.0 (floats)
    print(cities.fillna(0).astype(int)) #to change Nan to 0 (integers)

    obj.name = "Libero" #to give our series a name
    obj.index.name = "diocane" #to give the index comlumn a name
    obj.index #we see a list of the index column
```

Characteristics of Index object

- Immutable
- Behaves like a fixed size set (we can check if a certain index is in the column with the in method)
- Can contain duplicate labels

We have several useful methods for the index objects.

If we put multiple series one after the other we get a matrix called **DataFrame** which can be considered like an Excel spreadsheet. Like Excel a pandas dataframe has both row and column index.

```
    pd.concatenate([row1, row2, row3]) #to connect the rows if they share the same
        index
    pd.concatenate([row1, row2, row3], axis=1) #they become columns, default is 0

    df.colum.values #retrieve the columns names
    df = pd.Dataframe(df, index= a_list) #to rename the index column with the names
        of a list
    df = pd.Dataframe(df,
    columns=("name2", "name3", "name1") #to rearrange the columns order, same as df.
        reindex(["name3", "name1", "name2"])

    df.rename(columns={
    "name1" = "newname1",
    "name2" = "newname"
    }, inplace=True) #to rename columns, inplace False will return a copy of the
        dataset

    df = pd.Dataframe(df, columns=["name1", "name2"], index=df["other_column"]) #to
        put other_column as new index

    df.loc("label", "other_label") #select all the rows with that label
    df.loc(df.condition > number) #for conditions

    df.sum() #sum on all the columns
    df["column1"].sum() #sum on a single column
    df["column1"].cumsum() #cumulative sum

```

```
    #to add columns we put the new column name in the column list and we impute it
        with df["new_col"] = df["old_col].cumsum() for example. Default values for
        new columns are NaN

    df["attribute"] #access a column same as df.attribute()

    df["attribute"] = number #will fill the column with that number, for unique
        values use a list of the same size

    df.T #to get the transpose

    df = pd.read_csv("path/to/csv/file")

    df.head(n) #to see the first n lines of the dataframe

    df.to_csv("path/where/you/want/to/save/the/file")

    df.loc[] #access with label based approach, KeyError if it can't find the value
    df.iloc[] #access with index, IndexError if the requested index is out of bounds,
         except for slice indexers

    df["b": "m"] #slices can also work by labels, we will take also the middle values
```

How to select subsets

- Fetching like a dictionary, select the index that you want
- Slice by index
- Slice by labels
- Fetch multiple entries
- Condition based selection

You should use .loc[] and by index

iloc vs loc

- iloc = by label
- loc = by index
- duloc = lord Farquad city

```
    df.loc["label"] #select a row by index label
    df.loc["label", ["col1", "col3"]] #row and column selection
    df.loc[:"label", ["col1", "col3"]] #as before but with slicing

    df.iloc[[2, 1]] #fetch rows with that position
    df.iloc[[1, 2], [3, 0, 1]] select certain rows and columns
    df.iloc[:, :3] #with slices
```

2.5 Methods to modify views and data

To see statistical characteristics of our dataset we can do

```
    df.describe() #the arguments are percentiles, include and exclude
```

To see how many null values and the object in the column

```
    df.info() #the arguments are verbose, buf, max_cols, memory_usage, null_counts
```

To see the number of unique values

```
    df["name_col"].value_counts() #the arguments are normalize, sort, bins, dropna
```

To group or divide kind of data, basically binning

```
df["col1"] = df["col1"].map(lambda x: operation on x)
df.groupby("col1") ["col2"].value_counts() #with parameters by, group_keys, dropn
    , as_index
```

If we want to make a multi dimensional visualization of group by we can use pivot

```


pd.pivot(index="col1", columns="col2", values="col3") #with arguments data, values,
    index, columns, aggfunc
```

To aggregate df using a relational algebra join

```
df3 = pd.merge(df1, df2) #with arguments right, how, on, left_on, right_on
```

To make queries (SQL style)

```
df.head() #to see first 5 rows
df.query("price < 150").head() #with argument inplace.

#we can apply and, or and so on, basically SQL queries inside a string
```

For 1-Hot encoding

```
pd.get_dummies(df["key"], prefix="key", dtype=int) #to join with another df that
    has those columns we use
df = df[["data"]].join(dummies)
```

Now example of a full work with Pandas, not gonna write it.

2.6 Numpy

We use more efficient data objects than pandas. It works with multidimensional arrays. They are very pandas DF like. The arrays look like lists but more ordered for our computer sake. This is babsed on locality of reference.

Numpy arrays have a fixed size and the type is homogeneous (not if they have arrays of objects).

To create an array and work with them

```
a = np.array([1, 2, 3])
print(type(a))
print(a.shape)
a[0] = 5

b = np.array([[1, 2, 3], [4, 5, 6]])
b[0, 0] #to access matrix like

X = X.astype("int 64") #to change type
```

The numpy dtypes are many. Some of them have limits on the size that they can use

```
X = np.random.randint(1, 101, size=(3, 4)) #random array creation

a = np.linspace(0, 10, 3) #list with elements between the first 2 evenly spaced,
    the last is how many elements we want

a = np.arange(0, 10, 2)

np.zeros((3, 3)) #3x3 matrix of zeroes
```

To access and slicing arrays we can use the same rules seen in Pandas

```
data[1]
data[0:2]
data[1:]
```

```
    data[-2:]


    matrix[2] #third row
    matrix[0][2] #single element
    matrix[0, 2] #also works

    matrix [0:2, 0:2]

    a[a < 5] #true or false depending on the comparison
    a[a % 2 == 0] #same as above

    & and | for and or or

    data[data < 0] = 0 #assign values

    a.reshape((4, 2)) #reshape the array
```

Some operations will return a view, a shallow copy, or a copy, deep copy. If y is a view of x if we change x then y will change. Like using

```
    y = x[1:3]
```

if y.base returns the original array then y is a view of that. If returns none it own the data. When you assign an array to another variable using =, you're not creating a copy or a view. You're just creating a new reference (alias) to the same array. Both variables point to the same data, so changes made through one will appear in the other. y is not a new object, it just points to the same data as x. Their IDs are the same. Changing y[0] to 10 also changes x[0], because they're the same array under different names. You can use the .copy() method to force a deep copy and create a different object.

Vectorization describes the absence of any explicit looping, indexing, etc., in the code - these things are taking place, of course, just "behind the scenes" in optimized, pre-compiled C code. Vectorized code has many advantages.

A universal function (or ufunc for short) is a function that operates on ndarrays in an element-by-element fashion, supporting array broadcasting, type casting, and several other standard features. That is, a ufunc is a "vectorized" wrapper for a function that takes a xed number of speci c inputs and produces a xed number of speci c outputs. In NumPy, universal functions are instances of the numpy.ufunc class. Many of the built-in functions are implemented in compiled C code. The basic ufuncs operate on scalars, but there is also a generalized kind for which the basic elements are sub-arrays (vectors, matrices, etc.), and broadcasting is done over other dimensions. One can also produce custom ufunc instances using the frompyfunc factory function.

- arange
- sqrt
- exp
- sin
- standard_normal(8)
- maximum
- add

np.eye(10) will create a 10x10 identity matrix, np.full((10, 10), 0.5) will create a matrix filled by 0.5.

Operations like sum, mean, max are optimized with much faster than the traditional Python approach of looping through elements. These are aggregation operations.

np.dot makes a matrix multiplication. np.T the transpose, also np.transpose().

Broadcasting governs how operations work between arrays of different shapes. It can be a powerful feature, but it can cause confusion, even for experienced users. The simplest example of broadcasting occurs when combining a scalar value with an array. Like array * 5.

The broadcasting rule: Two arrays are compatible for broadcasting if for each trailing dimension (i.e., starting from the end) the axis lengths match or if either of the lengths is 1. Broadcasting is then performed over the missing or length 1 dimensions.

The numpy.random module supplements the built-in Python random module with functions for ef ciently generating whole arrays of sample values from many kinds of probability distributions. For example, you can get a 4 x 4 array of samples from the standard normal distribution using numpy.random.standard_normal. These random numbers are not truly random (rather, pseudorandom) but instead are generated by a con gurable random number generator that determines deterministically what values are created.

Linear algebra operations, like matrix multiplication, decompositions, determinants, and other square matrix math, are an important part of many array libraries. Multiplying two two-dimensional arrays with * is an element- wise product, while matrix multiplications require either using the dot function or the @ in x operator. dot is both an array method and a function in the numpy namespace for doing matrix multiplication.

SciPy is a collection of mathematical algorithms and convenience functions built on NumPy . It adds signi cant power to Python by providing the user with high-level commands and classes for manipulating and visualizing data. SciPy is organized into subpackages covering different scientific computing domains.

3 Machine Learning

3.1 Introduction

Machine Learning differs from traditional programming, we want to model a function to predict data or patterns. Machine Learning has many applications in multiple fields. Since we are using data is important to notice that those can have biases like people. Some biases related to people can be confirmation, falalcy of centrality or survivorship.

ML as said has many applications, some can be:

1. Image and object recognition
2. Videogames
3. Sound generation and recognition
4. Art and style imitation
5. Predictions

Many algorithms exist for those tasks but all of them are made from three main components:

1. **Representation** →Space of models
2. **Evaluation** →How to assess which model is better for a certain task
3. **Optimization** →How to improve the model

Optimizing a model means finding the best parameters and hyperparameters, we can do it via:

- Combinatorial optimization
- Convex optimization
- Constrained optimization

Different tasks require different kind of learning:

- **Supervised/Inductive** →We have the labels of the training data
- **Unsupervised** →We don't have the labels in the training data
- **Semi-supervised** →Mix of the previous two
- **Reinforcement** →We want to maximize a reward function after performing some actions

More specifically we will study the first one

Supervised →We have many examples of data, we want to find the function that describes best their distribution. We can do classification, regression or estimating probabilities. The core idea is that we want to fit a curve that discriminates between classes or that fit at best a scatterplot. Since we are estimating we will have for sure a certain degree of errors, that is called the **bias**. Bias can also be positive if is used in a smart way, if we have knowledge about something we can use it to avoid useless steps.

As we know in the 1-D case a math function is on the form of

$$y = f(x) \tag{3}$$

And if we open $f(x)$

$$f(x) = ax + b \tag{4}$$

This is the most basic linear case. We want to estimate the parameters a and b to fit the data in the best way. The core idea is to reduce as much uncertainty as possible.

3.2 Machine Learning tribes

Machine learning implies that the machine will learn something, there are different approaches for that. They can be applied to different tasks for better performances.

1. **Symbolism** →Fill gaps in existing knowledge
2. **Connectionism** →Emulate the brain
3. **Evolutionary Computation** →Simulate evolution
4. **Statistical Learning** →Reduce uncertainty
5. **Analogy Modelling** →Check for similarities between old and new

Tribe	**Origins**	**Master Algorithm**
Symbolists	Logic, Philosophy	Inverse Deduction
Connectionists	Neuroscience	Backpropagation
Evolutionaries	Evolutionary Biology	Genetic Programming
Bayesians	Statistics, Probability	Inference
Analogizers	Psychology	Support Vector Machines (SVM)

Symbolists:

- Logic programming
- Expert systems
- Decision trees
- Functional programming

Connectionists:

- Perceptron
- MLP
- DNN
- Hopfield networks
- Boltzmann networks

Evolutionary:

- Genetic algorithms
- Genetic programming
- Ant Colony optimization
- Particle Swarm optimization

Bayesians:

- Bayesian classifiers
- Bayesian Belief networks
- Markov models

Analogizers:

- KNN
- Self-organization
- SVM

If we cross together the tribes and the components seen in the previous class we can see that:

- **Representation**
  - Probabilistic Logic
  - Each rule has a weight
- **Evaluation**
  - Posterior probability
  - User defined objective function
- **Optimization**
  - Genetic programming
  - Backpropagation

3.3 Feature Selection

The most important part of a machine learning model is the dataset, sometimes the data aren't good enough on their own so we must account for that. Some ways are

- **Feature extraction and engineering** →extract something from simpler features
- **Feature transformation** →modify the features to make them more useful for the model
- **Feature selection** →use the most relevant features

In this course we will see mainly the third point. When we do **Feature selection** we aim to reduce the number of inputs variables to a subset that can be equally useful. Simpler models are easier to interpret, the training time is reduced, generalization is enhanced and we remove redundant variables.

Feature selection is done via 3 main methods:

- **Filter methods** →Statistical methods to evaluate the importance of the features, like correlation
- **Wrapper methods** →We train a model with subsets of features until we find the best one
- **Embedded methods** →During the training of our model the importance of the features is assessed.

Filter methods:

- Pearson correlation coefficient
- Spearman rank coefficient
- ANOVA correlation coefficient
- Kendall rank coefficient
- Chi-Squared test
- Mutual information

Pearson Correlation coefficient, it measures linear correlation:

$$\rho_{X,Y} = \frac{\text{cov(X,Y)}}{\sigma_X \sigma_Y}$$

For m samples

$$r = \frac{\sum_{i=1}^{m}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{m}(x_i - \bar{x})^2}\ \sqrt{\sum_{i=1}^{m}(y_i - \bar{y})^2}}$$

Spearman rank correlation coefficient, it asses how well the relationship between two variables can be described by a monotonic function

$$r_s = \rho_{rg_x, rg_y} = \frac{cov(rg_x, rg_y}{\sigma_{rg_x}\sigma_{rg_y}}$$

We shouldn't always discard variables with a small score. It depends by case to case

We can also use **Mutual information**, this can help finding non-linear dependencies but is harder to estimate than pure correlation.

Mutual information (continuous and discrete), for non linear and non monotonic relationships

$$I(X_i; y) = \int \int p(x_i, y) \log \frac{p(x_i, y)}{p(x_i)\, p(y)}\, dx\, dy$$

$$I(X_i; Y) = \sum_{x_i} \sum_{y} P(X = x_i, Y = y) \log \frac{P(X = x_i, Y = y)}{P(X = x_i)\, P(Y = y)}$$

The problem is that mutual information can be inconvenient for feature ranking. To solve the associated problems we can use the **Maximal information coefficient** that transforms the previous and binds it in [0;1]. We obtain the percent of a variable Y that can be explained by a variable x

$$\text{MIC}(X, Y) = \max\{\frac{I(x, y)}{\log_2 \min\{n_x, n_y\}}\}$$

Wrapper methods
We consider certain subsets of our features, at each iteration we try to find the best combination of those. We can vary which and how many features to consider. The selection process involves evaluate the subsets using a predictive model and using the subset with the best performance.

The search process can use different strategies:

- Methodical
- Stochastic
- Heuristics

One of the most used is RFE

1. The model is fitted on all the predictors
2. Each predictor is ranked using it's importance to the model
3. We choose the n top ranked predictors
4. We find the best number n with the best n features

RFE can encounter problems if it overfit on uninformative features.

Our training set is used for selecting predictors, fitting the model and evaluating the performance.
To improve RFE we can add **Cross-validation**

Embedded methods
This last kind of methods consists in regularizing the model while it's been created. The process consists in adding constraints into the optimization and adding a bias toward lower complexity, so reducing the number of coefficients

We have two main regularization algorithms:

1. Ridge, or L2
2. Lasso or L1

Ridge Regression tries to minimize the sum of squared residuals and the slope of the line. This can cause many lower coefficients and can solve for parameters where we do not enough data samples.

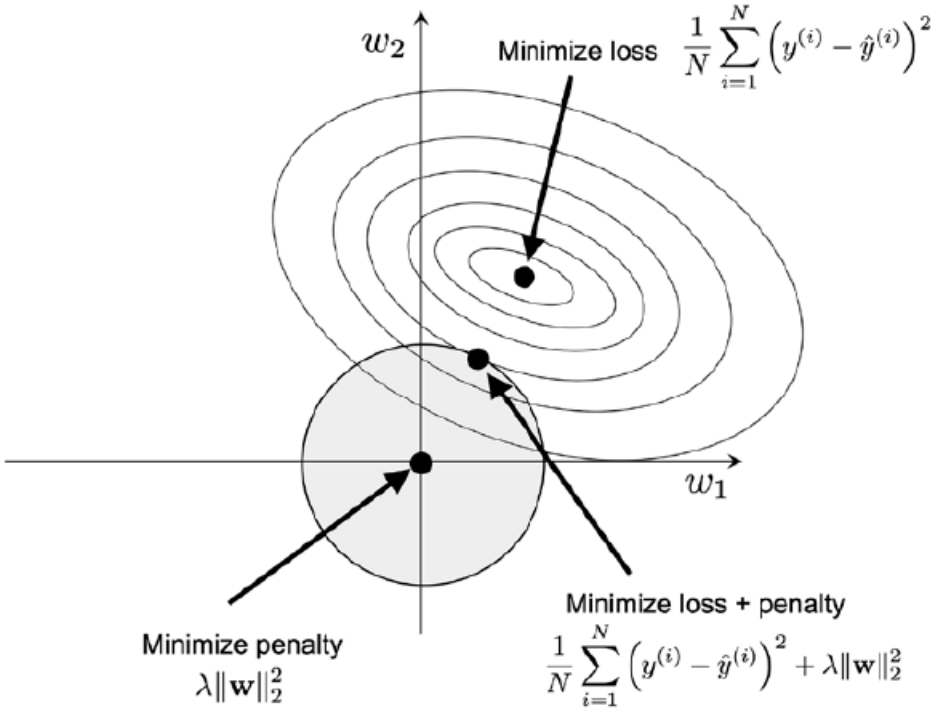


Figure 4.6: Applying L2 regularization to the loss function

Lasso Regression on the other hand tries to minimize the magnitudes. It can lead to many zero values in our coefficients set

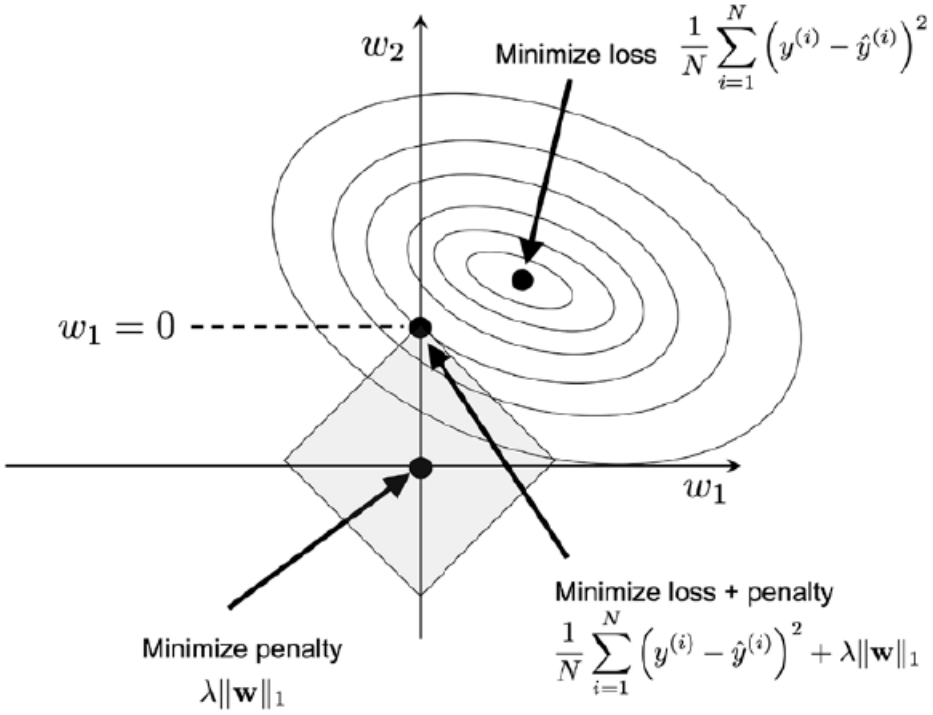


Figure 4.7: Applying L1 regularization to the loss function

3.4 Linear and Logistic Regression

Today we look at our first algorithms, we start with **Linear Regression**.

We want to find a relationship between independent variables and the dependent one. The objective is to make predictions based on the past observations.

As we know one kind of task for ML models is regression, we want to predict a continuous values. One way is using Linear Regression.

The steps are:

1. Use least-squares to fit a line on the data
2. Calculate the R^2
3. Calculate the p-value for R^2

A regression model can be simple or multiple

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n + \varepsilon \tag{5}$$

Where:

- Y →Model/output
- β_0 →Intercept
- β_n →Parameters
- X_n →Inputs
- ε →Residuals

For the simple model we only use the first 2 β and the first X

We want to minimize the squared sum of the residuals, this will give us a good model. Since this is a minimization problem on both the parameters we differentiate them getting the following formulas (notes that those are estimations)

$$\hat{\beta_1} = \frac{\Sigma(X_i - \bar{X})(Y_i - \bar{Y})}{\Sigma(X_i - \bar{X})^2} \tag{6}$$

$$\hat{\beta_0} = \hat{Y} - \hat{\beta_1}\bar{X} \tag{7}$$

As said these are estimations, so we want to evaluate how good they are.

Usually we use R^2, it explains how much variation in the output can be explained differently from using just the mean.

$$R^2 = \frac{SS(mean) - SS(fit)}{SS(mean)} \tag{8}$$

Where

- SS(mean) or SST →$\Sigma(y_i - \bar{y})^2$
- SS(fit) or SSE = →$\Sigma(y_i - \hat{y_i})^2$

We can also do

$$R^2 = \frac{\text{Var(mean) - Var(fit)}}{\text{Var(mean)}} \tag{9}$$

Where

- Var(mean) = $\frac{\Sigma(y_i - \bar{y})^2}{n} = \frac{SS(mean)}{n}$
- Var(fit) = $\frac{\Sigma(y_i - \hat{y}_i)^2}{n} = \frac{SS(fit)}{n}$

Since bigger models with more features will explain more variability we can use the adjusted R^2, it will penalize models with a lot of independent variables

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1} \tag{10}$$

Where

- R^2 →Current R^2
- N →Samples
- p →Number of independent variables

After that we can check the p-value for each independent variable to check it's statistical relevance, low p-value means high probability that the variable is useful.

Another model that we have to look at is **Logistic Regression**, despite the name is a classifier and not a regressor. We want to make a binary classification (1 or 0) based on our data. Differently from the Linear regression loss function (SSE) here we use MLE (Maximum Likelihood Estimation) to maximize the probability of being right.

Usually our decision is like this

$$\begin{cases} 1 & \text{if True} \\ 0 & \text{if False} \end{cases} \tag{11}$$

In such cases we can't easily (or we straight up can't) linearly separate those examples, the we need a non linear discriminator with an output between 0 and 1. We want to model the probabilities of being of either class. The function used is a Sigmoid.

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}} \tag{12}$$

As said we use MLE to estimate the parameters of the function. Basically we maximise the probability of observing the sample.

Since we want to get a probability between 0 and 1 we have to follow some passages.

- We start from the probability $p = P(Y = 1 \mid X)$.
- We convert it into **odds**: $\frac{p}{1-p}$, which range from 0 to $+\infty$.
- We take the **log** of the odds (logit): $\log\left(\frac{p}{1-p}\right)$, which can take any real value $(-\infty, +\infty)$.
- We model the log-odds with a **linear function of the inputs**:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

- We estimate the parameters $\beta_0, \beta_1, \ldots, \beta_k$ using **Maximum Likelihood**: each observed y_i contributes a probability

  $$p_i^{y_i}(1 - p_i)^{1-y_i}$$

  and we choose the parameters that maximize the likelihood of all observations.
- Once the parameters are learned, we plug the linear combination back and convert it to a probability:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k)}}$$

- This final expression is the **sigmoid (logistic) function**, which always returns a value between 0 and 1.

To estimate the parameters and how to interpret them:

- Parameters are chosen to maximize the likelihood of observing the sample.
- Unlike linear regression, there are no closed-form formulas.
- We maximize the **log-likelihood** numerically.
- ML estimates are obtained iteratively until the log-likelihood stabilizes.

- Predicted log-odds:
$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{1i} + \cdots + \hat{\beta}_k x_{ki}$$
- Estimated probability:
$$\hat{P}(Y_i = 1|X_i) = \frac{e^{\hat{y}_i}}{1 + e^{\hat{y}_i}}$$
- Sign of $\hat{\beta}_k$ indicates direction of curve:
  - $\hat{\beta}_k > 0$: probability increases with x_k
  - $\hat{\beta}_k < 0$: probability decreases with x_k
- Rate of change increases with magnitude of $\hat{\beta}_k$
- Example: $\hat{malign}_i = -7.7836 + 0.6683 \cdot thickness + 0.5540 \cdot size + 0.6807 \cdot shape$
  - thickness=5, size=2, shape=2 $\Rightarrow \hat{P} \approx 0.12$
  - thickness=10, size=8, shape=9 $\Rightarrow \hat{P} \approx 0.999$

3.5 Model Selection and Assessment

Today we see model selection with Carina Albuquerque (breaking bad mentioned).

We want to find the best model from a set of possible ones. Either comparing different models or different hyperparameters and the same model.

We want to reduce overfitting, emphasize understanding vs memorizing the data.

This step comes right before the Assessment, so how we check the effective quality of the model. The main idea of model selection is finding the model that generalizes better on unseen data. Based on the no free lunch theorem we know that we can't have a ML model that is perfect for every taks. Then we need to accept some trade-offs.

Since models rely on training data to learn we can encounter the risk that they will memorize those data, this is the case of **overfitting**. The other possibility is the one of **underfitting**, where the model is too rigid and won't learn enough.

To avoid overfitting we can use different strategies. One of them is splitting the training data into Train and Validation sets. The latter will be used to choose the optimal hyperparameters. The idea is to minimize the validation error as much as possible. Without a validation set we won't be able to have a good idea on the generalization on unseen data.

The usual methods employed are

1. **Hold-Out** →We take that validation set and we test the model on it
2. **K-fold Cross-Validation** →We repeat the hold-out k times on different validation sets, stratified version for imbalanced datasets. After the cross-validation we check the performance on the test set. K is usualy equal to 10.
3. **Repeated stratified Cross-Validation** →Further reduces variance
4. **Leave-One-Out Cross-Validation** →The cardinality of the validation set is 1

To compare and choose between two models

1. Choose a metric
2. Choose an evaluation method
3. Run the mathod for each model
4. Compare the metrics
5. Pick the model with the best performance

The best model usually is the simplest, the less prone to error and the one the has the simplest knowledge representation.

Remember that

- Use the test set and hold out for large data
- Use cross-validation for middle sized data
- Use leave one out for small data
- Use stratified sampling
- Use the validation data for the parameter tuning

We still need to understand how a model is better than another. To do that we use different possible metrics. This process is called **Model Assessment**.

An evaluation metric quantifies the performance of a predictive model. The metric depends on the nature of our task. For now we will focus on classification tasks.

All of our metrics will start from a confusion matrix, a matrix where we plot the predictions made by our model.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Table 10: Confusion Matrix

A confusion matrix can also be made for multiclass classifiers.

From statistics

- **Type 1 error** →False positive
- **Type 2 error** →False negative

To be clear

- True positive →We predict True for a True class
- False positive →We predict True for a False class
- False negative →We predict False for a True class
- True negative →We predict False for a False class

From these we get the following metrics:

- **Accuracy**
  $$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
- **Error Rate**
  $$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN}$$
- **Precision**
  $$\text{Precision} = \frac{TP}{TP + FP}$$
- **Recall (Sensitivity, True Positive Rate)**
  $$\text{Recall} = \frac{TP}{TP + FN}$$

- **Specificity (True Negative Rate)**
  $$\text{Specificity} = \frac{TN}{TN + FP}$$
- **Balanced Accuracy**
  $$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right)$$
- **F1 Score**
  $$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

Remember that **Balanced Accuracy** and **F1 Score** are better for imbalanced datasets.

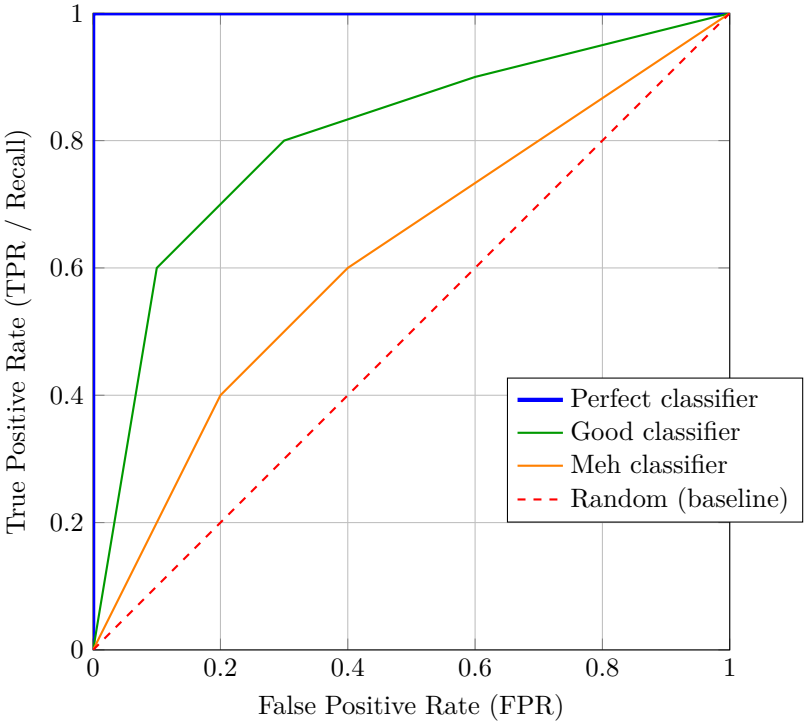
Use them accordingly to the task

- Precision if the cost for False Positive is high
- Recall if the cost for False Negative is high
- F1 for a balance between the previous two
- Accuracy only if the cost for False Positive and False Negative is equal and the dataset is balanced

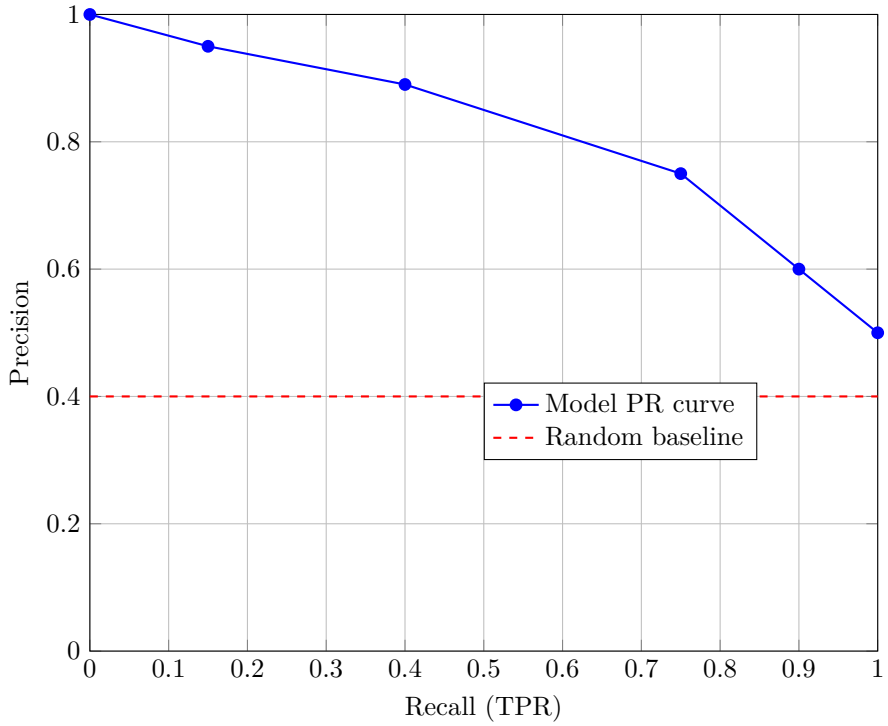
Most binary algorithms work in a 2-step process

1. Compute the probability of being of class 1
2. Compare this probability to the chosen cut-off value and classify

Usually the chosen cut-off is 0.5, probabilities under that will be considered 0.
If we measure the area under the following graph of a **ROC curve** we will get the **AUC** and as near to 1 it is the better the model is.



Another useful curve is the **Precision-Recall Curve**. We can find the best threshold for the F1 maximization.



For regression tasks we can use the following metrics

- **Mean Absolute Error (MAE)**
  $$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$
- **Mean Squared Error (MSE)**
  $$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
- **Coefficient of Determination (R^2)**
  $$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$
- **Adjusted R^2**
  $$R^2_{\text{adj}} = 1 - \left(1 - R^2\right)\frac{n-1}{n-p-1}$$

Remember that

- MSE and RMSE penalize large errors
- If we don't care use MAE
- If we have different numbers of predictors use Adjusted R^2

3.6 Bayesians classifiers

Bayesian classifiers are a class of probabilistic classifers, they give probabilities of being part of a certain class with respect to another. The reasons that we use them are:

- They are statistical
- Based on Bayes theorem
- They have very good performance
- They are incremental

- They set a standard of quality

Recall Bayes theorem

$$P(H|E) = \frac{P(H)P(E|H)}{P(E)} \tag{13}$$

Where:

- H is our hypothesis
- E is an evidence of the hypothesis

We can update both E and H with new knowledge.

If we assume E to be a data sample with attributes and an unknown class label and H to be an hypothesis that E belongs to a class C and we want to determine P(H|E) with the Bayes theorem we get

$$\text{Posterior} = \frac{\text{Likelihood * Prior}}{\text{Evidence}} \tag{14}$$

Basically

- Likelihood →The probability of observing the sample given that the hypothesis is true
- Prior →Probability that a certain event happens regardless of the evidence
- Evidence →Probability that our sample is observed

We can either maximise the likelihood or the posterior

Maximising the posterior means maximising:

$$P(C_i|X) = P(X|C_i)P(C_i) \tag{15}$$

A special case is the Naive Bayes Classifier where we assume that P(X) is equal for all the classes. So the attributes are conditionally independent. To avoid that the probabilities of events without occurence in our sample will be zero we add one occurence to every value class combination (Laplace estimator). In case of missing values the instance is not included in the frequency counts during training, while for classification the attribute will be omitted from calculation

Knowing that the probability density function of a normal distribution is based on the two paramenters mean and standard deviation we can use those information in the sample to estimate the probabilities

Naive Bayes can have good sides

- Easy to implement
- Good results

But also cons

- We lose accuracy assuming class independency
- Dependencies exist in real life

To solve the cons we use **Bayesians Belief Networks**. They allow for class conditional independencies between subsets of variables. Those networks are composed by two components

1. Directed acyclic graph →Shows dependency among the variables and gives specification of joint probability distribution.
2. Set of conditional probabilities

The nodes of the graph represent the variables while the arcs are the dependencies among the variables. The relationship among the variables →Each variable is conditionally independent of its non descendants in the network given the parennts.

3.7 Instance based learning

Instance based classifiers use the simplest form of learning, we just compare previous seen examples and we classifiy the new instance by similarity. We call this approach lazy learning. Some important concepts are feature space and similarity measures. The most basic algorithm is the nearest-neighbor that can be extended to KNN and K-d Trees. The simplest idea is that we store the training records and we use them to predict the class label of unseen cases. If we don't find an exact match we just use the closest point. Since using just one point would have too high of an impact with outliers we dilute the depencdency on individual points and we use the k closest ones.

As said we can use KNN, it requires 3 things:

- The set of stored records
- Distance metric to compute the distance between records
- The number k

With K equal to 1 we can get a Voronoi diagram, we plot a decision boundary.

Another thing that we have to decide is the choice of the distance metric.
The most used are

- **Euclidean:**
$$d_E(x,y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$
- **Manhattan:**
$$d_M(x,y) = \sum_{i=1}^{n}|x_i - y_i|$$
- **Minkowski:**
$$d_p(x,y) = \left(\sum_{i=1}^{n}|x_i - y_i|^p\right)^{\frac{1}{p}}$$

To determine the class we can either take a vote of class labels among the K nearest neighbors or vote according to the distance using the reciprocal squared distance

$$w = \frac{1}{d^2}$$

One variant of the KNN is the distance-Weighted KNN where nearer neighbours will have more weight in the vote

$$f(\mathbf{X}_q) = \frac{\sum_{i=1}^{k} w_i\, f(\mathbf{X}_i)}{\sum_{i=1}^{k} w_i} \text{where} w_i = \frac{1}{d(\mathbf{X}_q, \mathbf{X}_i)^2}$$

With this approach called Stepard method we can use all the training examples.

Another critical aspect is the choice of the optimal k, if its too small we risk overfit and will be too sensitive to noisy points. If k is too large we might not find the true pattern of the data and we risk underfitting. For a balanced dataset our decision boundary will resemble a linear regression, k→n

Since is a scale sensitive algorithm we have to normalize the features to prevent one measure to dominate on the others.
We need to put caution for irrelevant attributes. Also calculating the distances can be computationally expensive. KNN algorith are sensitive to outliers and to the choice of the distance.

To solve the problem of calculating all distances we can create an index of the distances to retrieve the neighbors efficiently. We get a k-d tree. A k-d tree is a balanced binary tree where each node indexes one of the instances of the training set.
The get a K-d tree

1. Make a list of the features in a randomized order
2. Find the meadian of the first feature and split the data
3. Split untill we have less than 2 datapoints in each node.

To find the nearest neighbor, we traverse the tree from the leaves upward. At each step, we calculate the distance between the query point and the current node, then compare it with the distance to the parent node. We always keep the node with the smallest distance. At each branching point, we check which branch gives the smaller distance and continue this process until we reach the root. The nearest neighbor is the node with the smallest overall distance.

K-d trees are useful when we have many more instances than features. We can also adappt them to predict continuous target ussing the average of the nearest neighbors.

Othe similarity measures that can be used are

1. Co-presence
2. Co-absence
3. Presence-absence
4. Absence-presence
5. Russel Rao similarity index →uses only CP
6. Sokal-Michener similarity index →Uses CP and CA
7. Jaccard similarity index →uses all but CA
8. Cosine similarity
9. Mahalanobis distance

3.8 Neural Networks

One new family of algorithm is the ones of the neural networks. They have many applications in many fields and like AI they had periods of summers and winters. The strength of neural networks comes from the fact that they learn automatically, are data driven and can be universal approximators also for non linear problems. Even though they are pretty much two completely different things neural networks are inspired by biological brain structures. The idea is that neurons will fire after they recieve a certain input and so will our perceptrons after their activation fucntion manages to go over a certain threshold.

Neural networks receive some inputs x from other neurons or the environment and will feed an output to a connection with a certain weight that will go to the next neuron. The total input that a neuron receives is the weighted sum of inputs from all the sources. The activation function will make the output from the input.

A neural network usually with one neuron (perceptron) has 4 elements

- Input
- Weights
- Bias
- Activation function

The activation function can be of 3 kind

- Tanh
- Sigmoid
- Threshold model (can be Relu or the identity)

As said the models will learn automatically by adjusting the weights in the network to reach as much generalization as possible. The idea is like trial and error

1. Random weights
2. Small adjustments until we reach a good result

We have two main kind of signals: the function (input to output) and the error (output to input).
After we calculate the error we go back to change a bit the weights every time we iterate with the model.

The model will generate an hyperplane to divide the dataset into the classes (if classification). For linear problems we can use just one perceptron (will converge after enough iterations) while for non linear ones we might need more layers.

With more layers we enter the domain of the **Multilayer Perceptron (MLP)**.

- **Input layer** → Each neuron receives one input from the outside.
- **Hidden layer(s)** → Connect the input layer to the output layer; usually one or two hidden layers are enough.
- **Output layer** → Produces the final prediction.

The training algorithm is called **Backpropagation**. We define a loss function (which measures the error) and we want to minimize it. This is done by computing its gradient with respect to the weights.
The variation of the loss for different weight values defines the **error surface**, while the difference between the network output and the target defines the **error signal**. Delta rule (error term)

For a unit j in the **output layer**:

$$\mathrm{Err}_j = O_j(1 - O_j)\,(T_j - O_j)$$

For a unit j in a **hidden layer**:

$$\mathrm{Err}_j = O_j(1 - O_j)\sum_k \mathrm{Err}_k\, w_{jk}$$

Weight update rule

For a weight w_{ij} connecting neuron i to neuron j:

$$\Delta w_{ij} = \alpha\, \mathrm{Err}_j\, a_i$$

where α is the learning rate and a_i is the activation of neuron i.
The weight update becomes:

$$w_{ij}^{\text{new}} = w_{ij}^{\text{old}} + \Delta w_{ij}$$

or equivalently:

$$w_{ij} = w_{ij}^{\text{old}} + \alpha\, \mathrm{Err}_j\, a_i.$$

3.9 Decision Trees

The class of decision trees can be applied both at classification and estimation. A good advantage is that the trees represent rules, so we can interpret the results very well.

At each level we set a rule and we divide our sample in different branches, we will do that untill we will reach a certain granularity in our results. Our test set data will flow in the tree and we will get a solution based on where they fall. If we want to discriminate between classes we want to have leaves that are as pure as possible, so that they represent only individuals of a particular class.
Each edge will add a conjunction and each new leaf will add a disjunction.

As said decision trees have many advantages

- Interpretability
- Can deal with different types of data
- Don't care about scale of the data
- Automatic definition of the useful attributes for each class
- Can be used for regression

- Non-parametric

But also disadvantages

- Most algorithms will require a discrete target
- Small data variations can result in different trees
- Sub-trees can be duplicated several times
- With many classes we have worse results
- Linear boundaires should be perpendicular to the axis

When we bulid a decision tree we start from our training set and we have to decide

- Which variable to query
- Where to cut
- Which node we should part
- How many edges per node
- When to stop

At each level we can divide an independet variable into different partitions where we want to measure the evaluate the quality of it. We do that continuously untill we don't reach the stopping criterion.

The algorithm is a greedy one

- The tree is built in a top-down recursive divide-and-conquer manner
- All training examples start at root
- If the attributes that are selected are continuous they are discretized
- Examples are partitioned based on the selected attributes
- Test attributes are selected on the basis of a heuristic or statistical measure

On the other hand to stop the partitions

- All samples in a node belongs to the same class
- We can't divide anymore and we use majority voting to classify
- There are no samples left

Also we can have different algorithms

- DDT
- ID3, C4.5, C5
- CART
- CHAID
- And many more

More specifically

DDT

- Greed search
- No backtraking
- Can be stuck in a local minimum
- We start with a dataset o pre-classified individuals and each node specifies an attribute used as a test

- The metric is a discriminative one $f(a) = \frac{1}{n}\sum_{i=1}^{|A|} C_i$ to measure the dominance or purity of the most frequent class

ID3, C4.5

- We select the attribute with the highest information gain
- So we want to minimize the entropy $Entropy(D) = -\sum_{i=1}^{m} p_i log_2(p_i)$
- If we choose the attribute A to split the node with the tuples in D in v partitions the information needed to classify D is $Entropy_A(D) = \sum_{j=i}^{y} \frac{|D_j|}{|D|} \times Entropy(D_j)$
- The information gained by branching on attribute A is $Gain(A) = Entropy(D) - Entropy_A(D)$

CART

- We use the Gini index $gini(D) = 1 - \sum_{j=1}^{n} p_j^2$
- If a data set D is split on A in D1 and D2 the gini index is $gini_{age}(D) = \frac{|D_1|}{|D|} gini(D_1) + \frac{|D_2|}{|D|} gini(D_2)$
- We have a reduction in impurity $\Delta gini(A) = gini(D) - gini_A(D)$
- We choose the attribute that maximises the reduction in impurity

One problem of trees is that they are very prone to overfit on the training data. To solve we can either do

- Prepruning →We will not split a node if the goodness of the measure will go under a threshold
- Postpruning →We remove leaves and branches after the tree is made

Trees can also be used for regression tasks. Every leaf represents a value and we will predict that value (or mean of several values) if our instance is falling in the designated area of our tree.

To find the best splitting point of a tree we can use the MSE and minimize the local split errors. We can also use MAE or sum of residuals. MSE at the leaf level will use the mean value while MAE will go with the median. But the important thing is to determine a stopping point since we can easily overfit if we have one leaf for every possible value.

Since we want to avoid overfit we can do that by removing a leaf, calculate the MSE and do that for everyone. We will stay with the best tree.

This can be problematic since every time we remove a leaf the MSE will grow, then we use the Tree score

$$TressScore = MSE + \alpha T$$

α is an hyperparameter that we find using our validation set, T is the number of leaves.

1. Compute the cost complexity pruning path by creating a fully grown tree and compute the α that minimizes the TreeScore.
2. Split the data for cross validation
3. For each α prune the tree
4. Find the average validation score
5. Select the optimal α
6. Train the final model

3.10 Ensemble Learning

Ensemble Learning means combinining multiple models to achieve better performance than any single one. It can overcome problems like overfitting, being stuck in local optima and expanding the space of representable functions. Based on the idea of "The wisdom of crowds". The google search algorithm is an ensemble.

Theoretical and empirical studies demonstrated that ensembles are tipically more accurate than single classifiers. Though we don't have a single and rigourous definiton of ensembled classifiers. We can see neural networks as ensembles and see that they can be seen as universal approximators with this idea.

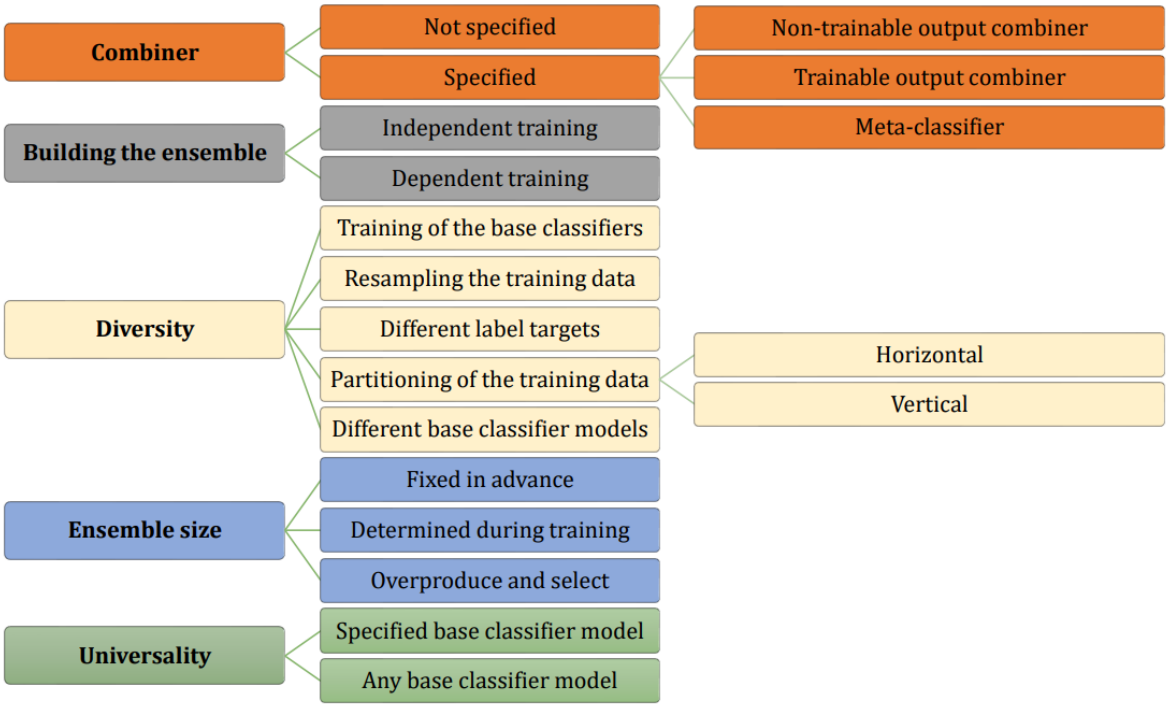
Ensembles work from the Bias-Variance decomposition

$$\text{Expected Prediction Error} = \text{Bias}^2 + Variance + IrreducibleError$$

- Bagging reduces variance
- Boosting reduces bias
- Stacking reduces both

Classifier ensembles work due to

- Statistical reasons →Since we are observing a large searching space we might not find the global optimum so we can average several local ones to get a solid solution
- Computational reasons →Can be given by imperfect training algorithms, too much data, small sample size, divide and conquer solutions and data fusion



How to set up an ensemble

- Combination Level
  - How are the individual inputs combined?
- Classifier Level
  - Do we use same or different classifiers?
  - What base classifier is best: decision tree, NN or another?
  - How many classifiers are needed? Do we overproduce and select?
  - Should the L classifiers be trained together or incrementally?

- Feature Level
  - Shall we use all features or use a bespoke subset for each classifier?
  - How do we select/extract such subsets?
- Data Level
  - How can we manipulate the data submitted for training to the base classifier so as to ensure high diversity and high individual accuracy?

- A combiner can be
  - Nontrainable
  - Trainable
  - Meta classifier
- To build the ensemble
  - The classifiers can be trained independently or in sequence
- How to introduce diversity
  - Use different approaches and parameters
  - Manipulate the training samples
  - Use different target labels
  - Partition the dataset
  - Use different classifier models or hybrid ensembles
- Ensemble size
  - Determine the number of classifiers in the ensemble
- Universality
  - Some approaches can be used with any classifier while others are specifically for a subset of them, like the random forest

Ensemble learning has some benefits

- Improved accuracy
- Less overfitting
- Increased robustness
- Versatility
- Handling of complex problems
- Bias reduction
- Imbalanced data performance

We have two main strategies

- Fusion →each ensemble member knows the whole feature space
- Selection →each member knows a subset of the space in a specialized way

The outputs can be of three levels

1. Abstract →each member output one prediction
2. Ranked →each member ranks the predictions in order of plausability
3. Measurement →each classifer outputs the confidence for each prediction

The final output is usually given by a majority voting choosing the class that has been selected the majority of the times. By Condorcet Jury Theorem as the number of classifiers L $\rightarrow\infty$ the ensemble accuracy P_maj $\rightarrow$1.0.

If we consider that the classifiers have different accuracy level we can use a weighted majority vote with some parameters that will sum to 1.

For regression the final output can be obtained by averaging the predictions of the others.

Specifically of meta-algorithms we have the idea pf ensemble methods, they combine several machine learning techniques in one predictive model to

- Decrease variance $\rightarrow$Bagging
- Decrease bias $\rightarrow$Boosting
- Improve predictions $\rightarrow$Stacking

Bagging $\rightarrow$acronym for Bootstrap AGGregatING, the ensemble is built on bootstrap replicates of the training set and the outputs are combined by plurality vote. We obtain diversity by using different training sets. The base classifier should be unstable, small changes will have to lead to large ones. Usually 25 to 30 decision trees are sufficient. If the classifiers are independent we are guaranteed an improve on the individual performance. The idea of bootstrapping is to randomly drawing with replacement B observations from our dataset and train on them.

The most famous algorithm is the Random forest. Is a very efficient algorithm on large data sets. The randomness is given by choosing a random training set for each base learner and the fact that the splits of the decision trees are chosen randomly.

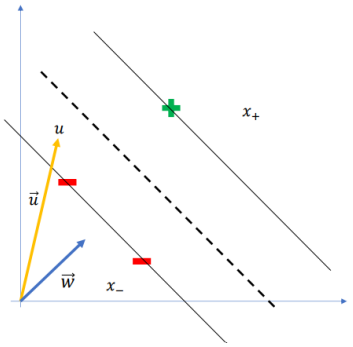
- From the training set, create t bootstrap samples
  - Similar to Bagging, but generally the size of bootstrap samples is the same as the size of the training set
- Attribute selection
  - If there are a total of X attributes in the observations, choose a number $x \ll X$, e.g., $x = \sqrt{X}$
  - This value of x is held constant while growing the forest
  - For the split of each node in each base learner (Decision Tree), randomly select x attributes out of X
  - Choose one attribute out of x to split the node
  - This is one reason Random Forest trees are created faster than regular trees, where each split decision involves all attributes
- Tree growth
  - Each tree is grown to the largest extent possible
  - No pruning, since we desire high variance and low correlation between any two base learners
- Steps summary
  1. Bootstrap data
  2. Randomly select attributes for each node
  3. Build tree until end
  4. Calculate performance from OOB (Out-Of-Bag) observations

3.11 SVM

Support vector machines are a supervised ML algorithm that can be used for classification and regression. Like for KNR we plot the points obtaining the coordinates but we try to create an hyperplane that maximises the difference between the different classes. The border is defined by the points nearest to them, those are our support vectors. The hyperplane is linear.

With previous approach every line that splits the classes are ok, here we aim at find the "best one", where the space between the border points is maximised. This idea can also be called widest street approach.

Imagine a vector $\mathbf{w}$ of any length and perpendicular to the median line (dashed line). Now imagine an unknown point, and a vector pointing to it $\mathbf{u}$. What we would like to know is whether $\mathbf{u}$ is on the left or on the right side of the street.



So, we project the vector $\mathbf{u}$ onto $\mathbf{w}$ (which is perpendicular to the street) and measure whether

$$\mathbf{w} \cdot \mathbf{u} \geq c$$

or equivalently

$$\mathbf{w} \cdot \mathbf{u} + b \geq 0,$$

with $c = -b$.

In this case, the event is classified as positive.

The decision rule of an SVM classifier is:

$$\mathbf{w} \cdot \mathbf{u} + b \geq 0$$

However:

- we do not know b,
- we do not know $\mathbf{w}$,
- there are infinitely many vectors $\mathbf{w}$ perpendicular to the separating hyperplane.

Therefore, we need additional constraints to determine $\mathbf{w}$ and b.

By introducing constraints, we assume:

$$\mathbf{w} \cdot \mathbf{x}^+ + b \geq 1$$

$$\mathbf{w} \cdot \mathbf{x}^- + b \leq -1$$

This imposes a separation from -1 to $+1$ for all known samples. We can now solve the system to find $\mathbf{w}$ and b that guarantee this separation for all training samples.

For convenience, we introduce labels y_i such that:

$$y_i = \begin{cases} +1 & \text{for positive samples} \\ -1 & \text{for negative samples} \end{cases}$$

Multiplying the constraints by y_i, we obtain:

$$y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b\right) \geq 1$$

This can be rewritten as:

$$y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b\right) - 1 \geq 0$$

For samples lying exactly on the margin (support vectors), the constraint becomes:

$$y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b \right) - 1 = 0$$

These constraints define the two boundaries of the street such that the separation between positive and negative examples is as wide as possible.

Consider one positive sample $\mathbf{x}^+$ and one negative sample $\mathbf{x}^-$. Their difference is:

$$\mathbf{x}^+ - \mathbf{x}^-$$

The width of the margin is obtained by projecting this difference onto the unit vector normal to the hyperplane. Since $\mathbf{w}$ is normal to the median line, its unit vector is:

$$\frac{\mathbf{w}}{\|\mathbf{w}\|}$$

Thus, the width of the street is:

$$\text{width} = (\mathbf{x}^+ - \mathbf{x}^-) \cdot \frac{\mathbf{w}}{\|\mathbf{w}\|}$$

From the boundary condition:

$$y_i \left(\mathbf{w} \cdot \mathbf{x}_i + b \right) - 1 = 0$$

we obtain:

$$\mathbf{w} \cdot \mathbf{x}_i = 1 - b \quad \text{for positive samples}$$

$$\mathbf{w} \cdot \mathbf{x}_i = -1 - b \quad \text{for negative samples}$$

Therefore:

$$\text{width} = \frac{\mathbf{x}^+ \cdot \mathbf{w} - \mathbf{x}^- \cdot \mathbf{w}}{\|\mathbf{w}\|} = \frac{(1 - b) - (-1 - b)}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|}$$

To maximize the margin width:

$$\frac{2}{\|\mathbf{w}\|}$$

it is sufficient to minimize $\|\mathbf{w}\|$. For mathematical convenience, this is usually formulated as minimizing:

$$\frac{1}{2}\|\mathbf{w}\|^2$$

From Calcus 2, to minimize a function on a constrained domain we can use the Lagrange multipliers method Assume that L (the Lagrangian) is the function that we want to minimize:

$$L = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i \left[y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 \right]$$

We subtract the sum of all constraints multiplied by constants α_i (one for each constraint).
Finding an extremum implies computing the derivatives and setting them equal to zero.

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i = 0$$

which implies:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$$

Therefore, $\mathbf{w}$ is a linear combination of the vectors in the training set.
The Lagrangian must be differentiated with respect to all variables that may vary, including b:

$$\frac{\partial L}{\partial b} = -\sum_i \alpha_i y_i = 0$$

which implies:

$$\sum_i \alpha_i y_i = 0$$

Recall the expression for the decision vector:

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$$

We now substitute $\mathbf{w}$ into the Lagrangian:

$$L = \frac{1}{2}\left(\sum_i \alpha_i y_i \mathbf{x}_i\right) \cdot \left(\sum_j \alpha_j y_j \mathbf{x}_j\right) - \sum_i \alpha_i y_i \mathbf{x}_i \cdot \left(\sum_j \alpha_j y_j \mathbf{x}_j\right) - \sum_i \alpha_i y_i b + \sum_i \alpha_i$$

Since b is a constant and

$$\sum_i \alpha_i y_i = 0,$$

we have:

$$\sum_i \alpha_i y_i b = b \sum_i \alpha_i y_i = 0$$

Thus, the Lagrangian can be rewritten as:

$$L = \frac{1}{2}\sum_i \alpha_i y_i \mathbf{x}_i \cdot \sum_j \alpha_j y_j \mathbf{x}_j - \sum_i \alpha_i y_i \mathbf{x}_i \cdot \sum_j \alpha_j y_j \mathbf{x}_j + \sum_i \alpha_i$$

which simplifies to:

$$L = \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

We find that the optimization depends only on the dot product of pairs of samples.
Recall the decision rule for a new observation $\mathbf{u}$:

$$\mathbf{w} \cdot \mathbf{u} + b \geq 0$$

Substituting $\mathbf{w}$, we obtain:

$$\left(\sum_i \alpha_i y_i \mathbf{x}_i\right) \cdot \mathbf{u} + b \geq 0$$

If the inequality holds, the sample is classified as positive.
Only points lying on the margin (i.e. the support vectors $\mathbf{x}_i$) have $\alpha_i > 0$.
Therefore, the decision rule depends only on the dot product between training samples and the unknown observation.

It can be shown (proof omitted) that this optimization problem is convex. This implies that there are no local minima.

As a consequence, any reasonable optimization algorithm (including Hill Climbing) can find the global optimum (or a very good approximation of it).

$$\begin{pmatrix} w_1 \\ w_2 \\ b \end{pmatrix} = \begin{pmatrix} -0.4615 \\ -0.3076 \\ -3.6153 \end{pmatrix}$$

$$\begin{pmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \end{pmatrix} = \begin{pmatrix} 0.1538 \\ 0 \\ 0.1538 \end{pmatrix}$$

Consider the vector of Lagrange multipliers:

$$\alpha = \begin{pmatrix} 0.1538 \\ 0 \\ 0.1538 \end{pmatrix}$$

Quick interpretation:

- $\alpha_i > 0$: the point is a **support vector** and contributes to the decision boundary.
- $\alpha_i = 0$: the point does not contribute, it is ignored.

For example, here points 1 and 3 are support vectors, point 2 does not influence the model.

—

The decision parameters are given by:

$$\mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \end{pmatrix} = \begin{pmatrix} -0.4615 \\ -0.3076 \end{pmatrix}, \quad b = -3.6153$$

- w_1, w_2 determine the **slope** of the decision line. - b shifts the line up or down without changing its slope.
The equation of the decision boundary is:

$$w_1 x_1 + w_2 x_2 + b = 0 \quad \text{or equivalently} \quad x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

—
To classify a new point $\mathbf{u} = (u_1, u_2)$:

$$\text{score} = w_1 u_1 + w_2 u_2 + b$$

- score $> 0 \Rightarrow$ positive side - score $< 0 \Rightarrow$ negative side - score $\approx 0 \Rightarrow$ near the margin
—
Summary (at a glance):

- Check α_i to identify support vectors.
- Use $\mathbf{w}$ and b to define the line slope and position.
- Evaluate $w \cdot u + b$ for a new point to see which side it falls on.

The main parameter of SVM is C, we can choose the trade-off between error and margin. C is adjusted by the user. Large C means high penalty to classification errors, the number of missclassified patterns is minimized. With smaller C the points go inside the margin.

If we have inseparable data we can project them into an higher feature space (like from 1 to 2 dimensions) to make them separable. Those functions are called kernel
Some examples are

- Polynomial
- Radial
- Sigmoid

Gamma (γ) is the free parameter of the Gaussian radial basis function (RBF).

- **Small gamma:** The Gaussian has a large variance. This means each support vector influences a **large area** around itself. Even points that are far away from the support vector are affected, making the model smoother and more general.
- **Large gamma:** The Gaussian has a small variance. Each support vector influences only points very close to it. Points farther away are hardly affected, making the model more sensitive to the training points (risking overfitting).

Analogy: Imagine a bell above each support vector:

- Small gamma $\rightarrow$ wide bell $\rightarrow$ many points hear it
- Large gamma $\rightarrow$ narrow bell $\rightarrow$ only nearby points hear it

4 Statistic for data science

4.1 Data exploration and basic concepts

4.1.1 Preliminary concepts

Statistics is the approach of answering questions using data. We aim to find rules that are true for a certain population basing our findings on a smaller subset of the mentioned population (sample)

Unit → Basic objrcts on which the data are collected
Variable → Characteristics of units that can take different values

We can have different types of variables

Categorical

- Nominal
- Ordinal

Quantitative

- Discrete
- Continuous

Our questions involve large **populations**. Since we can't measure all of the units of them we usually take a subset called **sample**. Values that concern the sample are called **statistics** while the ones that concern the whole population are called **parameters**.

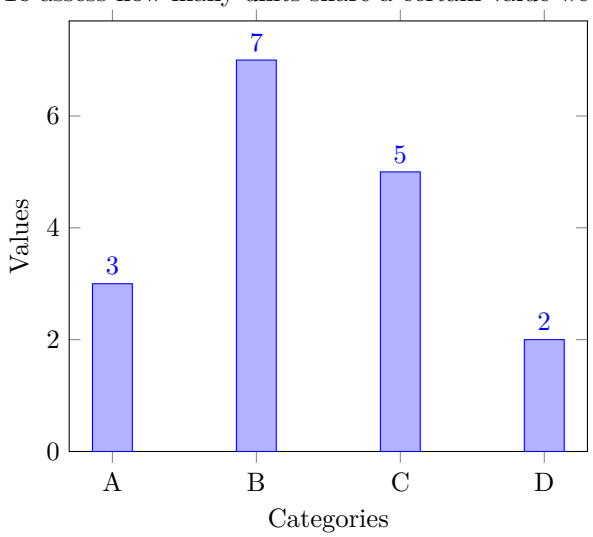
We want to estimate in a precise way those parameters basing the computation on the sample.
One of the problems regarding the samples is that those can be biased. To avoid that we want to get a sample that is representative of the population. One way is the use of **simple random sampling method**.

4.1.2 Describing and visualizing data

We have different ways to describe our data, they depend on multiple factors.
Categorical variables can be described by a proportion

$$p = \frac{\text{Number in the category}}{\text{Total number}}$$

To assess how many units share a certain value we can use a **Bar chart**



One important concept in statistics is central tendency, which refers to finding the center of our data. It also helps us understand how the other values are distributed around that center. Quantitative variables can be summarized using measures of central tendency.

Mean → Sum of the data values divided by the number of the values. For the mean of the sample we use $\bar{x}$ and for the one of the population μ. The formula is

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Median → The center of sample after it was ordered from smallest to largest. Better than mean if we have outliers.

Standard deviation → It describes how the units are distributed from the mean. We can square it to get the variance. For the sample we call it s, for the population we use σ.

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \overline{x})^2}$$

Remember that both $\bar{x}$ and s (or s^2) are relative to the sample. They are just an estimation that might be correct if we have a big enough sample.

Percentile → Is a measure that tells us the percentage of the observations that fall at or below it. At 75% we will find on the left the 75% of the observations and on the right the 25%.

Quartiles → Basically the same as the percentiles but now we are dividing the sample in 4 parts. Q_1 is the first quartile, we will find on the left 25% of the observations and the rest on the right. Q_2 is the median.

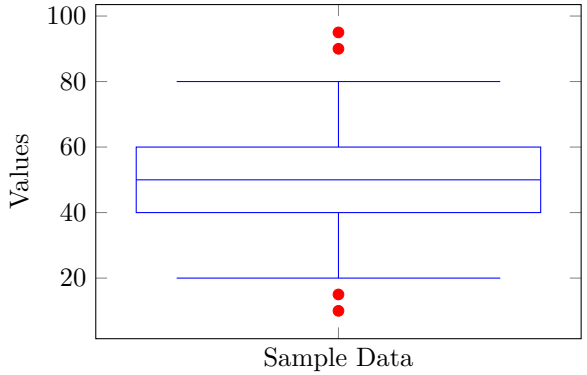
$$Q_1 = 25\text{th percentile}, \quad Q_2 = \text{median (50th percentile)}, \quad Q_3 = 75\text{th percentile}$$

$$\text{Interquartile Range (IQR)} = Q_3 - Q_1$$

$$\text{Lower fence} = Q_1 - 1.5 \cdot \text{IQR}, \quad \text{Upper fence} = Q_3 + 1.5 \cdot \text{IQR}$$

$$\text{Outliers} = \text{values outside [Lower fence, Upper fence]}$$

We can visualize those statistics using a **Boxplot**

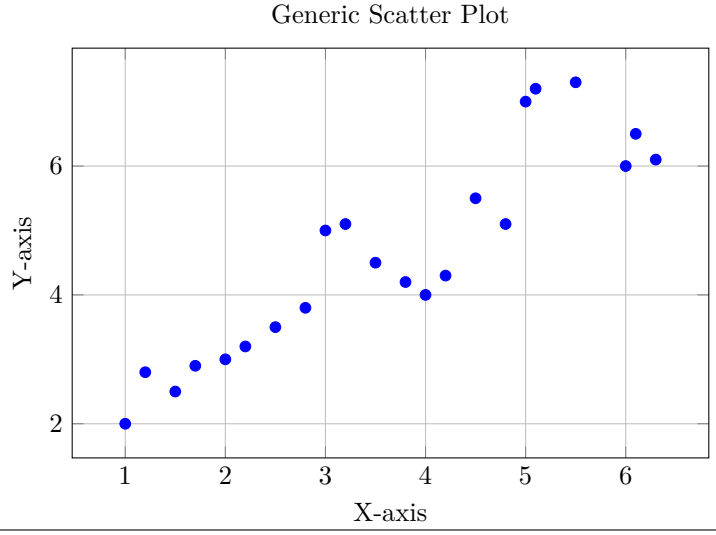


Correlation → Describes the strength and direction between two quantitative variables. For the sample we use r, for the population we use ρ.

Correlation has many properties:

1. ρ is bounded in [-1, 1]
2. $\rho > 0$ means positive association, $\rho < 0$ negative, $\rho = 0$ no relationship
3. The bigger $|\rho|$ the stronger the correlation
4. The sign tells the direction
5. ρ is unit free
6. ρ(X, Y) = ρ(Y, X)

To visualize correlation we use a **Scatterplot**



4.2 Statistical Distributions

4.2.1 Motivations and important concepts

Since random variables can take certain values with certain probabilities we would like to describe them somehow. The collection of those probability is called probability distribution. This can describe the probability of each event happening. Recall that the sum of the probabilities is 1.

CDF -> the **cumulative distribution function** gives me the probability that a random variable is less than or equal to a given value.

$$F(x) = P(X \leq x) \quad \text{for all } x \tag{16}$$

PMF/PDF -> the **probability mass function** or **probability density function** (for discrete or continuous case respectively) will give us the probability of a certain event happening

$$f(x) = P(X = x), \quad \text{for all } x \tag{17}$$

$$F(x) = \int_{-\infty}^{x} f(t)\,dt, \quad \text{for all } x \tag{18}$$

For the continuous case we calculate the integral between the 2 values that we care

4.2.2 Discrete distributions

If we can count the sample space we say that our random variable is discrete

Recall

$$Var[X] = E[X^2] - E[X]^2 \tag{19}$$

Bernoulli -> the probability that a certain event will happen

$$X \sim Ber(p) \tag{20}$$

$$P(X = 1) = p = 1 - P(X = 0) = 1 - q \tag{21}$$

With PMF

$$f(x) = p^x(1-p)^x \quad \text{for x} \in \{0, 1\} \tag{22}$$

Where

- p -> positive probability

- q -> negative probability, 1-p basically

And with

- E[X] = p
- Var[X] = pq = p(1-p)

Binomial -> probability of getting a certain number of getting n independent Bernoulli outcome

$$X \sim Bin(n, p) \tag{23}$$

$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x} \tag{24}$$

Where

- n -> number of trials
- p -> positive probability
- x -> successes that we want to calculate

Recall that

$$\binom{n}{x} = \frac{n!}{x!(n-x)!} \tag{25}$$

And with

- E[X] = np
- Var[X] = np(1-p)

Poisson -> probability of getting a certain number of events during a fixed interval. We need to know the probability of the event happening.

$$X \sim Poi(\lambda) \tag{26}$$

$$f(x) = P(X = x) = \frac{\lambda^x e^{-\lambda}}{x!} \tag{27}$$

Where

- e -> Euler number
- x -> number of successes
- λ -> known mean rate

With

- E[X] = Var[X] = λ

4.2.3 Continuous distributions

Normal distribution -> If we want to describe a series of random events that have a tendency of being near a certain mean.

$$X \sim N(\mu, \sigma^2 \tag{28}$$

$$\phi(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\{-\frac{1}{2\sigma^2}(x-\mu)^2\} \tag{29}$$

- E[X] = μ
- Var[X] = σ^2

It has a bell shape and if the parameter are $\mu = 0$ and $\sigma^2 = 1$ we say that is a standard normal distribution known as Z distribution. σ^2 will tell us how thin or wide is the distribution, so how variable it is, and so how unpredictable.

Kurtosis -> Measure of how heavy the tails are, how frequent are the outliers

Chi squared -> If we have n independent X variables that are distributed with the Z distribution if we square and sum them we say that this quantity follows a Chi-squared distribution

$$V = X_1^2 + X_2^2 + ... + X_n^2 \sim \chi^2_{(n)}, n > 0 \tag{30}$$

With

- E[V] = n
- Var[V] = 2n

The n parameter is called **degrees of freedom**. The shape is asymmetric

Student's t distribution → If we have a random variable Z distributed as a standard normal distribution and another independent variable V following a Chi-squared distribution with n degrees of freedom, then the following ratio defines a random variable that follows a t-distribution:

$$T = \frac{Z}{\sqrt{V/n}} \sim t_{(n)}, \quad n > 0 \tag{31}$$

With

- $E[T] = 0 \quad$ (for $n > 1$)
- $Var[T] = \dfrac{n}{n-2} \quad$ (for $n > 2$)

The parameter n is called **degrees of freedom**. The distribution is symmetric around zero and has heavier tails than the normal distribution. As $n \to \infty$, the t-distribution approaches the standard normal distribution.

Snedecor's F distribution → If we have two independent random variables U and V such that $U \sim \chi^2_{(n_1)}$ and $V \sim \chi^2_{(n_2)}$, then the following ratio defines a random variable that follows an F-distribution:

$$F = \frac{(U/n_1)}{(V/n_2)} \sim F_{(n_1,n_2)}, \quad n_1, n_2 > 0 \tag{32}$$

With

- $E[F] = \dfrac{n_2}{n_2 - 2} \quad$ (for $n_2 > 2$)
- $Var[F] = \dfrac{2n_2^2(n_1 + n_2 - 2)}{n_1(n_2 - 2)^2(n_2 - 4)} \quad$ (for $n_2 > 4$)

The parameters n_1 and n_2 are called **degrees of freedom** of the numerator and denominator, respectively. The distribution is asymmetric and always non-negative. It is mainly used to compare variances, especially in the context of ANOVA (Analysis of Variance) and regression models.

To summarize

Distribution	**PDF**	**CDF**	**Quantile**
Normal	dnorm	pnorm	qnorm
Chi-Squared	dchisq	pchisq	qchisq
Student-t	dt	pt	qt
F	df	pf	qf

4.3 Point estimation, hypothesis testing and confidence intervals

4.3.1 Point estimation

Suppose that we have a population distributed with certain parameters. Like μ as mean or p as proportion. Since we can't check the full population (usually), we start from a sample and we try to see if the estimate can be good enough.

We have two main ways

- Method of moments
- MLE

To start we need to define the **Parameter space**, the range of possible values of the parameter θ inside the parameter space Ω. Let $X_1, X_2, ..., X_n$ be n random variables that can take a value $x_1, x_2, ..., x_n$. The parameter space is bounded.

To estimate the θ we use a point estimator. For example the mean is a point estimator of the μ.

$$\bar{X} = \frac{1}{n}\Sigma X_i \tag{33}$$

We define the statistic T$(X_1, ..., X_n)$ as point estimator to get the parameter estimation. If T$(x_1, ..., x_n)$ we get the estimate.

The first way is the **Method of moments**, we equate sample moments with theoretical moments.

- E(X^k) is the kth theoretical moment of the distribution, k=1,2,3....
- $m_k = \frac{1}{n}\Sigma X_i^k$

We equate the first k sample moments to the corresponding k population moments until we have as many equations as se have parameters, then we solve the resultant system.

The second way is the so called **Maximum Likelihood Estimator**. We start from the assumption that in our sample the values follow a distribution based on an unknown parameter θ. We want to find a good point estimator and estimate for θ using the data from our sample. The idea is that we want to find the parameter that maximise the probability of getting those values. If $X_1, ..., X_2$ are identically independent distributed from a population with PDF f(x $\mid$ $\theta_1, ..., \theta_k$) the likelihood function is defined by

$$L(\theta|x) = L(\theta_1, ..., \theta_k|x_1, ..., x_k) = \Pi f(x_i|\theta_1, ..., \theta_k) \tag{34}$$

After we take the log of the likelihood to obtain the log-likelihood function. This is because the logarithm is a strictly increasing function so maximising the log of the likelihood is like maximising the likelihood itself. The candidates are the values that put the derivative with respectively to θ_i of the log-likelihood at 0. Also is easier to differentiate a log and working with sums

$$logL(\theta|x) = \Sigma log(f(x_i|\theta_1, ..., \theta_k)) \tag{35}$$

$$\frac{d}{d\theta_i} logL(\theta|\mathrm{X}) = 0 \tag{36}$$

We want to assess how good are our estimates. To do it we want to look at the sampling distribution that we receive. Our estimators have some properties.

- **Unbiasedness** →An estimator T is unbiased iff E(T) = θ for every θ in Ω. Usually the population mean is an ubiased estimator.
- **Efficiency** →We have two unbiased estimators T and T' for the same parameter θ. T is more efficient than T' iff

  $$Var(T) \leq Var(T') \tag{37}$$

  T is the most efficient if the equation is true for any other unbiased estimator T'. This is called absolute efficiency, before it was only relative efficiency. To know that we have absolute efficiency we use the Frechet-Cramer-Rao inequality.

Let $(X_1, X_2, \ldots, X_n)$ be a sample from a population with pdf $f(x \mid \theta)$ and let $T = T(X_1, X_2, \ldots, X_n)$ be an unbiased estimator of θ. Then

$$\mathrm{Var}(T) \geq \frac{1}{nI(\theta)}$$

where

$$I(\theta) = \mathrm{E}\left[\left(\frac{\partial \ln f(X \mid \theta)}{\partial \theta}\right)^2\right] = -\mathrm{E}\left[\frac{\partial^2 \ln f(X \mid \theta)}{\partial \theta^2}\right]$$

and $I(\theta)$ is the Fisher information.

This inequality allows us to find a lower limit to compare our variance with.

If we want to compare relative efficiency, we can use the mean squared error (MSE).
Let $T = T(X_1, \ldots, X_n)$ be an unbiased estimator of the parameter θ. The mean squared error is defined as

$$\mathrm{MSE}(T) = \mathrm{E}[(T - \theta)^2]. \tag{38}$$

The estimator with the lower MSE is considered more efficient.

- **Consistency** →Final property, it tells us that if the sample increases the precision of the estimator will also do that. An estimator T_n is said to be consistent in quadratic mean iff

$$lim_{n \to +\infty} \mathrm{E}[(T - \theta)^2] = 0 \tag{39}$$

4.3.2 Hypothesis Testing

Since we are doing inferential statistics we want to make inferences about our parameters. As said we can't check the whole population but we need to consider a sample of it. We use hypothesis testing to see if the value of a certain parameter can be ok with respect to the distribution of our population.

Hypothesis testing is done in 5 main steps

1. Write the hypothesis
2. Calculate the test statistic
3. Determine the p-value
4. Make a decision
5. State a conclusion

Let's delve into this list

The first step is to write the hypothesis, specifically the **Null** and the **Alternative**. The Null is denoted as H_0 while the Alternative as H_1. The Null is always the statement that there is no difference in the population. The Alternative is always that we have differences.

Research question	Is the population mean different from μ_0	Is the population mean greater than μ_0	Is the population mean less than μ_0
Null Hypothesis (H_0)	$\mu = \mu_0$	$\mu = \mu_0$	$\mu = \mu_0$
Alternative Hypothesis (H_1)	$\mu \neq \mu_0$	$\mu > \mu_0$	$\mu < \mu_0$
Type of Hypothesis Test	Two-tailed, non-directional	Right-tailed, directional	Left-tailed, directional

Table 11: Types of hypothesis tests for population mean, mu_0 is the hypothesized population mean

The successive step is to calculate the test statistic. A value that tells us how much different are the values from the Null hypothesis.

For tests on the μ we use the following

$$t = \frac{\bar{X} - \mu_0}{\frac{\hat{\sigma}}{\sqrt{n}}} \sim t(n-1)$$

To estimate the sampling distribution for samples > 30 or with an unknown σ we use a T-Student distribution with n-1 degrees of freedom.

If the standard deviation is known we use the normal distribution.

Now we want to calculate the p-value, or the probability that a population with the specified parameter would produce a sample statistic as extreme or more extreme to the one we observed in our sample. A test is significant if the p-value is less than or equal to the level of significance α. If the p-value is greater we fail to reject the Null, if is less or equal we reject the Null.

To calculate a p-value for a T distribution on a two-tailed test we use

```
2 * (1 - pt(test_statistic, df))
```

Another approach is to use the **Critical region method**. In our case we get the critical region with

$$|t_obs| \geq t_{\frac{1-\alpha}{2};n-k-1}$$

Where the rightmost term is the quantile of the distribution. α is divided by 2 since is a two-tailed test.

If our test statistic is more extreme than the quantile we reject the Null.

To get the quantiles

```
qt(alpha/2, df)
```

We state the conclusion based on the results of the test.

Another possible kind of test is done to the proportions of the population. We want to see if the proportion between different classes is realistic.

Research question	Is the population proportion different from p_0	Is the population proportion greater than p_0	Is the population proportion less than p_0
Null Hypothesis (H_0)	$p = p_0$	$p = p_0$	$p = p_0$
Alternative Hypothesis (H_1)	$p \neq p_0$	$p > p_0$	$p < p_0$
Type of Hypothesis Test	Two-tailed, non-directional	Right-tailed, directional	Left-tailed, directional

Table 12: Types of hypothesis tests for population proportion, p_0 is our hypothesized proportion

Usually we define the Null as p equal to a number

The test statistic is

$$Z_{obs} = \frac{\hat{p} - p_0}{\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}} \sim N(0,1)$$

With critical region

$$Z_{obs} \geq Z_{1-\alpha}$$

Where 1 - α is the percentile of a standard normal distribution

When conducting a test we can make 2 possible decisions, either reject or fail to reject the null hypothesis. In most cases we can't be sure about our inference. If it's correct or not.

When we reject the Null we can make a **Type I** error. Also known as false positive.
When we fail to reject the Null we can make a **Type II** error. Also known as false negative.

The probability of rejecting a false Null hypothesis is known as power.

$$\text{Power} = 1 - \beta$$

Where β is the probability of committing a Type II error. α is the probability of making a type I error.

4.3.3 Inference for two samples

If we want to make **tests on two samples** we can follow the same idea.
For proportions

$$\begin{cases} H_0 : p_A = p_B \\ H_1 : p_A > p_B \end{cases}$$

The test statistic will be

$$Z_{obs} = \frac{\hat{p_A} - \hat{p_B}}{\sqrt{\frac{\hat{p_A}(1-\hat{p_A})}{n_A} + \frac{\hat{p_B}(1-\hat{p_B})}{n_B}}} \sim N(0,1)$$

With critical region

$$Z_{obs} \geq Z_{1-\alpha}$$

For means

$$\begin{cases} H_0 : \mu_A = \mu_B \\ H_1 : \mu_A < \mu_B \end{cases}$$

The test statistic will be

$$t_{\text{obs}} = \frac{\hat{\mu}_A - \hat{\mu}_B}{\sqrt{\frac{\hat{\sigma}_A^2}{n_A} + \frac{\hat{\sigma}_B^2}{n_B}}} \sim t(n_A + n_B - 2)$$

If the samples have the same variance:

$$t_{\text{obs}} = \frac{\hat{\mu}_A - \hat{\mu}_B}{\hat{\sigma}\sqrt{\frac{1}{n_A} + \frac{1}{n_B}}} \sim t(n_A + n_B - 2)$$

With critical region:

$$t_{\text{obs}} \leq t_{\alpha;\, n_A + n_B - 2}$$

Before testing for differences for means we should determine if the variances are equal

$$\begin{cases} H_0 : \sigma_A^2 = \sigma_B^2 \\ H_1 : \sigma_A^2 \neq \sigma_B^2 \end{cases}$$

With test statistic

$$F_{\text{obs}} = \frac{\hat{\sigma}_A^2}{\hat{\sigma}_B^2} \sim F(n_A - 1,\, n_B - 1)$$

With critical region

$$F_{\text{obs}} \geq F_{\alpha;\, n_A - 1,\, n_B - 1}$$

4.3.4 Confidence Intervals

Another way to perform statistical testing is via the use of Confidence Intervals. A CI is a range computed using sample statistic used to estimate a parameter of a population with a certain level of confidence.

The confidence is the difference between 1 and the alpha of the test statistic.

The general form of a confidence interval is

$$\bar{X} \pm t_{1-\frac{\alpha}{2}}(\frac{\hat{\sigma}}{\sqrt{n}})$$

Basically with this form we want to estimate the population mean using the sample mean , the percentile of the t distribution and the standard error.

When we use confidence Intervals we don't know if the parameter will fall inside the interval but we interpret it as we are 95% confident that the population parameter is between X and X'. Hence we get a range of acceptable estimates for the parameter while values that are outside won't be reasonable estimates.

It's possible to notice that if we increase the sample size we will reduce the standard error, then we can get more precise estimates of our parameter.

4.4 Linear regression model

4.4.1 Introduction

Regression analysis is a disciplie that tries to uncover relationships between 2 or more variables. This analysis is based on cross-sectional data, data that we obtain by random sampling from a population. Since we are having a random sampling process the covariance between the observations is zero. So for i≠j

$$Cov(y_i, y_j) = Cov(x_i, x_j) = Cov(y_i, x_j) = 0$$

We want to calculate the causal effect of x on y, so how does y changes if x is changed and all other factors are held constant.

4.4.2 Simple Linear Regression Model

One of the most basic model that we can use is the simple linear regression model. We assume to have two variables y and x. We want to explain y in terms of x, so how y changes with changes in x.

To do that we need to confront some issues

- How do we allow for other factors that can affect y
- What is the functional relationship between y and x
- How to be sure to capture a ceteris paribus relationship between y and x

The model follows this structure

$$y = \beta_0 + \beta_1 x + u$$

In this equation u is the error term, a random disturbance in our population. It represents unobserved factors that aren't on x that can affect y. The equation also addresses the issues of the functional relationship between x and y. Since if Δu=0 then x has a linear effect on y.

$$\Delta y = \beta_1 \Delta x$$

The linearity part means that one unit change on x will have the same effect on y regardless of the initial value of x.

Another assumption is the conditional mean independence one. We say that unobserved factors do not contain information on explanatory variables.

$$E[u|x] = 0$$

Given the previous notes about the simple linear regression model now we want to estimate the parameters.

Starting from

$$E[u \mid x] = 0 \Rightarrow \begin{cases} E[xu] = 0 \\ E[u] = 0 \end{cases} \Leftrightarrow \begin{cases} E\big[x(y - \beta_0 - \beta_1 x)\big] = 0 \\ E\big[y - \beta_0 - \beta_1 x\big] = 0 \end{cases}$$

Assuming that we have a random sample of n individuals and using the method of moments we get

$$\frac{1}{n}\sum_{i=1}^{n} x_i \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i\right) = 0 \tag{2}$$

$$\frac{1}{n}\sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i\right) = 0$$

That we can solve obtaining

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \tag{4}$$

$$\hat{\beta}_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

The estimates for the parameters are called OLS estimates and they are the best possible values for a simple linear regression model.

The error term can be calculated as

$$\hat{u}_i = y_i - \hat{y}_i = y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i$$

Those values are obtained by minimising the SSR by derivative with respect to both the β_i.

Recall

$$\text{SST} = \sum_{i=1}^{n}(y_i - \bar{y})^2 = \text{SSR} + \text{SSE} \rightarrow \text{Total sum of squares}$$

$$\text{SSR} = \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2 \rightarrow \text{Regression sum of squares}$$

$$\text{SSE} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \rightarrow \text{Error sum of squares}$$

With the OLS method we will find the best solution every time. To evaluate it we can use the R^2, it measures the fraction of the sample variation in y that is explained by x.

$$R^2 = \frac{SSE}{SST} = 1 - \frac{SSR}{SST}$$

Sometimes linear dependencies are not enough so we can incorporate many non linearities into our models. Some of the most used are logarithmic transformations.

We can start by defining the interpretations

- Level-Level →No transformation is needed
- Log-Level →From exponential shape to linear
- Log-Log →We have both logarithmic and exponential shapes and we transform them into linear
- Level-Log →From logarithmic shape to linear

Model	Dep. Variable	Regressor	Interpretation of β_1
Level–Level	y	x	$\Delta E[y \mid x_i] = \beta_1 \, \Delta x$
Log–Level	$\log(y)$	x	$\%\Delta E[y \mid x_i] \approx 100 \, \beta_1 \, \Delta x$
Log–Log	$\log(y)$	$\log(x)$	$\%\Delta E[y \mid x_i] \approx \beta_1 \, \%\Delta x$
Level–Log	y	$\log(x)$	$\Delta E[y \mid x_i] \approx \frac{\beta_1}{100} \, \%\Delta x$

4.4.3 Multiple Linear Regression Model

When we use simple regression we have the drawback that is very difficult to draw ceteris paribus conclusions about how x affects y. The assumption that all the other factors affecting y are uncorrelated with x is often unrealistic. A solution is using multiple regression models. With this new setting is easier to fix certain values to get the actual importance of a certain x.

The general model with k explanatory variables has the following shape

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik} + u_i, \qquad \text{assuming } E[u_i \mid x_{i1}, \ldots, x_{ik}] = 0.$$

If we fix $x_2, ..., x_k$ as $\Delta x_j = 0$ for j = 2,...,k we have

$$\Delta \hat{y}_i = \hat{\beta}_i \Delta x_{i1}$$

The residuals are obtained as

$$\hat{u}_i = y_i - \hat{y}_i$$

And the following properties hold

1. $\bar{y} = \bar{\hat{y}}$
2. $\bar{y} = \hat{\beta}_0 + \hat{\beta}_1 \bar{x}_1 + \cdots + \hat{\beta}_k \bar{x}_k$
3. The sample covariance between each x_j, for $j = 1, \ldots, k$, and $\hat{u}_i$ is zero. Hence, the sample covariance between $\hat{y}_i$ and $\hat{u}_i$ is also zero.

The multiple linear model follows certain assumptions

1. **MLR1: Linear in Parameters**
   The model in the population can be written as
   $$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_k x_{ik} + u_i = x_i' \beta + u_i.$$
2. **MLR2: Random Sampling**
   We have a random sample of n observations,
   $$\{(x_{i1}, x_{i2}, \ldots, x_{ik}, y_i)\}_{i=1}^n,$$
   following the population model in Assumption MLR1.
3. **MLR3: No Perfect Collinearity**
   In the sample there are no exact linear relationships among the independent variables. Put otherwise, the matrix X has rank $k + 1$.
4. **MLR4: Zero Conditional Mean**
   Conditional on the entire matrix X, each error u_i has zero mean:
   $$E[u_i \mid X] = 0.$$
5. **MLR5: Homoskedasticity**
   The error u has the same variance given any value of the explanatory variables:
   $$\mathrm{Var}(u \mid X) = \sigma^2 I_n,$$
   where I_n denotes the n-dimensional identity matrix.

Theorem 1 (Unbiasedness of OLS)**.** *Under assumptions MLR1–MLR4, the OLS estimators are unbiased estimators of the population parameters. We have:*

$$E[\hat{\beta} \mid X] = \beta.$$

Theorem 2 (Variances of OLS Estimators)**.** *Under assumptions MLR1–MLR5, the variance–covariance matrix of the OLS estimators is:*

$$\mathrm{Var}(\hat{\beta} \mid X) = \sigma^2 (X'X)^{-1}.$$

Theorem 3 (Gauss–Markov Theorem)**.** *Under assumptions MLR1–MLR5, the OLS estimators are the Best Linear Unbiased Estimators (BLUE) of the regression coefficients. That is,*

$$\mathrm{Var}(\hat{\beta}_j \mid X) \leq \mathrm{Var}(\tilde{\beta}_j \mid X),$$

for any estimator $\tilde{\beta}_j$ that is linear in y and unbiased. In matrix form:

$$\mathrm{Var}(\hat{\beta} \mid X) \leq \mathrm{Var}(\tilde{\beta} \mid X).$$

4.4.4 Introduction to inference

Now that we have defined our assumptions and description of the linear models we want to make inferences on the population parameters. We can do that by constructing confidence intervals or testing possible values. We know that OLS estimators are random variables and we know their expected values and variances. But we want to know their distribution. In order to derive those distributions we need additional assumptions, for example that the errors are normally distributed.

6. **MLR6: Normality**
   We have
   $$\mathbf{u}|\mathbf{X} \sim N(\mathbf{0}, \sigma^2 \mathbf{I})$$

We know that the error term is a sum of many different unobserved factors that are normally distributed. Under normality assumption OLS is BLUE. If we can't assume normality we can use a large sample size.

- Assumptions 1-5 →Gauss-Markov assumptions
- Assumptions 1-6 →Classical assumptions

And

- 1-3 →We can use the OLS
- 1-4 →The estimate is unbiased
- 1-5 →The OLS is BLUE
- 1-6 →We can make inferences

4.4.5 The t-Test

Theorem 4 (Normal Sampling Distribution of OLS)**.** *Under Assumptions MLR1–MLR6 we have*

$$\hat{\beta}_j \mid X \sim N\Big(\beta_j,\ \mathrm{Var}\Big(\hat{\beta}_j \mid X\Big)\Big).$$

Therefore,

$$z = \frac{\hat{\beta}_j - \beta_j}{\sqrt{\mathrm{Var}\Big(\hat{\beta}_j \mid X\Big)}} \sim N(0,1).$$

Theorem 5 (t-distribution for Standardized Estimators)**.** *Under Assumptions MLR1–MLR6 we have*

$$t = \frac{\hat{\beta}_j - \beta_j}{\sqrt{\widehat{\mathrm{Var}}\Big(\hat{\beta}_j \mid X\Big)}} \sim t(n-k-1).$$

If we want to make inferences on β we will have

$$H_0 : \beta_j = a$$
$$H_1 : \beta_j \neq a$$

We have two possibilities

- If $|t_{obs}| > t_{\alpha/2}$ →Reject H_0 at α level
- If $|t_{obs}| \leq t_{\alpha/2}$ →Do not reject H_0

where

$$t_{\text{obs}} = \frac{\hat{\beta}_j - a}{\sqrt{\widehat{\text{Var}}\left(\hat{\beta}_j\right)}}$$

$$t_{\alpha/2}: \quad P(t > t_{\alpha/2}) = \frac{\alpha}{2}$$

Definition: The p-value is the probability of obtaining a test statistic at least as extreme as the one actually observed, assuming that the null hypothesis is true.

- Calculate the p-value for $H_0 : \beta_j = a$:
  - $H_1 : \beta_j \neq a \implies$ p-value $= 2P\big(|t| > |t_{\text{obs}}| \mid H_0 \text{ is true}\big)$
  - $H_1 : \beta_j > a \implies$ p-value $= P\big(t > t_{\text{obs}} \mid H_0 \text{ is true}\big)$
  - $H_1 : \beta_j < a \implies$ p-value $= P\big(t < t_{\text{obs}} \mid H_0 \text{ is true}\big)$
- For example, a p-value of 0.02 indicates little evidence supporting H_0.
- At the 5% significance level, you would reject the null hypothesis.
- **Rejection rule:**
  - p-value $> \alpha \implies$ do not reject H_0 at the $\alpha \times 100\%$ level
  - p-value $\leq \alpha \implies$ reject H_0 at the $\alpha \times 100\%$ level

Very often we want to test whether there is any relation between x_j and the expected value of y, without imposing a sign on the partial effect a priori.

Thus, in most applications, our primary interest lies in testing the null hypothesis

$$H_0 : \beta_j = 0.$$

Here, β_j measures the partial effect of x_j on the expected value of y, after controlling for all other independent variables.

If $\beta_j = 0$, this means that, once the other variables have been accounted for, x_j has no effect on the expected value of y.

Suppose we want to test a single hypothesis involving more than one of the β_j.

The null and alternative hypotheses are:

$$H_0 : \beta_1 - \beta_2 = 0 \quad \text{vs.} \quad H_1 : \beta_1 - \beta_2 < 0$$

Under the null, the corresponding t-statistic is

$$t = \frac{\hat{\beta}_1 - \hat{\beta}_2}{\text{se}(\hat{\beta}_1 - \hat{\beta}_2)}.$$

and

$$\text{se}(\hat{\beta}_1 - \hat{\beta}_2) = \sqrt{\widehat{\text{Var}}(\hat{\beta}_1) + \widehat{\text{Var}}(\hat{\beta}_2) - 2\,\widehat{\text{Cov}}(\hat{\beta}_1, \hat{\beta}_2)}.$$

4.4.6 The F-test

If we want to make inferences on multiple parameters using multiple restrictions we can use the **F-test**.

First we need to consider an unrestricted model

$$y = \beta_0 + \beta_1 x_1 + ... + \beta_k x_k + u$$

We can test if a subset of the independent variables has no partial effect on the dependent

$$H_0 : \beta_{k-q+1} = \beta_{k-q+2} = \beta_k = 0$$

Assuming that it is the last q variables in the list of independent variables, which puts q exclusion restrictions on the model.

Formally we have

$$F = \frac{(\text{SSR}_R - \text{SSR}_{UR})/q}{\text{SSR}_{UR}/(n-k-1)}$$

$$= \frac{(R^2_{UR} - R^2_R)/q}{(1-R^2_{UR})/(n-k-1)} \sim F(q,\, n-k-1)$$

Where

- q = $df_{H_0} - df_{H_1} = df_R - df_{UR}$ (number of restrictions under the null)
- n - k - 1 = df_{UR} (df of the unrestricted model)

Let F_{obs} be the observed test statistic.

$$\text{reject } H_0 \quad \text{if} \quad F_{\text{obs}} > F_\alpha \quad (\text{or if p-value} < \alpha)$$

$$\text{do not reject } H_0 \quad \text{if} \quad F_{\text{obs}} \leq F_\alpha \quad (\text{or if p-value} \geq \alpha)$$

The reasoning is as follows. Under the null hypothesis we have

$$F \sim F(k,\, n-k-1)$$

If we observe $F > F_\alpha$ and H_0 is true, then a low-probability event has occurred.

The p-value is defined as

$$\text{p-value} = P(F > F_{\text{obs}} \mid H_0 \text{ is true}).$$

If the null is

$$H_0 : \beta_1 = \beta_2 = ... = \beta_k = 0$$

We are testint the overall significance of the regression with a restricted model

$$y = \beta_0 + u$$

The F statistic becomes

$$F = \frac{\text{SSE/k}}{\text{SSR/n-k-1}}$$

$$= \frac{R^2/k}{(1-R^2)/(n-k-1)} \sim F_{(k,n-k-1)}$$

4.4.7 Heteroskedasticity

Consider a basic multiple linear regression model and the notion of homoskedasticity, that the variance of the unobserved error is constant

$$Var = (u|\mathbf{X}) = \sigma^2$$

If this is not true than the errors are heteroskedastic

$$Var = (u|\mathbf{X}) = \sigma_i^2$$

The consequences of Heteroskedasticity are

- Gauss-Markov doesn't hold, than OLS is not BLUE
- The estimators of the variances $Var(\hat{\beta}_j|\mathbf{X})$ are biased
- The standard errors of the OLS are no longer valid for constructing confidence intervals and t statistics
- The usual OLS t statistics do not have t distributions
- F statistics are no longer F distributed (and the LM statistic no longer has an asymptotic chi-square distribution)
- In summary the statistics that we used to test hypotheses under Gauss-Markov are not valid in the presence of Heteroskedasticity

However

- OLS estimators are still unbiased and consistent

We want to test for the presence of Heteroskedasticity so

$$H_0 : \text{Var}(u \mid \mathbf{X}) = \sigma^2$$
$$H_1 : \text{Var}(u \mid \mathbf{X}) = \sigma_i^2$$

The idea is that we have Var[u|**X**] = E[u^2|**X**] because E[u|**X**]=0. We want to see if u^2 is related in expected value to one or more of the explanatory variables. We assume a linear function

$$u^2 = \alpha_0 + \alpha_1 x_1 + ... + \alpha_k x_k + error$$

And then we test using F-stat or LM-stat

$$H_0 : \alpha_1 = \alpha_2 = ... = \alpha_k = 0$$

We can use the following tests

- Breusch-Pagan
- White (full)
- White (special)

Breusch-Pagan

1. Estimate the model
$$y_i = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_k x_{ki} + u_i$$
by OLS and obtain the residuals $\hat{u}_i$.

2. Run the auxiliary regression
$$\hat{u}_i^2 = \alpha_0 + \alpha_1 x_{1i} + \cdots + \alpha_k x_{ki} + v_i, \qquad i = 1, \ldots, n,$$
and record the R^2 from this regression, denote it R_u^2.

3. Test
$$H_0 : \ \alpha_1 = \alpha_2 = \cdots = \alpha_k = 0 \quad \text{vs.} \quad H_1 : \ \text{not all } \alpha_j \text{ are zero.}$$
Two equivalent test statistics are commonly used:

- **LM (Lagrange multiplier) test:**

$$\text{LM} = n\, R_u^2 \xrightarrow{d} \chi_k^2 \quad \text{under } H_0.$$

- **F–approximation:**

$$F \;=\; \frac{(R_u^2/k)}{(1-R_u^2)/(n-k-1)} \quad \text{with (approx.) } F_{k,\,n-k-1} \text{ distribution under } H_0.$$

White test

The White (1980) auxiliary regression considers also the squares of the independent variables and all cross products.

- Consider, without any lost of generality, $k = 3$:
  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + u$

The auxiliary regression is:

$$\begin{aligned}\hat{u}^{\,2} = {} & \alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_3 + \\ & \alpha_4 x_1^2 + \alpha_5 x_2^2 + \alpha_6 x_3^2 + \\ & \alpha_7 x_1 x_2 + \alpha_8 x_1 x_3 + \alpha_9 x_2 x_3 + \text{error}\end{aligned}$$

- With only 3 independent variables the auxiliary has 9 independent variables. With 6 the auxiliary regression involves 27 regressors. This abundance of regressors is a weakness in the pure form of the White since it uses many degrees of freedom for models with just a moderate number of independent variables. The special White test considers the following auxiliary regression:

$$\hat{u}^{\,2} = \alpha_0 + \alpha_1 \hat{y} + \alpha_2 \hat{y}^2 + \text{error}$$

  We can preserve the spirit of the White test while conserving on degrees of freedom by using the OLS fitted values in a test for heteroskedasticity.

4.4.8 Asymptotic Properties of the OLS

So fare we saw finite sample, small sample and exact properties of the OLS estimators in the population model.

Properties of OLS that hold for any sample size

- Unbiasedness under MLR1 to MLR4
- Variances formulas under MLR1 to MLR5
- Gauss-Markov Theorem under MLR1 to MLR5
- Exact sampling distributions/tests under MLR1 to MLR6

Properties of OLS in large samples

- Consistency under MLR1 to MLR4
- Asymptotic normality tests under MLR1 to MLR5

Practically the normality assumption MLR6 can be questioned and if it does not hold the results of the t or F tests may be wrong. They still work if the sample size is large enough, also OLS estimates are normal in large samples even without MLR6.

Asymptotic normality of OLS, under MLR1 to MLR5 we have

$$z = \frac{\hat{\beta}_j}{se(\hat{\beta}_j)} \sim N(0,1)$$

Consequently we have that

- In large samples the t-distribution is close to the N(0,1)
- Than t-tests are valid in large samples without MLR6
- Same for confidence intervals and F-tests
- But MLR1 to MLR5 are still necessary expecially for homoskedasticity

4.4.9 RESET Test

Another test that we have is the RESET (Resegression Specification Error Test) which is used to see if our model makes sense. Given an original model

$$y = \beta_0 + \beta_1 x_1 + ... + \beta_k x_k + u$$

we should have that no non-linear functions should be significant when added to this equation. Even though it can detect functional form problems it can use too many dof if there are many explanatory variables in the original model. Also certain kind of nonlinearities will not be picked if we add quadratic terms.

The RESET adds polynomials in the fitted OLS to detect general kinds of functional form misspecification.

1. Estimate the model $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k + u$ and obtain the fitted values $\hat{y}_i$.
2. Estimate the augmented model

$$y_i = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k + \gamma_1 \hat{y}_i^2 + \gamma_2 \hat{y}_i^3 + u_i$$

3. Test the null $H_0 : \gamma_1 = \gamma_2 = 0$ using an F-test with 2 and $n - k - 3$ dof (or an LM with 2 dof).

4.4.10 Multiple Regression Analysis with Qualitative Information

If we want to test the effect of qualitative information we can use dummy variables, variables that take either 1 or 0 as values. We can incorporate them inside our model like any other variable.

$$y = \beta_0 + \beta_1 D + \beta_2 x + u$$

We can interpret the 1 in β_1 as a shift in the intercept or the slope if we have an interaction with the x_s.

- D $= 0 \rightarrow y = \beta_0 + \beta_2 x + u$
- D $= 1 \rightarrow y = (\beta_0 + \beta_1) + \beta_2 x + u$

β_1 is the difference with respect to the base group (D $= 0$) and we have that

$$\beta_1 = E[y|\mathbf{x}, D = 1] - E[y|\mathbf{x}, D = 0]$$

The dummy variables trap occurs because dummy variables create perfect collinearity; to avoid it, one must drop one category from each set of dummies (or omit the intercept). The latter will make more difficult the test for the difference between parameters and the R^2 formula only works if the regression has the intercept.

It can be easily tested if the difference in means is significant

$$H_0 : \delta_0 = 0 \text{ vs } H_1 : \delta_0 \neq 0$$

A common specification uses the dependent variable in logs. In this case, the coefficient on the dummy has a percentage interpretation. For example, in the model

$$\log(\text{sal}) = \beta_0 + \delta_0 \, \text{fem} + \beta_1 \, \text{educ} + u,$$

the quantity $\delta_0 \times 100$ represents the approximate percentage difference in a woman's wage relative to that of a man. However, this log approximation may be inaccurate when the percentage difference is large. A more accurate measure is given by

$$100 \left(e^{\delta_0} - 1 \right).$$

We can use several dummy independent variables in the same equation. For example, we could add the dummy variable `married` to the wage equation, as follows:

$$\text{sal} = \beta_0 + \delta_0 \,\text{fem} + \delta_1 \,\text{married} + \beta_1 \,\text{educ} + u$$

The introduction of these dummies changes the model intercepts, depending on the combination of categories:

	Male	Female
Married	$\beta_0 + \delta_1$	$\beta_0 + \delta_0 + \delta_1$
Single	β_0	$\beta_0 + \delta_0$

Any categorical can be turned into a set of dummy variables, if we have a lot of them we can thing about grouping them like top 10, 11 to 20 and so on.

In the wage example, if `married` and `female` are included, we have the following possibilities:

Female	Married	Characterization
1	0	Single woman
1	1	Married woman
0	1	Married man
0	0	Single man

Including only `female` and `married` allows us to estimate the gender gap and the marriage premium.

However, a major limitation of this model is that the marriage premium is assumed to be the same for men and women.

A way to solve can be to choose as baseline single men and see the differences between different possibilities using marrmale, marrfem and singfem.

4.5 Panel Data

4.5.1 Motivations

We saw both cross-sectional and time series, if we consider data that follows both the dimensions we have panel. We look at how cross-sectional units are followed over time. Panel data can be used to analyze time-invariant unobservable.

The basic model is

$$y_{it} = \beta_0 + \beta_1 x_{it} + u_{it}$$

Where y and x are observed fo i=1,...,N individuals over t=1,...,T time periods

Those data help with issues that can't be addressed with pure cross-sectional or time-series. We are not using only the i index since individuals in the samples can differ in terms of x and other dimensions. Omitted variables can explain many trends and in panel data we try to solve for those. Let y and $\mathbf{x}=(x_1, x_2, ..., x_k)$ be observable random variables and c an unobservable random variable, or unobserved heterogeneity. We want to fix c and get the partial effects of the observable variables.

$$E[y \mid x_1, x_2, \ldots, x_k, c]$$

Assuming

$$E[y \mid \mathbf{x}, c] = \beta_0 + \mathbf{x}\beta + c$$

If c is uncorrelated with any x we just consider it as an error term but if the

$$\text{Cov}(x_j, c) \neq 0$$

for some j, if we put it as an error we encounter problems.

4.5.2 Estimators

The estimators employed in this setting are

- Pooled OLS
- Random effects estimation
- Fixed effects estimation

Let's look at them.

A **pooled OLS** applies the OLS for a sample of N X T observations ignoring the panel structure. With certain assumptions the pooled OLS can be used to obtain a consistent estimator of β

$$y_{it} = \mathbf{x}_{it}\boldsymbol{\beta} + v_{it}$$

Where $v_{it} = c_i + u_{it}$ are the composite errors. For each t v_{it} is the sum of unobserved effect and an idiosyncratic error (u_{it}). Pooled OLS estimation is ok when the regressor are not correlated with c.

The assumptions of pooled OLS are

1. **POLS.1: Linearity**
   The data sequence $\{y_{it}, x_{1it}, x_{2it}, \ldots, x_{kit}\}$ follows the linear model:
   $$y_{it} = \mathbf{x}_{it}\boldsymbol{\beta} + v_{it}, \qquad t = 1, 2, \ldots, T,\ i = 1, 2, \ldots, N$$
   where $\{v_{it}\}$ is the sequence of composite errors.
2. **POLS.2: Random Sampling**
   The individuals are selected by random sampling. This implies that $\{y_i, x_i\}$ are i.i.d., i.e., the observations are independent across individuals but not necessarily across time.
3. **POLS.3: Contemporaneous exogeneity**
   $$E[v_{it} \mid x_{it}] = 0,$$
   which implies that
   $$E[x_{it}c_i] = 0 \qquad \text{and} \qquad E[x_{it}u_{it}] = 0,$$
   hence
   $$E[x_{it}v_{it}] = 0.$$
4. **POLS.4: No Perfect Collinearity**
   In the sample, no independent variable is constant nor a perfect linear combination of the others.
5. **POLS.5: Homoskedasticity and no serial correlation**
   $$\mathrm{Var}(u_i \mid x_i, c_i) = \sigma_u^2 I.$$

Knowing that we have a sample of N independent cross sections observed during T periods of time and that c_i is independent of u_{it}, under POLS1–POLS4, the pooled OLS is consistent. However, if c_i is correlated with any element of x_t, then pooled OLS is biased and inconsistent.

In this setting the composite errors will be serially correlated due to c in each time period. So inference with pooled OLS requires a robust variance matrix estimator.

As with pooled OLS a **random effects analysis** puts c in the error term. The rationale is that the variation across individuals is assumed to be random and uncorrelated with the predictor or the independent variables included in the model. This estimation is appropriate when c is random and influences the dependent variable but is not correlated with the independent variables.
Random effects analysis has these properties

1. **RE.1: Linearity**
   The data sequence $\{y_{it}, x_{1it}, x_{2it}, \ldots, x_{kit}\}$ follows the linear model:
   $$y_{it} = \mathbf{x}_{it}\boldsymbol{\beta} + v_{it}, \qquad t = 1, 2, \ldots, T,\ i = 1, 2, \ldots, N$$
   where $\{v_{it}\}$ is the sequence of composite errors.

2. **RE.2: Random Sampling**
   The individuals are selected by random sampling. This implies that $\{y_i, x_i\}$ are i.i.d., i.e., the observations are independent across individuals but not necessarily across time.

3. **RE.3: Strict exogeneity**
   Random effects estimation imposes stricter assumptions than pooled OLS: strict exogeneity and orthogonality between c_i and x_{it}.

   (a) $E[u_{it} \mid x_i, c_i] = 0, \qquad t = 1, \ldots, T$
   (b) $E[c_i \mid x_i] = E[c_i] = 0$

   where $x_i = (x_{i1}, x_{i2}, \ldots, x_{iT})$.

4. **RE.4: No perfect collinearity**
   For consistency of GLS, the usual GLS rank condition is required (see later explanation):

$$\operatorname{rank}\big(E\big[X_i'\Omega^{-1}X_i\big]\big) = K.$$

5. **RE.5: Homoskedasticity**

   (a) $E[u_i u_i' \mid x_i, c_i] = \sigma_u^2 I_T$
   (b) $E[c_i^2 \mid x_i] = \sigma_c^2$

The random effects approach exploits the serial correlation in the composite error

$$v_{it} = c_i + u_{it},$$

in a generalized least squares (GLS) framework.

- We have
$$E[v_{it}^2] = E[(c_i + u_{it})^2] = E[c_i^2] + 2E[c_i u_{it}] + E[u_{it}^2] = \sigma_c^2 + \sigma_u^2, \tag{7}$$
- For $t \neq s$,
$$E[v_{it} v_{is}] = E[(c_i + u_{it})(c_i + u_{is})] = E[c_i^2] = \sigma_c^2. \tag{8}$$
- Hence, the variance–covariance matrix Ω of the T observations for individual i is

$$\Omega = \begin{pmatrix} \sigma_c^2 + \sigma_u^2 & \sigma_c^2 & \cdots & \sigma_c^2 \\ \sigma_c^2 & \sigma_c^2 + \sigma_u^2 & \cdots & \sigma_c^2 \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_c^2 & \sigma_c^2 & \cdots & \sigma_c^2 + \sigma_u^2 \end{pmatrix}.$$

Let $\hat{\beta}_{RE}$ be the random effect estimator

- Under RE1 to RE4 is consistent
- Under RE1 to RE5 is asimptotically efficient

Given that individuals have characteristics that may or may not influence the outcome we have that c may be correlated with the explanatory variables, in this case we use **fixed effects estimation**.
The assumptions are

1. **FE.1: Linearity**
   The data sequence $\{y_{it}, x_{1it}, x_{2it}, \ldots, x_{kit}\}$ follows the linear model:

$$y_{it} = \mathbf{x}_{it}\boldsymbol{\beta} + c_i + u_{it}, \qquad t = 1, 2, \ldots, T,\ i = 1, 2, \ldots, N \tag{9}$$

2. **FE.2: Random Sampling**
   The individuals are selected by random sampling. This implies that $\{y_i, x_i\}$ are i.i.d., i.e., the observations are independent across individuals but not necessarily across time.

3. **FE.3: Strict exogeneity**

$$E[u_{it} \mid x_i, c_i] = 0, \qquad t = 1, 2, \ldots, T$$

4. **FE.4: No perfect collinearity**

$$\text{rank}\left(\sum_{t=1}^{T} E[\ddot{x}'_{it}\ddot{x}_{it}]\right) = k$$

5. **FE.5: Homoskedasticity**

$$E[u_i u'_i \mid x_i, c_i] = \sigma_u^2 I_T$$

In fixed effects analysis E$[c_i|x_i]$ is allowed to be any function of x_i. We relax assumption RE3b which allows to consistenly estimate partial effects in the presence of time constant omitted variables that can be arbitrarily related to the observale x_{it}. Therefore fixed effects analysis is more robust than random effects one. The idea for estimating β under FE3 is to transform the equations to eliminate c. This is called fixed eefects transformation. Considering the mean values within each individual:

$$\bar{y}_i = \frac{1}{T}\sum_{t=1}^{T} y_{it}, \qquad \bar{x}_i = \frac{1}{T}\sum_{t=1}^{T} x_{it}, \qquad \bar{u}_i = \frac{1}{T}\sum_{t=1}^{T} u_{it}.$$

Subtracting these terms for each t gives the FE transformed equation:

$$y_{it} - \bar{y}_i = (x_{it} - \bar{x}_i)\beta + (u_{it} - \bar{u}_i) \tag{10}$$

or, in shorthand notation:

$$\ddot{y}_{it} = \ddot{x}_{it}\beta + \ddot{u}_{it}, \qquad t = 1, 2, \ldots, T \tag{11}$$

Downside: cannot be used to investigate time-invariant causes of y.

Fixed Effects Estimation:

- Since the assumption $E[\ddot{x}'_{it}\ddot{u}_{it}] = 0$ holds under Assumption FE.3, we can apply pooled OLS.
- The fixed effects (FE) estimator, denoted by $\hat{\beta}_{FE}$, is the pooled OLS estimator from the regression of $\ddot{y}_{it}$ on $\ddot{x}_{it}$.

It can be shown that the errors $\ddot{\mathbf{u}}_{it}$ are negatively serially correlated, but as T gets large the correlation tends to 0.

Under FE1 to FE4 the fixed effects estimator is consistent.

To summarize

1. **Pooled OLS**
   - c_i is not correlated with x_{it}
   - Ignore panel data structure and estimate through OLS
   - Under POLS.1 to POLS.4, it is consistent
2. **Random Effects**
   - c_i is not correlated with x_{it}
   - Consider correlation structure and estimate through GLS
   - Under RE.1 to RE.5, it is consistent and efficient
3. **Fixed Effects**
   - c_i is correlated with x_{it}
   - Demean each variable in the regression and estimate through OLS
   - Under FE.1 to FE.4, it is consistent

4.5.3 Hausman Test

Since the key consideration in choosing between a random effects and fixed effects approach is whether c_i and x_{it} are correlated, it is important to have a method for testing this assumption.

- Hausman (1978) proposed a test based on the difference between the random effects and fixed effects estimates. Since FE is consistent when c_i and x_{it} are correlated, but RE is inconsistent, a statistically significant difference is interpreted as evidence against the random effects assumption RE.3b.
- **Hausman test**
  - The hypotheses are

    $$H_0 : E[x_{it}c_i] = 0 \quad \text{(both FE and RE are consistent estimators, RE is the most efficient)}$$

    versus

    $$H_1 : E[x_{it}c_i] \neq 0 \quad \text{(only FE is consistent).}$$

  - The test applies only to variables that vary over time.
  - If the assumption RE.4 fails, one must use the *Robust Hausman test*.

4.6 Causal inference

4.6.1 Motivation

So far we looked at the study of the ceteris paribus relationship between dependent variable and one or more independent ones. We looked for associations among variables, based on which we may do some predictions. Causal inference is about counterfactual prediction, what would have happened to the same units after an exposure to a different condition.

Traditional prediction uses data to find patterns and find a functional relationship between the variables. Causal inference tries to find the effects that a different choice could have give us.

Recall that

- Correlation doesn't mean causality
- Coming first doesn't mean causality
- Causality may mask correlations

4.6.2 Potential outcomes

Until recently econometrics used realized outcomes to model causality. Now we can also care about potential outcomes. Take the covid and ventilators example. The treatment is the variable that we can manipulate (D). We model the potential outcomes assuming that we have two possible states for the same receiver if it had or not the treatment. Our analysis is to understand how big those differences are.

- $D_{i,t}$ →where i is the index of the receiver, t the ime of receivement and D either 0 or 1 if it received the treatment
- $Y_{i,t}^{j}$ →same as above but j is the treatment status and Y the status obtained from D

We denote potential outcom Y^1 as a priori fixed but unknown (outcomes associated with different possible treatment assignments) and realized Y^0 as posterior and known (outcome associated with a specific treatment assignment).

Definitions

1. Individual treatment effect →The individual treatment effect δ_i associated with a ventilator is $Y_i^1 - Y_i^0$
2. Switching equation →An individual's realizd health outcome Y_j, is determined by treatment assignment D_i which selects one of the potential outcomes

$$Y_i = D_i Y_i^1 + (1 - D_i) Y_i^0$$

$$Y_i = \begin{cases} Y_i^1 \text{ if } D_i = 1 \\ Y_i^0 \text{ if } D_i = 0 \end{cases}$$

3. Fundamental problem of causal inference →If you need both potential outcomes to know causality with certainty, then since it is impossible to observe both Y_i^1 and Y_i^0 for the same individual, δ_i , is unknowable. The Fundamental problem is deep and impossible to overcome. Causal inference is a missing data problem. All of causal inference involves imputing missing counterfactuals and not all imputations are equal

4. Average treatment effect (ATE) →The average treatment effect is the population average of all i individual treatment effects

$$E[\delta_i] = E[Y_i^1 - Y_i^0] = E[Y_i^1] - E[Y_i^0]$$

   Cannot be calculated because Y_i^1 and Y_i^0 do not exist for the same unit i due to switching equation

5. Average Treatment Effect on the Treated (ATT) →The average treatment effect on the treatment group is equal to the average treatment effect conditional on being a treatment group member:

$$E[\delta|D = 1] = E[Y^1 - Y^0|D = 1] = E[Y^1|D = 1] - E[Y^0|D = 1]$$

   Cannot be calculated because Y_i^1 and Y_i^0 do not exist for the same unit i due to switching equation.

6. Average Treatment Effect on the Untreated (ATU) →The average treatment effect on the untreated group is equal to the average treatment effect conditional on being untreated:

$$E[\delta|D = 0] = E[Y^1 - Y^0|D = 0] = E[Y^1|D = 0] - E[Y^0|D = 0]$$

   Cannot be calculated because Y_i^1 and Y_i^0 do not exist for the same unit i due to switching equation.

Average treatment effects are simple summaries. Because aggregate causal parameters are summaries of individual treatment effects, each of which cannot be calculated, the aggregates cannot be calculated either. Here the missing data are really non-existent. While we cannot measure average causal effects, we can estimate them, but only in situations and we review one randomization.

4.6.3 DAGs

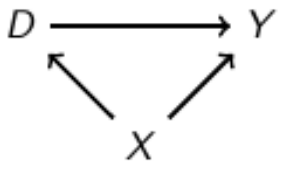
Causal Graphs are a kind of graphs that let us model a representation of causal effects.

Why

- Facilitates the task of designing identification strategy for estimating average causal effects
- Facilitates the task of testing compatibility of the model with your data
- Visualizes the identifying assumptions which opens up the model to critical scrutiny

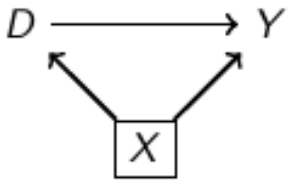
Notation

1. Treatment →D
2. Outcome →Y
3. Observed covariates or confounders →X
4. Unobserved covariates or confounders →U



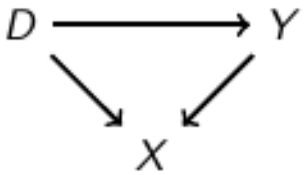
Here we have 3 random variables X, D and Y. There is a direct path from D to Y , which represents a causal effect. But there is also a second path from D to Y called the backdoor path. The backdoor path is given by D ← X → Y . The backdoor path is not causal, we therefore call X a confounder.

Confounding occurs when the treatment and the outcomes have a common cause or parent which creates spurious correlation between D and Y . The correlation between D and Y no longer reflects the causal effect of D on Y because of selection bias due to a confounder. We need then to control for X to close the backdoor path.



We can ”block” any particular backdoor path by conditioning on variable X so long as it is not a collider. Once we condition on X , the correlation between D and Y estimates the causal effect of D on Y Conditioning means calculating E [Y | D = 1, X] - E [Y | D = 0, X] for each value of X

Now we have an unobserved confounder. Since U is unobserved that backdoor pathway is open.



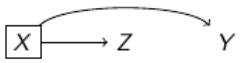
D → Y (causal effect of D on Y) and D → X ← Y (backdoor path 1). Just like last time, there are two ways to get from D to Y . But this time, X has two arrows pointing to it, not away from it. When two variables cause a third variable along some path, we call that third variable a “collider”. But so what? Colliders are special because when they appear along a backdoor path, that backdoor path is closed simply because of their presence. DAGs allow us to map the relationship between variables. Identify colliders and confounding factors will determine which variables we should (and should not) condition, i.e., control for in the analysis. To identify a causal relationship, we want to block (close) all the backdoor paths.

A backdoor path is closed if and only if

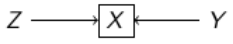
- It contains a noncollider that has been conditioned on
- Or it contains a collider that has not been conditioned on

Examples

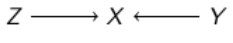
1. Conditioning on a noncollider blocks a path:



2. Conditioning on a collider opens a path (i.e., creates spurious correlations):



3. *Not* conditioning on a collider blocks a path:



Conditioning on X satisfies the backdoor criterion with respect to (D, Y) directed path if:

1. All backdoor paths are blocked by X
2. No element of X is a collider

In words: If X satisfies the backdoor criterion with respect to (D, Y), then controlling for or matching on X identifies the causal effect of D on Y . And again note that a path which has a conditioned-on-collider can still be closed, but only with a noncollider-conditioned-on.

You can identify average causal effects using a covariate adjustment strategy so long as you do not in the process inadvertently "open" backdoor paths. Backdoor paths can remain open in covariate adjustment strategies through two ways:

1. You did not close the path because you did not condition on the confounder
2. Your conditioning variable opened up a previously closed backdoor path because on that path the variable was a collider

Colliders are "bad controls"; they are often outcomes, but not always and not all outcomes create this problem. This is the risk of blindly controlling for variables ("kitchen sink regressions")

4.6.4 Instrumental variables

An instrumental variable (Z) is a variable that influences treatment assignment, is independent of unmeasured confounders and has no effect on the outcome except through its effect on the treatment. We should use them when we have unobserved confounders. With them we can estimate the variation in the treatment that is free of the unmeasured confounders. We can use this confounder free variation to estimate the causal effect of the treatment. The biggest challenge in using IV methods is finding a good IV.

To assess the values of the correlation of instrumental variables we can do Two stage least squares

- Start from the causal model of the main research question
- Do a regression on the parameter explained by the instrumental variable and find the relative value of the second
- Use the fitted values of education in the model

The fitted value, educ, can be seen as the estimated version of educ that is uncorrelated with u. Therefore, 2SLS first "purges" educ of its correlation with u before doing the OLS regression.

We can have some specification tests

- Weak instruments →The null is "All instruments are weak". We reject if all are strong
- Hausman test for endogeneity →The null is that the covariance between x and u is 0. We reject if we need instrumental variables for the existence of endogeneity
- Sargan test →This test for the validity of instruments (whether the instruments are corrrelated with the error term) can only be performed for the extra instruments, those that are in excess of the number of endogenous variables. The null is that the covariance between z and u is 0. So if we reject at least one of the extra instruments are not valid.

4.6.5 Difference-in-difference

Very old but easy approach for research design. A group of units are assigned some treatment and then compared to a group of units that weren't.
We can do this with and without regression.

- Without →We start by taking the average value of each group's outcome before and after the treatment. Then, calculate the difference-in-differences of the averages $(\bar{y}_A^T - \bar{(y)}_B^T) - (\bar{y}_A^C - \bar{y}_B^C)$
- With →We fit a regression model $y_{it} = \beta_0 + \beta_1 D_i + \beta_2 \text{Post}_t + \beta_3 (D_i \times \text{Post}_t) + u_{it}$ Where D=1 if i has received the treatment and Post=1 is t is after the treatment. We have two groups and two periods.

We can simplify the with regression model by assuming that we have one equation for the treated and one for the non treated. The regression framework allows to add regressors as controls. In that case, the interpretation is no longer as sample average, but as partial effects after controlling for other factors.
How we would expect the treated group to behave in the absence of treatment is called the parallel trends assumption. Parallel trends assumes away the selection bias associated with comparisons The assumption is thought to be more plausible than simply assuming simple comparisons held equal

$$E[Y^0|D=0] = E[Y^0|D=1]$$

We do a visual inspection of the plots and/or test the difference in means in the periods before the intervention. If the parallel trends assumptions holds, then the differences in means between the control and treated group, before the intervention are zero.

4.6.6 Advanced topics

not copying this

5 Database

5.1 Introduction

5.1.1 What is a database

A database can be defined as a collection of related data items within a specific business. DBMS are sotwares that define, create, use and maintain a database, if we combine a DBMS with a database we get a database system The majority of DBMS softwares rely on SQL systems.

5.1.2 Conceptual Modelling of a database

To work with databases we need to model them to understand how they work. Modelling means representing reality in a scaled version. The same with databases we connect information to represent how relationships in the world exist. Based on the business needings we design the database in a logical fashion and we translate it to a our language of choice. The step is passing from conceptual design to logical design. The first is DBMS independent while the second is DBMS dependent. The process is:

1. Requirement collection and analysis
2. Conceptual design
3. Logical design
4. Physical design

An SQL database is based on entities and relationships. Entities are business concepts with an unambiguous meaning to a particular set of users. Entities have certain characteristics called attributes. The rows are the entities (also called instances). The columns are the attributes. An association between one or more entities is called relationship, it defines how tables are related to each other.

Another useful concept is the one of cardinality, how many instances of the entities can be associated with each other.
We can have

- 1 to 1
- 1 to many
- Many to many
- None to many
- 1 to None

5.1.3 Normalization

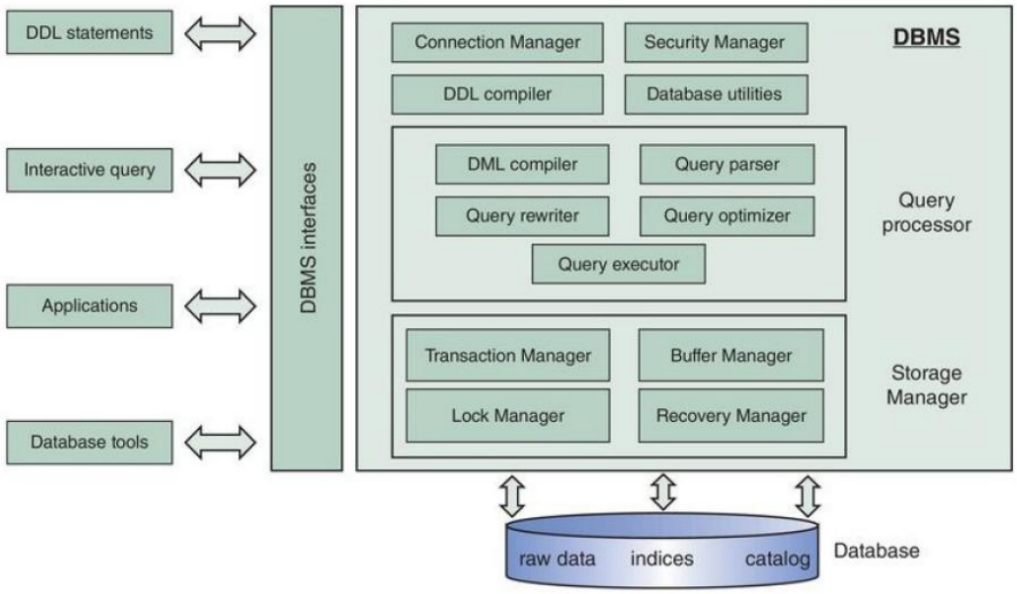
An important step in the design is the one related to data normalization, we want to avoid redundant data. We split the tables into smaller tables to avoid anomalies during insertion, deletion or updates. The procedure follows a step by step transformation. The database is more complex from the structural point of view but can be easier to manage.
We have 3 possible normal forms.

- First NF →Each cell in the table can have only one value, never a list of values
- Second NF →An entity type is in 2NF when it is in 1NF and when all of its non-key attributes are functionally dependent on its primary key. Features should be fully dependent on the entire primary key
- Third NF →An entity type is in 3NF when it is in 2NF, and non-key attributes are directly (non-transitively) dependent on the primary key. In other words: non-prime attributes must be functionally dependent on the key(s), but they must not depend on another non-prime attribute.

5.2 Database modelling

5.2.1 Architecture

All DBMS follow a certain structure made by certain elements



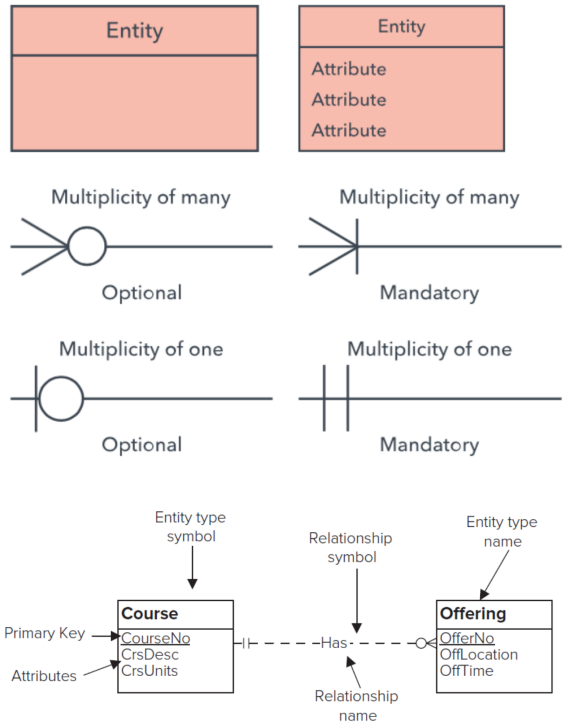
- **Connection manager** →provides facilities to set-up a database connection. It can be set-up locally or through a network
- **Security manager**→verifies whether a user has the right privileges to execute the database actions required
- **DDL compiler**→compiles the data definitions specified in DDL. Most relational databases use SQL as their DDL
- **Utilities**→A loading utility (load data from a variety of sources), reorganization utility (reorganizes the data), user management utilities (support the creation of user groups or accounts), etc
- **DML compiler**→compiler compiles the data manipulation statements specified in DML
- **Query parser**→parses the query into an internal representation format that can then be further evaluated by the system. It checks the query for syntactical and semantical correctness
- **Query rewriter**→optimizes the query, independently of the current database state. It simplifies it using a set of predefined rules and heuristics that are DBMS-specific
- **Query optimizer**→optimizes the query based upon the current database state. It can make use of predefined indexes that are part of the internal data model and provide quick access to the data
- **Query executor**→takes care of the actual execution by calling on the storage manager to retrieve the data requested
- **Transaction manager**→supervises the execution of database transactions. Remember, a database transaction is a sequence of read/write operations considered to be an atomic unit
- **Buffer manager**→is responsible for managing the buffer memory of the DBMS. The DBMS checks first the memory when data need to be retrieved. Retrieving data from the buffer is significantly faster than retrieving them from external disk-based storage
- **Lock manager**→essential component for providing concurrency control, which ensures data integrity at all times
- **Recovery manager**→supervises the correct execution of database transactions. It keeps track of all database operations in a logfile and will be called upon to undo actions of aborted transactions or during crash recovery

A DBMS needs to interact with various parties, such as a database designer, a database administrator, an application, or even an end-user (Ex. APIs).

5.2.2 ERD

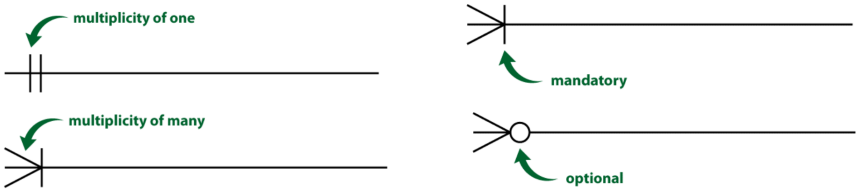
To design our databases we can use an ERD (Entity Relationship Diagram) notation.

Some of them can be Chen, IDEF1X, Bachman, Crow's Foot (we use this), Min-Max or UML.



An entity is represented by a rectangle, with its name on the top. The name is singular. The attribute(s) that uniquely distinguishes an instance of the entity is the identifier.

Relationships have two indicators. The first one refers to the maximum number of times that an instance of one entity can be associated with instances in the related entity. The second one describes the minimum number of times one instance can be related to others. It can be zero or one, and optional or mandatory.



5.2.3 From ERD to tables

Relations can be

- One to one
- One to many
- Many to many

Since we are working with tables we need to pass from schemas to tables, we use keys (foreign and primary). We can also create a linking table to model many to many relations.

5.3 SQL

5.3.1 What is SQL

SQL means Structured Query Language. It was created by IBM in 1974 and can be implemented by each vendor (SQL dialects).

Key characteristics

- Set-oriented and declarative
- Free form
- Case insensitive
- Can be used via prompt or via a program

5.3.2 Create a database

To show existing databases

```
SHOW DATABASES;
```

Create database

```
CREATE DATABASE [IF NOT EXISTS] database_name
[CHARACTER SET charset_name]
[COLLATE collation_name]
```

Use Database

```
USE database_name;
```

Drop database

```
DROP DATABASE [IF EXISTS]
database_name;
```

See tables in the database

```
SHOW TABLES;
```

5.3.3 Create tables

Create a Table

```
    CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
col2 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
col3 data_type(length) [NOT NULL] [DEFAULT value] [AUTO_INCREMENT]
) ENGINE=storage_engine;
```

Data Type	Description
CHAR(size)	Holds a fixed length string (can contain letters, numbers, and special characters). The fixed size is specified in parenthesis. Can store up to 255 characters.
VARCHAR(size)	Holds a variable length string (can contain letters, numbers, and special characters). The maximum size is specified in parenthesis. Can store up to 255 characters. Note: If you put a greater value than 255 it will be converted to a TEXT type.
TEXT	Holds a string with a maximum length of 65,535 characters.
INT(size)	Integer. Range: -2,147,483,648 to 2,147,483,647 (normal), 0 to 4,294,967,295 (UNSIGNED). Maximum number of digits may be specified in parenthesis.
FLOAT(size,d)	Small number with floating decimal point. Maximum number of digits may be specified in `size`. Maximum digits to the right of the decimal point are specified in `d`.
DOUBLE(size,d)	Large number with floating decimal point. Maximum number of digits may be specified in `size`. Maximum digits to the right of the decimal point are specified in `d`.
DATE()	A date. Format: YYYY-MM-DD. Supported range: '1000-01-01' to '9999-12-31'.
YEAR()	A year in two-digit or four-digit format. Values allowed in four-digit format: 1901 to 2155. Values allowed in two-digit format: 70 to 69, representing years from 1970 to 2069.

Table 13: Common SQL Data Types

Primary keys

```
CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length) [PRIMARY KEY]
col2 data_type(length) [PRIMARY KEY]
col3 data_type(length) [PRIMARY KEY]
);

CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length)
col2 data_type(length)
col3 data_type(length)
PRIMARY KEY(col1, col2, ...)
);

CREATE TABLE Person (
pid int AUTO_INCREMENT NOT NULL,
firstname varchar(25) NOT NULL,
surname varchar(25) NOT NULL,
PRIMARY KEY (pid)
);

CREATE TABLE [IF NOT EXISTS] table_name(
col1 data_type(length)
PRIMARY KEY(col1, col2, ...)
CONSTRAINT constraint_name  \ra define constraint name for the foreign key
FOREIGN KEY (columns)
REFERENCES parent_table (columns)
ON DELETE action    \ra CASCADE, SET NULL, NO ACTION, RESTRICT
```

```
ON UPDATE action    \ra CASCADE, SET NULL, NO ACTION, RESTRICT
);
```

Foreign key examples

```
CREATE TABLE category(
categoryId INT AUTO_INCREMENT PRIMARY KEY,
categoryName VARCHAR(100) NOT NULL);

CREATE TABLE products(
productId INT AUTO_INCREMENT PRIMARY KEY,
productName varchar(100) not null,
categoryId INT NOT NULL,
CONSTRAINT fk_category
FOREIGN KEY (categoryId)
REFERENCES category(categoryId)
ON UPDATE SET NULL
ON DELETE SET NULL );
```

Drop a table

```
DROP TABLE [IF EXISTS] table_name [, table_name]
```

5.3.4 CRUD operations

CRUD operations refere to those operations that are meant to Create, Read, Update and Delete data in our database.

To insert data

```
   INSERT INTO table_name(c1, c2,...)
   VALUES (v1,v2,...);
```

To read data

```
   SELECT column_1, column_2,\dots
   FROM table_1
   [INNER|LEFT|RIGHT] JOIN table_2 ON condition
   WHERE conditions
   GROUP BY column_1
   HAVING group_conditions
   ORDER BY column_1
   LIMIT offset, length;
```

To update our data

```
UPDATE [LOW_PRIORITY] [IGNORE] table_name
SET column_name1 = expr1,
column_name2 = expr2,
[WHERE condition];
```

To delete data

```
DELETE FROM table_name
WHERE condition;
```

5.3.5 SQL clauses and operators

The main clauses and operators that we have in SQL are

- LIMIT
- DISTINCT
- ORDER BY
- BETWEEN

NOT LIKE

LIMIT is used in the select statement to costrain the number of rows to return

```
SELECT select_list
FROM table\_name
LIMIT [offset] row_count;
```

Offset specifies the offset of the first row to return (from 0) while row_count the maximum number of rows to return

LIMIT will remove duplicates

```
SELECT DISTINCT e.first_name FROM employees AS e LIMIT 10;
```

The BETWEEN operator is a logical one that we can use to specify a value within or not a range

```
SELECT e.emp_no, e.first_name, e.last_name, e.hire_date
FROM employees AS e
WHERE e.hire_date BETWEEN '1999-01-01' AND '1999-12-31';
```

The LIKE operator is a logical operator that tests whether a string contains a specified pattern or not

```
SELECT e.first_name, e.last_name
FROM employees AS e
WHERE e.last_name LIKE 'T%';


SELECT e.first_name, e.last_name
FROM employees AS e
WHERE e.last_name NOT LIKE 'Tr%';
```

We can use the keyword AS to refere in a different way

```
table_name AS table_alias

SELECT
[column_1 | expression] AS descriptive_name
FROM table_name;

SELECT
e.firstName, e.lastName
FROM employees e
ORDER BY e.firstName;

SELECT
CONCAT (lastName, ' ', firstname) AS Full_name
FROM employees;
```

The IN operator allows you to determine if a specified value matches any value in a set of values or returned by a subquery.

```
SELECT column1,column2,...
FROM table_name
WHERE column_n IN (value1,value2,...);
```

The SELECT statement does not provide the result set sorted. To sort the result set, you add the ORDER BY clause to the SELECT statement

```
SELECT select_list
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC], ...;
```

5.3.6 SQL joins

There are four basic types of SQL joins: inner, left, right, and full.

INNER

```
SELECT column_list
FROM table_A
INNER JOIN table_B ON join_condition;

SELECT column_list
FROM table_A
INNER JOIN table_B USING (column_name);

SELECT column_list
FROM table_A A, table_B B
WHERE A.column_name = B.column_name;

SELECT column_list
FROM Table_A A
INNER JOIN Table_B B
ON A.Key = B.Key;
```

LEFT

```
SELECT column_list
FROM Table_A A
LEFT JOIN Table_B B
ON A.Key = B.Key;
```

RIGHT

```
ELECT column_list
FROM Table_A A
RIGHT JOIN Table_B B
ON A.Key = B.Key;
```

OUTER/FULL

```
SELECT * FROM t1
LEFT JOIN t2 ON t1.id = t2.id
UNION ALL
SELECT * FROM t1
RIGHT JOIN t2 ON t1.id = t2.id
```

Left excluding

```
SELECT column_list
FROM Table_A A
```

```
LEFT JOIN Table_B B
ON A.Key = B.Key
WHERE B.Key IS NULL;
```

Right excluding

```
SELECT column_list
FROM Table_A A
RIGHT JOIN Table_B B
ON A.Key = B.Key
WHERE A.Key IS NULL;
```

Self

```
SELECT m.lastName AS Manager, e.lastName AS 'Direct report'
FROM employees e
INNER JOIN employees m ON m.employeeNumber = e.reportsTo;
```

5.3.7 Functions

Functions in SQL can be

- Aggregate →Performs a calculation on multiple values and returns a single value. Can be AVG(), MAX(), MIN(), STDEV(), COUNT() or SUM(). COUNT() has three main forms
  1. COUNT(*) →returns the number of rows including duplicates, non-NULL and NULL rows.
  2. COUNT(expression) →returns the number of rows with non NULL values
  3. COUNT(DISTINCT expression) →returns the number of distinct rows that do not contain NULL values

```
SELECT count(employee_id) as employees
FROM employee
WHERE department_id = 50;
```

- Math →mathematical computations. Can be MOD(), ROUND(), TRUNCATE(), SIN(n), SQRT(n), RAND(), LOG(n) or POW()

```
SELECT round(salary, 2)
FROM employee;
```

- Date →make computations on date values. Can be CURDATE()/CURRENT_DATE(), DATEDIFF(), DAY(), DAYOFWEEK(), NOW(), MONTH(), TIMEDIFF() or YEAR().
- String →work on strings. Can be CONCAT(), LENGTH(), LOWER(), REPLACE(), SUBSTRING(), TRIM(), UPPER() or INSTR()

5.3.8 Group By

The GROUP BY clause tries to take a set of rows and summarize them by values. It returns one row for each group. Is used with SUM, AVG, MAX, MIN and COUNT.

```
SELECT c1, c2, ..., cn, aggregate_function(ci)
FROM table_name
WHERE where_conditions
GROUP BY c1, c2, ..., cn;
```

5.3.9 Having

The HAVING clause is used in the SELECT statement to specify filter conditions for a group of rows or aggregates. Oten used with GROUP BY to filter groups based on a specific condition. HAVING applies a filter condition to each group of rows, while the WHERE does that to each individual row.

```
SELECT select_list
FROM table_name
WHERE search_condition
GROUP BY group_by_expression
HAVING group_condition
```

5.3.10 Views

A VIEW is defined by means of an SQL query and its content is generated upon invocation of the view by an application or other query. Queries can be run against the view. A view will not physically store the data. Each time a view is queried the underlying query called view definition is executed. We create views to do queries that we might want to do frequently.

```
CREATE[OR REPLACE] VIEW [db_name.]view_name[(column_list)]
AS select-statement
```

5.3.11 Triggers

A TRIGGER is a piece of SQL code made of declarative and/or procedural instructions and stored in the catalog of the RDBMS. Is automatically runned whenever a specific event (INSERT, UPDATE, DELETE) occurs and a specific condition is evaluated as true.

```
CREATE TRIGGER trigger_name trigger_time trigger_event
ON table_name
FOR EACH ROW
BEGIN what trigger does
END;
```

Where trigger time can be BEFORE or AFTER.
Where trigger event can be INSET, UPDATE or DELETE.

Advantages

- Aumatization and verification in case of specific events or situations
- Can model more rules without change the code
- Can assign default values for new tuples
- Synchronic updates in case of data replication
- Automatic auditing and logging
- Automatic exporting of data

Disadvantages

- Hidden, difficult to manage
- Cascade effects
- Uncertain outcomes
- Deadlock situations
- Debugging complexities
- Maintainability problems

5.4 Query Optimization

5.4.1 The problem

Users can complain that applications are slow. One reason can be that the queries in the SQL database are slow and not optimized.

User's feeling about response time

- 0.1 second →feels lika instantaneous
- 1 second →limit to that feeling
- 10 seconds →the user will do something else

A query of 3.19 seconds can be considered as slow, we need to understand if it has problems. We can do that by using a QEP (Query Execution Plan). A query plan is a list of instructions that the database needs in order to execute a query on the data. The query optimizer will generate multiple query plans for a single query and will select the most efficient one.

```
SELECT *
FROM tools
WHERE name="Screwdriver";
```

vs

```
SELECT *
FROM tools
WHERE id = 3;
```

Both are returning the same results but the second one because the index is usually unique and comparisons between strings is slower.

5.4.2 EXPLAIN

The EXPLAIN command is used to show query plans. It doesn't run the actual SQL statement (except subqueries). It doesn't provide any tuning recommendation but information that can be used to do that. For example we can use a newly created index with an alter statement.

How to identify problems using EXPLAIN

- No index used (NULL in the key column)
- A lot of rows are processed (rows column)
- A lot of indexes are evaluated (possible_keys column)

Remember

- We can optimize queries in many ways
- The generated query plan can change depending o many factors
- For UPDATE and DELETE statements we need to rewrite the query as a SELECT statement to confirm index usage

5.4.3 Indexes

We need to be sure that our indexes make sense. An index is a structure that holds the field the index is sorting and a pointer from each record to their corresponding record in the original table where the data is stored. The id is incrementing based on the order in which the data was added. To fasten the search the data are organized like a binary tree based on the middle entry. Indexing makes columns faster to query by creating pointers to where data is stored within a database.

Index cardinality (amount of unique values)

- Multiple different indexes make MYSQL try to identify the most effective index for the query.
- The optimizer chooses an index based on the estimated cost to do the least amount of work
- Index cardinality can be used to to that

Normally indexes are created when the table is created. After we can do that using

```
CREATE INDEX index_name ON table_name
(column_list)
```

5.4.4 Data types

Even data types can impact performance based on their structures. Usually the simpler teh datatype is the better it performs. The best way to do this is to use a data type that is native for the data that we need to use

- tinyint
- int
- bigint
- json
- longtext
- longblob
- decimal
- float
- double
- enum

5.4.5 Tips

1. Define SELECT fields instead of SELECT *
2. Avoid SELECT DISTINCT if possible
3. Use WHERE instead of HAVING
4. Use WILDCARDS at the end of the phrase
5. Use LIMIT
6. Run queries during off-peak times
7. Index the queries properly →the column is queried frequently, foreign keys to other tables and a unique key exists on the columns

5.5 CAP theorem and NOSQL databases

5.5.1 ACID properties

Transactions are collections of actions that make consistent transformations of systems preserving consistency

- A →atomicity, the changes must be fully done
- C →consistency, any database transaction must change affected data only in allowed ways
- I →isolation, how and when the changes made by one operation become visible to others
- D →durability, transactions that have been commited must survive permanently

RDBMs must be ACID compliant. New data or modifications are noit accepted unless they comply with the given schema in terms of data, integrity etc. In big databases we need flexibility since we have a lot of data variety. Also new functionalities need all the elements to support the new structure.

5.5.2 NOSQL

Since we might have multiple queries together we can scale our server both vertically or horizontally

- Vertically →extending storage capacity and/or CPU power of the database server
- Horizontally →multiple DBMS servers being arranged in a cluster

RDBMs can have problems in horizontally scaling. NOSQL databases solve this by using formats other than tabular relations. They aims at near linear horizontal scalability distributing data on clusters of database nodes for improving performance and availability. Also we have eventual consistency: the data will become consistent at a certain point. The languages can vary from different languages

	Relational Databases	NoSQL Databases
Data paradigm	Relational tables	Key–value (tuple) based Document based Column based Graph based XML, object based Others: time series, probabilistic, etc.
Distribution	Single-node and distributed	Mainly distributed
Scalability	Vertical scaling, harder to scale horizontally	Easy to scale horizontally, easy data replication
Openness	Closed and open source	Mainly open source
Schema role	Schema-driven	Mainly schema-free or flexible schema
Query language	SQL as query language	No or simple querying facilities, or special-purpose languages
Transaction mechanism	ACID: Atomicity, Consistency, Isolation, Durability	BASE: Basically Available, Soft state, Eventual consistency
Feature set	Many features (triggers, views, stored procedures, etc.)	Simple API
Data volume	Capable of handling normal-sized datasets	Capable of handling huge amounts of data and/or very high frequencies of read/write requests

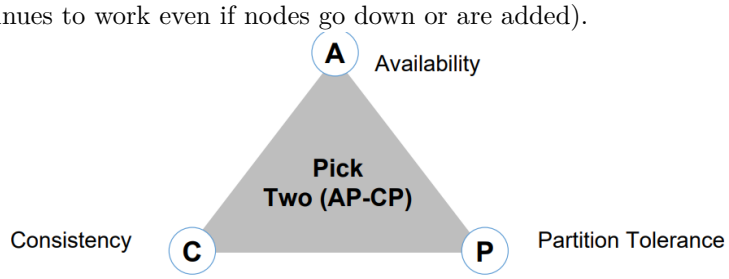
NOSQL databases as rule of thumb are not acid compliant than can be problematic for critical trnasactions.

NOSQL relies on BaSE model

- Basically available →Guarantees the availability of the data . There will be a response to any request (can be failure too).
- Soft state →The system can change over time, even without receiving input (since nodes continue to update each other)
- Eventual consistency →The system will become consistent over time but might not be consistent at a particular moment

5.5.3 CAP theorem

A distributed computer system, such as the one we've looked at above, cannot guarantee the following three properties simultaneously: consistency (all nodes see the same data simultaneously); availability (guarantees that every request receives a response indicating a success or failure result); and partition tolerance (the system continues to work even if nodes go down or are added).



We have to choose between 2 of those points

- CA →Vertical scalability, SQL
- P →horizontal scalability

6 Ethics for data science

6.1 Introduction

6.1.1 Law, moral and ethics

Ethics as concept is old, but the application in data science is fairly recent. Ethics is really important in fields where the consequences are really important, like healthcare.

- **Ethics** →Moral principles, branch of philosophy that studies the basis of moral. Set of rules of conduct. Aims to direct the individual towards good practices. Individual perfecting. Based on the relationship of the individual with himself. Ethical rules are filtered by the individual, and it's conscience. Violation of ethical rules can lead to self-imposed sanctions. Focussed on duties and obligations but not on rights.
- **Law** →Aims to regulate the essential relationships within a given society. Harmonious interactions and to solve conflicts between different people. Laws don't need individual conscience to be applied. Legal violations will lead to loss of liberties and life. Equal focus on duties and rights.

Laws waits for a conduct while ethics anticipate that conduct. Everyone can form an ethical opinion while legal ones are given by a judge. In ethical settings suspicion can affect someone while in legal ones people are innocent until proven guilty.

Historically we saw innovations that lead to ethical principles and definitions (artificial life support created the legal definition of death or human genes discovery gave us the legal distinction of person vs thing).

6.1.2 What is ethics

We use ethics to make decisions related to choices that have moral dimensions. How one should act.
Ethics is made from two big branches

1. Non-normative ethics →Studies what effectively the case is and not what ethically ought to be. We can have in this setting descriptive ethics (studies people beliefs about moralities) and meta-ethics (studies the meaning of moral propositions like right, obligations etc.)
2. Normative ethics →Studies the practical means of determining a moral course of action. It's applied ethics.

We can also define the professional ethics branch, made of ethical guidelines set accepted by the majority of professionals. At a more granular level there exists also specific professional ethical rules. Some professions will adopt more detailed codes called Codes of ethical conduct. A public policy is a set of enforceable guidelines set by an official public body.

$$\text{Public policy} = \text{Ethical principles} + \text{Empirical factors}$$

6.1.3 Ethical dilemmas

- Psychologist and patient
- Cancer genes daughter

Ethical Dilemmas are circumstances in which ethical obligations demand or appear to demand that a person adopt each of two (or more) alternative but incompatible actions, such that the person cannot perform all the required actions. Conflicts between ethical requirements and self-interest are practical dilemmas and not necessarily ethical dilemmas

Tools of ethical decision-making

- **Basic ethics principles** →Aim to express general norms of common ethics (respect for autonomy, non-maleficence, beneficence, justice)
- **Ethical rules** →More specific in content and more restricted in scope than principles. (however, both establish rights and obligations). Function as precise guides to action that direct us in each circumstance. They can be (substantive (rules of content), authority (who should make decisions and perform actions) and procedural(procedures to be followed in certain circumstances))

- **Weighting and balancing** →Principles, rules, professional obligations, and rights often need to be weighted and balanced. Balancing is the process of finding reasons to support beliefs about which ethical norms should prevail. It is a process of practical astuteness, discriminating intelligence, and sympathetic responsiveness. Necessary conditions are:

  1. Good reasons can be offered to act on the overriding norm rather than on the infringed
  2. The moral objective justifying the infringement has a realistic prospect of achievement
  3. No morally preferable alternative actions are available
  4. The lowest level of infringement, commensurate with achieving the primary goal of the action, has been selected
  5. Any negative effects of the infringement have been minimized
  6. All affected parties have been treated impartially

- **Virtues and emotions** →We want tho avoid extremes situations like detachment or extreme attachment. In the absence of public and institutional constraints, partiality towards others is ethically permissible and is the socially expected form of interaction. Some necessary elements are **discernment** (the ability to make fitting judgements and reach decisions without being influenced by extraneous factors), **trustworthiness** (Confident belief and reliance on the moral character and competence of another person. Confidence that another will act with the right motives and in accordance with ethical norms), **integrity** (Soundness, wholeness and integration of moral character. In a restricted sense means fidelity in adherence to moral norms), **conscientiousness** (Motivation to do what is right because it is right after trying with due diligence to determine what is right) and **compassion** (Active regard for another's welfare with an imaginative awareness and emotional response of sympathy, tenderness and discomfort at another's misfortune or suffering.)

Main ethical theories

- Divine command
- Natural Law
- Virtue ethics
- Deontology
- Utilitarianism

A core point of ethical dilemmas is the disagreement that they can create given by different backgrounds, ideas or the quantity of information that everyone participating has. Disagreements may persist even among ethically committed persons. Ethical disagreements may discourage persons who deal with practical problems but should be no cause for skepticism about ethical thinking.

Data ethics works on many levels based on the strength of normative instruments. Remedies to violations can be

- Prevention
- Detection/Investigation/Accusation
- Sanctions
  - Legal
  - Disciplinary
  - Professional
  - Social
  - Individual

6.2 History of data ethics

Some case studies

- Love is blind day
- Nuremberg trial
- Tuskegee Experiment
- Milgram
- Rosenhan
- Tearoom trade
- Stanford prison

Possible ways to approach digital revolution

- Unchecked optimism
- Go backwards
- Accept with uneasiness
- Bend it towards justice and fairness

An important element for the last point is informed consent created after the Nuremberg code, it says that voluntary consent of the human subject is absolutely essential. Also, the degree of risk should be proportional to the humanitarian impact. Informed consent is made of

- Previous and precise acknowledgement of the situation
- Freedom from coercion
- Competence
- Flexibility
- Attention to vulnerable groups

Sometimes consent is impossible to obtain, but it must be regularly evaluated. Consent must be

- Informed
- Voluntary
- Ethical

The data ethics framework is made of

- Overarching principles →Transparency, accountability, fairness
- Specific actions

6.3 Basics of organizational ethics

Some snippets from Antonio Martins life. We learn our names and origins.

Ethics is a set of principles, values and standards of conduct which guides individuals and organizations determining what's right and wrong. It involves making decisions and taking actions that are morally right considering the impact on stakeholders. In the context of organizations ethics plays a crucial role in shaping the behavior and culture of the workplace.
Video on ethics.

We can consider organizational ethics as

- Trust and reputation →Ethical behavior builds trust among stakeholders
- Legal compliance →Ethical principles often align with legal requirements

- Employee morale and engagement →A culture of ethics fosters a positive work environment where employees feel valued and will be motivated to perform

We live in a society (professor said it). João from Leiria talks.

Why companies need to be ethical

- Building trust and reputation →foundation of successful relationships between companies and customers
- Enhancing employee engagement and retention →employees will feel proud to work for a certain company if it adheres to ethical principles
- Mitigating risks and legal liabilities →by adhering to ethical principles organizations will minimize the risk of facing costly legal proceedings

Area where to use ethical approaches

- Improving decision-making and innovation →Ethical culture improves innovation by promoting open communication and can make a supportive environment
- Maintaining stakeholder relationships →Ethical culture can demonstrate integrity and so it can lead to stronger partnerships
- Demonstrating corporate social responsibility (CSR) →Companies can improve the world and how people see them

When we talk about ethics we talk about frameworks for organizational behavior thus shaping how relationships work in an organization. Prioritizing ethical principles will make organizations creating sustainable and socially responsible business environment. We can also talk about ethical conduct, so how the company actually acts, but is not strictly morally right.

Some ethical theories

1. **Utilitarianism** →Maximize the happiness of as many people as possible. Actions are morally right if their consequences lead to happiness. Utilitarianism can inform decisions aiming at maximizing overall social utility
2. **Deontology** →We have some moral duties to follow. Actions are right or wrong based on their adherence on moral principles. These principles can guide the behavior in areas like employee treatment.
3. **Virtue ethics** →Focus on virtues will lead to morally good lives. We focus on developing virtuous characteristics like honesty or courage.

Exercise about questions.

Ethical stakeholders

- Individuals →Ethical practices ensure data protection
- Organizations →Ethical data use builds trust
- Society →Responsible data practices promote equity and foster advancements

Key issues in data ethics

- Privacy →Data must be used with the consent of the users
- Fairness →Mitigation of biases to avoid discriminatory outcomes
- Accountability →Holding stakeholders responsible for misuses
- Emerging challenges →Addressing risks in AI

6.3.1 Privacy and confidentiality

Personal data is a cornerstone in digital economies, its misuse can lead to privacy violations and heavy harm on all the actors involved. Two important ways to save these data are **anonymization** (removal of personally identifiable information) and **differential privacy** (addition of statistical noise to the dataset). We define personal data as any information that relates to an identified or identifiable individual. Like basic identifiers, sensitive data, behavioral data and location data.

Personal data have many values

- Economic →Companies can monetize them
- Strategic →Can be used for decision-making
- Technological advancement →ML models rely on them to learn
- Societal insights →From aggregated data we can find social trends
- Personal →Individuals can customize their user experience

The main beneficiaries are

- Businesses →Monetization
- Governments →Data can be used for policymaking
- Individuals →For personalized services

The risks are

- Data breaches →Unauthorized access to sensitive data
- Identity theft →Impersonate someone else
- Surveillance and data erosion →Excessive data collection can lead to loss of privacy
- Exploitation →Companies can target vulnerable individuals
- Lack of consent →If data are collected without proper user consent

Two example of frameworks to save data privacy can be the **GDPR** (General Data Protection Regulation) and the **CCPA** (California Consumer Privacy Act).

To enforce **Personal Data Protection** we have some principles like:

- Transparency →Why data are collected
- Consent →It must be clear, informed and revocable
- Data minimization →Collection of only the necessary information
- Security →Safeguard against breaches
- Accountability →Companies must take responsibility for managing user data responsibly and ethically

Organizations should

- Implement data privacy policies to inform users
- Use encryption to protect stored and transmitted data
- Adopt anonymization and differential privacy to minimize risks →Masking, pseudonymization, generalization, suppression, data swapping and K-anonimity. Noise addition, privacy budget, Laplace and Gaussian mechanisms
- Perform data audits to ensure compliance with legal standards

With data anonymization we can have some challenges to tackle:

- Re-identification risk
- Utility vs privacy

- Linkability

Data anonymization can be used for

- Healthcare
- Banking
- Marketing

Differential privacy has some key features

- Mathematically proven privacy
- Aggregated insights
- Composable privacy

And some benefits

- Strong privacy is guaranteed
- We can share data without violating regulations
- Prevention of re-identification

But also challenges

- Balancing privacy and accuracy
- Complex to implement
- Cumulative noise

Differential privacy can be applied to

- Census data
- Tech platforms
- Healthcare research

Individuals should

- Manage consent
- Use strong passwords
- Be aware of terms
- Secure devices
- Minimize digital footprint

One real example of data breach was the Cambridge Analytica scandal

6.3.2 Bias and fairness in statistical models

A very big problem is the one of bias in data science. It can be caused by many elements and mitigated by

- Audits
- Fairness metrics
- Diverse datasets

The bias can be created

- At data collection →Non-representative samples or historical inequalities
- At model building →Algorithms reflecting biases in training data or design choices

6.3.3 Data sharing and collaboration

Responsible data science must follow certain principles

- Informed consent →Ensure that participants understand data usage
- Data ownership →Clarify who controls the data
- FAIR principles →Promote findability, accessibility, interoperability and reusability
- Cross-border collaboration →International data sharing demands compliance with varying laws and ethical considerations for privacy and sovereignty

Informed consent

Ensures that individuals understand

- What data is being collected
- Why the data is being collected
- How the data will be shared, stored or processed
- Any associated risk

Informed consent empowers individuals and builds trust. It provides autonomy and control over personal data. Transparency strengthens the relationship between data collectors and participants.

Recommendations

- Provide concise summaries for who wants more information
- Use interactive tools to help individuals understand data flows
- Allow individuals to opt-in than requiring opt-out actions
- Regularly update consent forms and notify participants of changes

Challenges

- Information overload →Long TOS can overwhelm participants
- Power imbalances →Individuals may feel pressured to consent when services are essential
- Digital platforms →Ensuring consent on apps and website can be complex due to third-party tracking and unclear data flows

Data ownership

Control, rights, and responsibilities individuals or organizations have over data.

Individuals have some rights

- They own their personal data
- They decide who can access, use or process their data
- They can demand correction, deletion or portability of data

Challenges

- Ambiguity →Who owns derived data or AI insights on personal inputs
- Cross-border data ownership →Ownership becomes complex when data flows across countries with differing laws
- Cloud services →Who own data stored on third-party cloud platforms?

Strategies

- Legal contracts →Define ownership, access and rights in data sharing agreements
- Data governance frameworks →Establish policies for collection, usage and ownership
- Empowering individuals →Use tools like data portability to allow users to move or reclaim their data

FAIR principles

Guidelines to ensure data is managed and shared ethically, promoting collaboration and innovation

- **Findability**→Data should be easy to find via the use of standardized metadata
- **Accessibility** →Data should be easy to be retrieved using certain protocols
- **Interoperability** →Data should integrate easily with other datasets using standardized formats
- **Reusability** →Data should be well-documented for re-use in future research

Benefits

- Ethical collaboration →Supports transparency and trust across institutions
- Efficiency →Reduces duplication of work by promoting reuse of datasets
- Innovation →Facilitates advancements by enabling cross-domain research and analysis

How to apply FAIR principles

1. Use open repositories for data storage
2. Apply machine-readable metadata to make data searchable
3. Use open formats for compatibility
4. Document all the steps of data processing for future reproducibility

Cross-border collaborations Sharing and processing data between institutions or individuals located in different countries.

Opportunities

- Global research →Expertise can be across borders
- Diverse insights →Combining data from different populations
- Innovation →Drives technological and scientific advancements via large-scale collaborations

Challenges

- Legal conflicts →Different countries have different data privacy laws
- Data localization laws →Some countries requires personal data to be stored within their borders

Ethical concerns

- Ensuring informed consent when data moves across jurisdictions
- Avoiding exploitation of populations with weaker protections

Technical barriers

- Variations in data standards, infrastructure and formats

Best practices

- Adopt global standards →Align with GDPR, FAIR and ISO frameworks
- Draft clear agreements →Include clauses for data usage, ownership and compliance with all local regulations
- Secure data transfer →Use encryption and robust security protocols for sharing sensitive data
- Promote transparency →Inform participants how their data will be used across borders

6.3.4 Responsible use of predictive analytics and machine learning

Predictive analytics can have different kinds of societal impacts, both on the negative and on the positive side

- Positive →Enhance healthcare
- Negative →Over-reliance can perpetuate inequality

Over-reliance can cause

- Loss of human oversight →Organizations can be overly dependent on algorithmic recommendations without questioning their problems
- Automation bias →People can over-trust algorithmic outputs
- Context ignorance →Algorithms may fail to account for cultural, societal or organizational nuances

To fight over-reliance we can

- Maintain a human-in-the-loop approach →Ensure that critical decisions have human review
- Algorithm audits →Regularly test algorithms for bias, performance and unintended consequences
- Empower human judgement →Train employees to critically interpret and challenge algorithmic results

Algorithms entail by design some flaws. Some targets that we want to reach are the maximization of fairness and transparency and the minimization of the effects of bias.

Some ethical dilemmas that arise in this context of automation are

- The impact on employment →Automation may displace workers, creating economic inequalities and societal tensions
- Accountability of the decisions →Who is responsible for decisions made by an algorithm?
- Risks for privacy →Predictive analytics often require large amounts of data, raising concerns about consent, data ownership and misuse

Some examples of success stories in automation yadda yadda.

Guidelines for responsible use

- Ethical frameworks →Adopt principles like fairness, transparency and accountability for designing and deploying ML systems
- Stakeholder involvement →Engage affected parties in assessing ethical risks and impacts
- Regular bias assessments →Conduct audits to identify and address hidden biases in data or algorithms
- Explainability and documentation →Design models that allow stakeholders to understand how decisions are made. Provide clear documentation of data sources, training process and known limitations
- Impact assessments →Evaluate social, ethical and environmental impacts of deploying ML systems

6.3.5 Transparency and accountability

Transparency and accountability are two very important elements if we want to ensure that our discipline is ethical, trustworthy and aligned with societal values. As Data science and AI grow companies should adopt strategies and frameworks to explain how models work, hold stakeholders accountable and establish governance structures that ensure responsible data use.

Explainability and transparency are important because we want to be able to understand why a decision was made and to go back if we find some bias. The challenges to take care of are

- ML models usually are black boxes, we have difficulties explaining how they work
- We lack data provenance, we miss documentation of data sources, transformations and biases. These can create transparency gaps
- Stakeholders might not be able to understand how the models work

Explainability can be obtained by

- Add interpretability layers →SHAP, or LIME
- Add transparency during the collection of the data →documentation on data sources, inclusion criteria and any transformation applied to them
- Communicate effectively with the stakeholders →translate technical outputs into understandable terms for business leaders, customers and regulators

To ensure accountability we can work with different frameworks

1. Accountability mechanisms →audits and impact assessment
2. Regulatory compliance →data protection laws, algorithmic accountability acts and cultural accountability
3. Governance models →Ethics committee, cultural principles (ISO and NIST), stakeholder involvements, building trust, risk mitigation and regulatory compliance

Practitioners can be recommended to do

- Adopt explainability by design →apply these frameworks at the beginning
- Conduct audits regularly →Establish a cadence for the reviews of datasets, models and outcomes
- Engage with stakeholders →Include diverse groups in decision-making processes to ensure that the models align with societal values and needs
- Establish clear governance policies →Implement formal structures like data ethics charters or committees to oversee ethical considerations

6.3.6 Ethical decision-making process

Now we see some courses of action to make ethical decisions.

Structured framework for individuals and organizations to evaluate ethical dilemmas and determine the most appropriate course of action

1. Recognize the ethical issue →identify the ethical dilemma or issue at hand. We can have conflicts of values, interests or obligations
2. Gather relevant information →Collect all pertinent facts, data and information related to the dilemma. Consider the perspective of the various involved stakeholders
3. Evaluate alternative actions →Generate a range of possible courses of action to address the ethical issue. Assess the potential consequences, risks and benefits of each option
4. Make a decision →choose the right course of action that aligns with ethical principles, values and standards. Consider both long and short-term implications (ethical or not)
5. Implement the decision →Put the chosen course of action into practice, communicate the decision to the stakeholders and ensure that the implementation is effective
6. Reflect on the outcome →Reflect on the impact of the decision, and it's impact on the stakeholders. Evaluate whether the chosen course of action was effective in addressing the ethical dilemma

Kohlberg proposed a theory of moral development divided into 3 levels and 6 steps

Level I: Pre-conventional morality

1. Obedience and punishment orientation → maintain personal safety
2. Instrumental relativist orientation → gain rewards

Level II: Conventional morality

3. Good interpersonal relationships → conforming to societal norms
4. Maintaining social order → promote stability

Level III: Post-conventional morality

5. Social contract and individual rights → greater good
6. Universal ethical principles → conscience

The application of the kinds of ethics can be done on certain parts of our previously defined framework

- Utilitarianism →focus on the outcomes, we look for actions that mitigate harm and maximize benefits. We might justify morally questionable means.
- Deontology →we want to ensure that our moral principles are respected. We obtain clear and on-negotiable ethical boundaries. We can be stuck if we are too rigid, consequently we will disregard possible beneficial outcomes if we don't want to compromise on certain principles
- Virtue ethics →We accept implementations delays if it means that we are respecting ethical principles. This can lead to trust between the organization and the stakeholders. The decision-making process will be values-based but nonetheless subjective

In substance Utilitarianism is to evaluate the situation, Deontology to ensure that ethical boundaries are not crossed and virtue ethics to guide to actions aligned with the organization's identity and principles.

6.3.7 Promoting ethical culture

Organizations should promote an ethical culture for different reasons, both towards their public image and towards a better way to organize towards themselves.

- To communicate expectations →Clearly communicate expectations about ethical standards policies to all employees
- To lead by example →Demonstrate ethical behavior and values at all levels of the organization, starting with top leadership
- To foster an open communication →Create an environment where employees feel comfortable raising ethical concerns, seeking guidance and reporting misconduct without fear of retaliation
- To align incentives and rewards →Ensure that reward systems and incentives align with ethical behavior and performance, reinforcing the commitment to ethics
- To provide training and support →Offer ethical training, resources and support mechanisms to navigate ethical dilemmas and make informed decisions.

In this setting we can identify three core elements

1. Values →identify the core principles and beliefs that guide the decision-making. They are made of alignment, reinforcement and role modelling
2. Norms →recognize unwritten rules with respect to what is acceptable or not. Key factors are socialization, peer influence and norm enforcement
3. Leadership →assess the behavior and set the tone for the culture of the organization. Composed of tone at the top, decision-making practices and accountability

Ethical values must be communicated, usually this is done in two main ways:

- By example →ethical leaders should demonstrate and embody ethical behavior in their actions, decisions and interactions
- By training and education →Provide ethics training and education programs to help employees understand the organization's ethical expectations and develop the needed skills

The development of ethical policies is done by

- Whistleblower protection →Implement processes that can protect whistleblowers when they have to report wrongdoings done by the company
- Ethical decision-making tools →provide employees with resources and tools like decision-making frameworks and ethical guidelines to support them in making ethical decisions

The promotion of open communication and transparency is done by

- Encouraging dialogue →foster a culture of open communication where employees can feel free of discussing ethical concerns, seeking guidance and reporting misconduct
- Transparency about decisions →Be transparent about organizational decisions, policies and practices
- Enforcement of feedback mechanisms →Establish feedback mechanisms like anonymous surveys or suggestion boxes

To recognize ethical conduct

- Performance evaluation →Apply a check of these ethical behaviors when evaluating the performance of the employees
- Promotion of ethical leaders →Promote individuals who demonstrate ethical leadership qualities. This to reinforce the importance of certain characteristics

Enforcement of ethical standards and accountability

- Consistent enforcement →continuously enforce the standards with consistence and fairness. All employees should be held accountable
- Consequences for violations →Establish consequences for violations, misconducts to ensure that the right disciplinary action is taken when necessary
- Ethical oversight →Implement mechanisms for ethical oversight and governance. Committees and review boards can be established

To improve and adapt over time

- Regular evaluation →Evaluate the ethical culture regularly
- Adaptation to change →Adapt policies to reflect changes at organizational and societal level
- Learning from ethical lapses →Learn from past errors, conducting investigations and implement corrective actions

Ethics in organizational process

- Integration into decision-making →Integrate ethical considerations into organizational decision-making to ensure that ethical implications are systematically evaluated
- Embedding in organizational structure →Embed ethics into the organizational structure, governance mechanisms and performance management systems
- Alignment with stakeholders expectations →Align the ethical culture with the expectation and values of its stakeholders

6.3.8 Ethical leadership

Ethical leadership is a form of leadership characterized by integrity, honesty, transparency and commitment to ethical principles and values. Its basic characteristics are

- Integrity →consistency between words and actions, acting in accordance with moral principles and values
- Honesty and transparency →Ethical leaders communicate openly and honestly with their followers, providing accurate information and avoiding deception and manipulation
- Fairness and justice →Ethical leaders treat all individuals with fairness, dignity and respect, regardless of their background
- Empathy and compassion →Ethical leaders must understand the perspective of their employees
- Accountability →Ethical leaders should take responsibility for their actions and decisions
- Courage →Ethical leaders should demonstrate courage by standing up for what is right
- Ethical decision-making →Ethical leaders must engage in ethical decision-making considering the possible implications and consequences of their actions

- Vision and purpose →Ethical leaders should inspire and motivate their followers promoting the greater good
- Ethical culture promotion →Ethical leaders actively cultivate an organizational culture that values and prioritizes ethics, integrity and creates mechanisms for ethical oversight and governance
- Continuous learning and improvement →Ethical leaders are committed to personal and professional growth. They also encourage a culture of learning and reflection

Practicing ethical leadership involves examining the obstacles that leaders may encounter in upholding ethical standards and promoting integrity, as well as identifying practical approaches to address these challenges.

Challenges

- Balancing competing interests →Leaders often face competing interests which can create dilemmas
- Pressure to compromise ethics →Leaders may face pressure to compromise ethical principles in pursuit of short term gains
- Ambiguity and uncertainty →Ethical dilemmas may involve complex and ambiguous situations with no clear rights and wrongs
- Resistance to change →Employees and stakeholders might resist changes
- Ethical blind spots →Leaders may have blind spots and biases when evaluating implications
- Ethical dilution in hierarchies →Ethical standards may become diluted or distorted as they cascade through hierarchical levels

Strategies to beat these challenges

- Clarify ethical values and standards →Articulate the ethical values in a clear way
- Lead by example →Demonstrate integrity, honesty, transparency and fairness
- Communicate ethical expectations →Communicate and reinforce ethical expectations through formal policies
- Encourage open dialogue →create an environment where employees feel comfortable raising ethical concerns
- Provide ethical decision-making support →Offer training, resources and support to help employees navigate ethical dilemmas
- Empower employees →Employees should be able to speak up and take action when they encounter unethical behavior or problematic situations
- Foster a culture of trust and accountability →Build trust and credibility among employees by demonstrating consistency, fairness and accountability
- Seek feedback and continuous improvement →solicit feedback from employees, stakeholders and external observers
- Address ethical lapses promptly and transparently →address ethical lapses or misconduct promptly and transparently
- Invest in ethical leadership development →Invest in the development of ethical leadership skills and competencies

6.4 Theranos & Black Mirror

I don't know, we write the essay on these things I guess.

If it's an essay.

I don't know.

7 Computational intelligence

7.1 Introduction

Definitions to start the course

- **Problem** →Task to be solved automatically (find the maximum number in a list)
- **Algorithm** →Process to solve a problem, set of precise and not ambiguous actions that all together will allow us to reach the solution
- **Program** →Translation of an algorithm into a programming language

Motivation

- In classical computational methods we start from a problem, and we have a human being that imagines an algorithm (problem-solving, it's creative and informal). Then software engineers will put the ideas into a computer in order to get a program (implementation, it's automatic and not creative).
- We need computational intelligence because traditional methods in many cases fail, usually on the first step. We can have many problems where imaging solutions or algorithms it's either impossible or very hard for human being to imagine a solution. This is the case of **complex problems**

Some examples of complex problems

- Categorization of clients
- Face recognition
- Find a new drug to cure a disease
- Driving a robot in a 3-D space

Our idea is to give computers or informatic systems the ability to learn autonomously how to solve a problem. We define **learning** as improving something by means of experience. Those computers can also understand that the generated solution in not good enough, and so they should understand a better way to find the optimal solution.

One of the most famous ways to improve those computers are bio-inspired algorithms. Like the **neural networks** (took inspiration from the brain), **evolutionary computation** (theory of evolution from Darwin, considering evolution as a sort of learning) where we have genetic algorithms, differential evolution or genetic programming, **swarm intelligence based algorithms** (system of animals are better at solve problems with respect to singular entities) like particle swarm optimization, **fuzzy systems** and **local search** like hill climbing simulated annealing.

One category of complex problems are the so-called optimization problems. Even though they are complex they are easy to comprehend and to analyze.

Informally solving an optimization problem means to find the best solution(s) in a typically huge set of possible alternative solutions. We are able to find if something is a solution and to evaluate the quality of the solution with respect to the others. More formally an optimization problem is a pair of objects S and f where

- S →set of all existing solutions (search space)
- f →function that evaluates the solutions (fitness or cost or quality function)

An optimization problem can be:

$$\textbf{Maximization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \geq f(y) \quad \forall y \in S.$$

$$\textbf{Minimization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \leq f(y) \quad \forall y \in S.$$

To solve this kind of problems we can use an optimization algorithm, a machine that is able to output multiple solutions (iterative machine).

No free lunch theorem →the average performance of all optimization algorithms calculated on all existing optimization problems is the same. We care about the probability of finding the best solution and the time to reach it. We can't have one algorithm for every problem then we need to find a solution for every problem.

7.2 Optimization algorithms

Optimization problem →a pair (S, f) where

- S is the set of all solutions (search space)
- f is the function (fitness function) where f : S →$\mathbb{R}$

An optimization problem can be:

$$\textbf{Maximization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \geq f(y) \quad \forall y \in S.$$

$$\textbf{Minimization:} \quad \text{find } x^* \in S \text{ such that } f(x^*) \leq f(y) \quad \forall y \in S.$$

x can be either a global optimum or a local one, we want to find the global(s) optima.

To find the solution we will use some optimization algorithm, by definition an optimization algorithm is an iterative method that at the end of each iteration returns a solution ∈ S. One of the usual algorithm is **Random search**.
As already seen we have the no free lunch theorem that stops us from finding a perfect solution.

No Free Lunch Theorem →Let A_1 and A_2 be two optimization algorithms. When performance is averaged uniformly over all possible objective functions, A_1 and A_2 have identical expected performance.

The consequences of this theorem are that

1. It is impossible to have an algorithm that is better than all the others and on all possible problems (no super algorithm can exist).
2. Given a problem it can't exist an automatic method to find the best algorithm. Then we need to test, experiment and use our prior knowledge.

Now we will see some algorithms. We start with **Hill climbing**. Before anything we need to clarify that we can't find every possible solution, so we need to use certain starting points.

Neighborhood. Let $N : S \to 2^S$ be a neighborhood function that maps a solution $x \in S$ to a set of neighboring solutions $N(x) \subseteq S$.
Starting from an initial solution x_0 (typically chosen at random), the objective function is evaluated over $N(x_k)$, and the best neighbor is selected:

$$x_{k+1} = \arg \min_{y \in N(x_k)} f(y)$$

(for a minimization problem).
This process is repeated iteratively until a stopping criterion is met.

Example

- S = students in the room
- f = student number
- Maximization problem
- Neighborhood = the students around the selected student

Local optimum →given an optimization problem (S, f) a local optimum is a solution x such that $\forall y \in N(x), f(x)$ is better than $f(y)$. By definition hill climbing will return a local optimum with no guarantee that it will also be global. We can find the global optimum by expanding the radius or by trying more initialization

In pseudocode (general case)

1. Initialize the current solution x (typically at random).
2. Repeat

2.1 Generate a neighbor y of x.

2.2 If $f(y)$ is better than or equal to $f(x)$, set $x := y$.

Until for all neighbors y of x, $f(y)$ is worse than $f(x)$.

3. Return x.

In strict hill climbing we change the point only if the fitness is strictly better than the previous. With this setting we can have an infinite iteration between 2 equal points. We can set a timeout if we see the same points n times.

Taboo search can help the search by using memory, avoid the already checked points.

Example

- $S = \{i \in \mathbb{N} : 0 \leq i \leq 15\}$
- f(i) number of 1s in the binary code of i
- Maximization
- Neighborhood = i and j $\iff$ |i - j| = 1

We produce a plot

- X →all solutions produced ordered by their neighborhood
- Y →all the fitness values

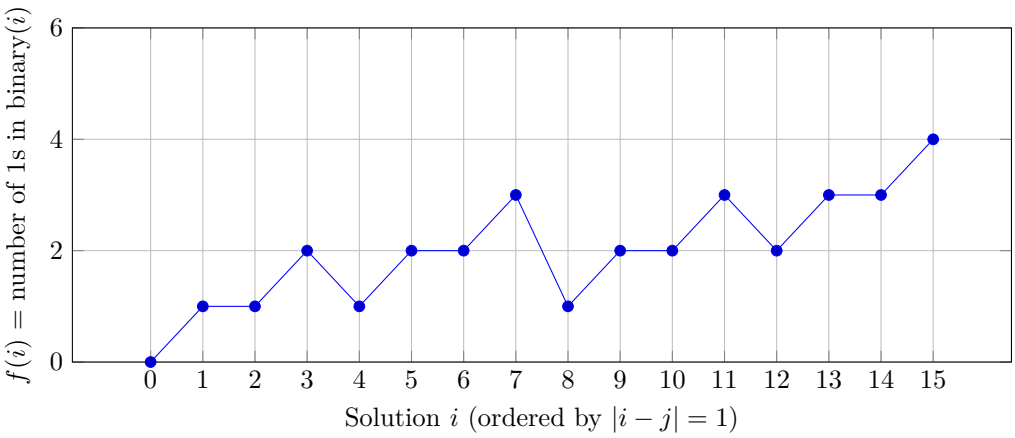


Figure 1: Fitness landscape for $S = \{0, \ldots, 15\}$ with neighborhood $|i - j| = 1$.

With the fitness landscape we can find the difficulty of the problem. A very smooth one is usually of a simple problem while a very oscillating one will be probably more difficult.

7.3 Fitness landscape

In the last class we saw what is a fitness landscape. The plot in the 2-D setting is obtained by plotting on the axis

- Horizontal →All individual in the search space sorted according to the neighborhood
- Vertical →the fitness

Given an optimization problem (S, f) and a neighborhood N, a solution $x \in S$ is called a local optimum if $\forall y \in N(x)$ f(y) is worse than f(x). By construction hill climbing will return a local optimum with no guarantee that it will also be a global optimum.

The fitness landscape gives a visual idea of how easy/hard is for an algorithm to solve the problem

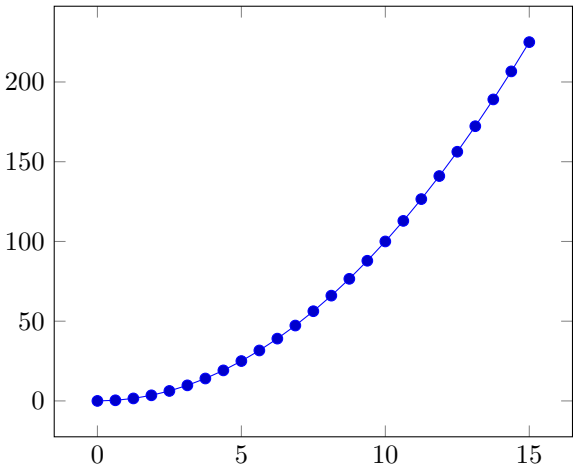
- Smooth →easy

- Rugged →hard

But in practice we can't draw the landscape because the search space is humongous, and the neighborhood is usually multidimensional.

Example

- $S = \{i \in \mathbb{N} | 0 \leq i \leq 15\}$
- f(i) = i^2
- N = The solutions x and y are neighbors $\iff |x - y| = 1$
- Maximization



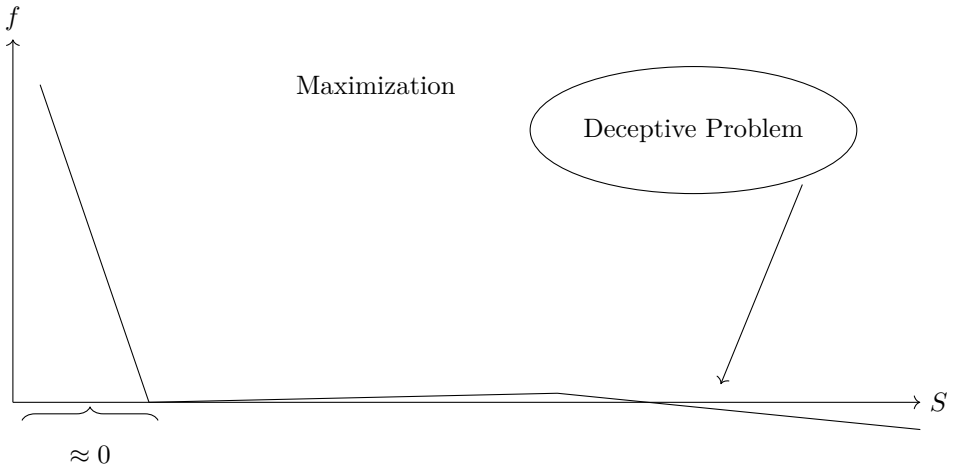
Here hill climbing will arrive at the global optimum.

If we want to look for the number of 1 in the binary code of i we can encounter problems and stay stuck in a local optimum without being able to find the global one.

The definition of neighborhood is not fixed, we can define it in 2 ways.

- N(x) →{ $y \in s | d(x, y) \leq k$} →distance based
- N(x) →{ $y \in s | y = op(x)$} →action based

Example



Random search is "good" for this type of problem.

Hill climbing pros

- Simple
- Efficient
- Versatile

Cons

- Always stops on local optima

How to improve hill climbing

- Parallel hill climbing
- Enlarge the neighborhood
- Give the possibility of worsening the fitness of the current solution (within a given probability)

7.4 Annealing

We now introduce the concept of annealing →an experiment aimed at finding the solid state of a material with the minimum level of energy.

Metropolis algorithm

1. Consider the initial state of the material with energy E_i
2. Alter the state to get a new energy E_j
3. If $E_j \leq E_i$ accept the new state and continue
4. Else accept the state with probability $exp(-\frac{|E_i - E_j|}{K_B T})$

Where

- T = Temperature
- K_B = Boltzmann constant

Simulated annealing

- Current solution = x
- Neighbor = y

$$P(y) = \begin{cases} 1 \text{ if f(y) better or equal than f(x)} \\ exp(-\frac{|f(x) - f(y)}{C}) \text{ otherwise} \end{cases}$$

Where C is a control parameter

7.5 Practical applications of Hill climbing

Test case 1 →Given a set of possible investments (N), for each investment we know

- Cost
- Estimated gain

We also know the budget of the client. What subset of the investments should our client do?
Such that

- The total cost is smaller or equal to the budget
- The profit is maximized

Using a numeric example

- N = 10
- Costs = {23, 31, 29, 44, 53, 38, 63, 85, 89, 82 }
- Profits = {92, 57, 49, 68, 60, 43, 67, 84, 87, 72 }
- Budget = 165

Questions to answer

1. What are the solutions ?
2. How it is convenient to represent the solutions?
3. What's the fitness?
4. If the algorithm is neighborhood based, then what is the neighborhood?

Golden rule →**DO NOT solve the problem**
Answers

1. All the subsets of the set of investments
2. As sequences of values, a string of bits of length N. The array will have a pair of cost and profit. We use 0s and 1s to "1-hot encoding" them.
3. The sum for all the indexes where x is 1 of all the profits. $\sum_{i:x[i]=1} p[i]$. We have a maximization problem. We only care about solutions that have a cost less than the budget. The fitness if this condition is not accepted we have -1 or 0 as fitness. Another way to define a non-good fitness is to use the negative sum of the costs such that the distance ourselves as much as possible from a bad solution.
4. The neighborhood is obtained here using the bit flip operator.

Test case 2 →Travel-sale-person: given N cities for which we know all the pairwise distances between themselves. Give one of these cities there is one specific one that is called home. We want the cyclic path that starts from home, visits all the cities once and only once and terminates at home using the shortest possible path.

We start from a matrix

- N = 5 →the matrix is 5x5
- Inside the matrix we have the costs/distances

Answers

1. Solutions →All cyclic paths starting and finishing in home and visiting all cities once.
2. Representation →An ordered string of the cities
3. Fitness →the sum of the distances in the string
4. Neighborhood →We can use a swapping operator

7.6 Simulated annealing

Simulated annealing is similar to hill climbing, but we add the concept of probability. This can lead to a worse fitness. The idea is that we can explore more possible solutions and obtain a better optimum. But we can also miss some of those. Sometimes can be convenient that our current solution is a random neighbor.

We can start by defining our current solution x and our candidate neighbor y.
The probability $P(y)$ is defined as:

$$P(y) = \begin{cases} 1 & \text{if } f(y) \leq f(x) \\ \exp\left(-\frac{|f(x)-f(y)|}{C}\right) & \text{otherwise} \end{cases}$$

Where C is a control parameter $C > 0$.

Afterward, we look for a random number in the interval $[0, 1]$ and if this number is $< P(y)$, we accept the move.
We start with a large C, and we decrease it during the algorithm. C will tend to 0 without reaching it, following the cooling schedule:

$$C = \frac{C}{h}$$

Simulated annealing follows the following structure

1. Initialize the current solution x (typically at random)
2. Initialize C and L
3. Repeat the following until termination
   - Repeat L times
     - Create a random neighbor x of y
     - Accept or not y with a certain probability
   - Update C (not mandatory update L)
4. Return x (or the best solution so far)

We terminate when we have found a certain condition that can be either a good enough solution or we have run a prefixed maximum number of iterations of the external loop.

Example

- S = i $\in \mathbb{N} | 0 \leq i \leq 15$
- $\forall i \in S, f(i)$ = number of 1s in the binary code of i
- Maximization
- Neighborhood →x and y are neighbors $\iff |x - y| = 1$

Step 1: Initialization
$x = 5$ (random initialization) $\rightarrow bin(5) = 101$
$f(x) = 2$
$N(x) = \{4, 6\}$
Assume $y = 6$ (random neighbor) $\rightarrow bin(6) = 110$
$f(y) = 2$
Since $f(y) \leq f(x)$, we accept the move: $x = 6$.

Step 2: Iteration
$x = 6$
$N(x) = \{5, 7\}$
Assume $y = 7$ (random neighbor) $\rightarrow bin(7) = 111$
$f(y) = 3$
Since $f(y) > f(x)$, we calculate the acceptance probability.

Step 3: Probability Calculation
Assume in this phase the control parameter is $C = 1$.
Current state: $f(x) = 3$
Candidate neighbor: $f(y) = 1$

$$P(acc\ y) = \exp\left(-\frac{|3-1|}{1}\right) = e^{-2} \simeq 0.13$$

This value represents a *"large" probability* for this phase of the algorithm.

Theorem of asymptotic convergence of simulated annealing

Given an optimization problem (S, f) and an appropriate neighborhood, if we decrease C so that it tends to 0 (without reaching it):
As $t \to \infty$ and $C \to 0$, we analyze the convergence:

$$\lim_{C \to 0} q_i(C) = \frac{1}{|S_{opt}|} \cdot g(i)$$

Where:

- $q_i(C)$ is the probability that Simulated Annealing (SA) will stabilize on solution i with control parameter C.
- S_{opt} is the set of global optima.

- $g(i)$ is an indicator function defined as:

$$g(i) = \begin{cases} 1 & \text{if } i \in S_{opt} \\ 0 & \text{otherwise} \end{cases}$$

This limit shows that as the temperature C approaches zero, the probability of being in a state i becomes uniform over the set of global optimal solutions and zero elsewhere. To be sure to find a global optimum we should execute the process a very large number of times.

The parameters L and C are respectively Large and slow (decreasing)

7.7 Genetic algorithms

Genetic algorithms share similarities with simulated annealing

- Both are optimization algorithms
- Both allow to look for good solutions without an exhaustive analysis of the search space
- Both start from random solutions and improve their quality iteratively

The differences on the other hand are

- The solutions (individuals) that are considered at a time are more than 1 (population)
- Inspired by the theory of evolution of Charles Darwin
- Individuals must be represented by strings of constant length

The theory of evolution has the following key elements

- Reproduction
- Ability of adaptation
- Inheritance
- Variation
- Competition

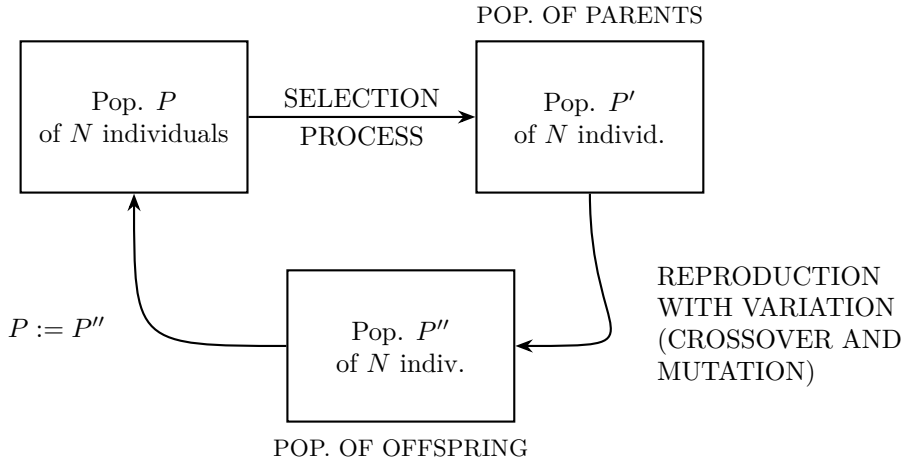
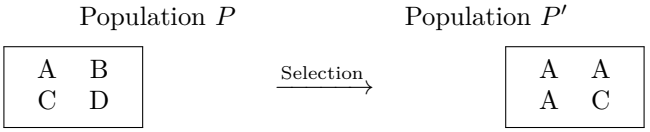


Figure 2: Genetic Algorithms Cycle

The **Selection Process** maps the initial population P to the intermediate population P' (parents):



In this example, individual **A** is selected three times due to its high fitness, **C** is selected once, while **B** and **D** are discarded.

The selection process is made of iterations of N independent executions of a selection algorithm. The selection algorithm chooses 1 solution from a population P.
The algorithm has the following general principles

1. It's probabilistic
2. Given any pair of independent A and B in P if f(A) is better than f(B) then A has a bigger probability than B of being selected
3. No individual in P has a probability equal to 0 of being chosen

Fitness Proportionate (Roulette Wheel) Selection
Given N individuals in population $P = \{1, 2, \ldots, N\}$, for each $i = 1, 2, \ldots, N$ we define the probability of selection as:

$$P(sel\ i) = \frac{f_i}{\sum_{j \in P} f_j}$$

Mechanism ($N = 4$):
This method acts like a roulette wheel where the area of each slice i is proportional to its fitness f_i. Individuals with higher fitness occupy a larger portion of the wheel, increasing their chances of being selected when the wheel is "spun" (represented by a random number $U \in [0, 1]$).

- i_1, i_2, i_3, i_4: Individuals in the population.
- U: Random pointer used to select the individual.

Other ways that can be used are **Ranking** and **Tournament** selection.

7.8 Again Genetic algorithms

We forgor something about simulated annealing. The theorem that we talked about miss something.
Basically we can only use simulated annealing in an appropriate algorithm, but we didn't define what we meant by appropriate.

From the part of asymptotic convergence. We must have a certain property →given any pair of solutions x and y it must be possible to build a sequence of solutions $a_1, a_2, ..., a_n$ such that $\forall i = 1, 2, 3, ..., n - 1, a_i$ and $a_{i+1}$are neighbors. So $a_1 = x$ and $a_n = y$. We say that the neighborhood is completely connected.

Now going back to genetic algorithms.
We start with a population P of N individuals. N is a hyperparameter. We modify the population with two steps.

- Selection phase →We have a population P' of N individuals
- Reproduction with Variation phase →We have a population P" of N individuals

The new current population will be P" and we repeat the two phases until we reach a stopping criterion, and we select as solution the best individual in the final P.
We create new individuals (offspring) via means of crossover and mutation.

The idea behind this algorithm comes from the Darwinian ideas of adaptation, competition, reproductions, inheritance and variation. The first two are for the first phase, while the other 3 for the second one. The selection phase is an iteration of N independent executions of a selection algorithm where the objective is to choose one solution from a population of N solutions.

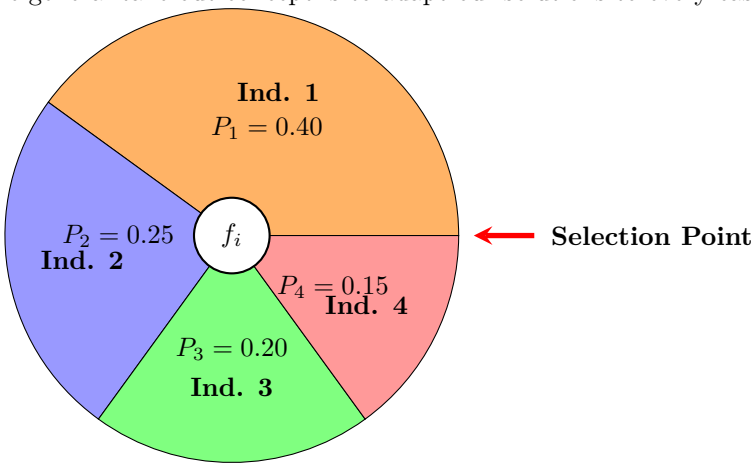
Is important that the selection algorithm is probabilistic since we want to have different solutions every time. The probability of selecting a solution must be based on fitness. All individuals must have a non-zero probability of being selected.

In this course we will talk about 3 main selection algorithms

7.8.1 Fitness proportionate selection

Also known as roulette wheel. We have a population of N individuals called 1, 2,..., n. For each i in 1, 2,..., n we have that the probability of selecting i we have $P(i) = \frac{f(i)}{\sum_{j \in P} f(j)}$. This is a maximization only algorithm, for minimization we can do 1 divided by fitness but if we have 0 fitness we use to add a constant to every possible value.

The general take out concept is to adapt our solutions to every case.

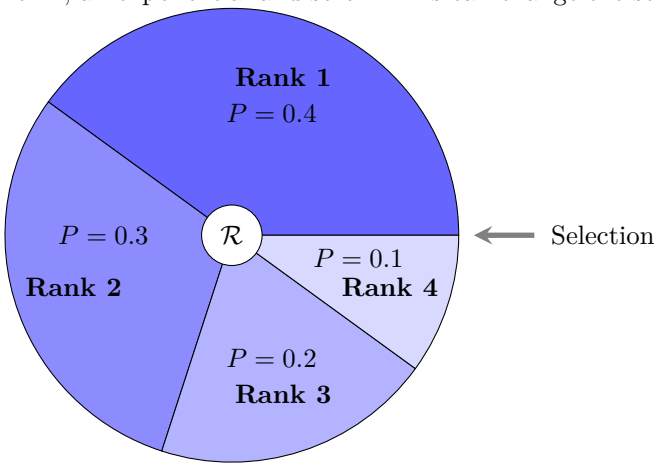


7.8.2 Ranking selection

We follow this procedure

1. Sort the individuals from the worst to the best based on fitness.
2. We calculate the probability selection based on the indexes. So $P(i) = \frac{\text{index}}{\sum \text{indexes}}$

This algorithm is not sensitive to fitness differences, we don't care about how much a certain individual is better than another one. We can generalize this algorithm by changing the function (now is linear) to have a logarithm, an exponential and so on. This can change the selection pressure.

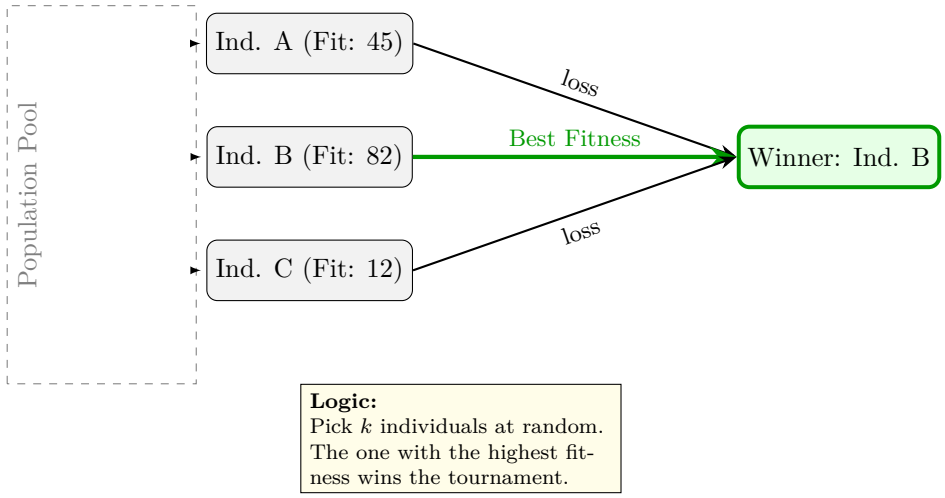


7.8.3 Tournament selection

We follow the following schema (To draw).
This can be considered as both random and deterministic.

1. It is possible (in some cases) to not evaluate all the possible individual's fitness.
2. We can change the selection pressure by changing the tournament size

Tournament Selection ($k = 3$)



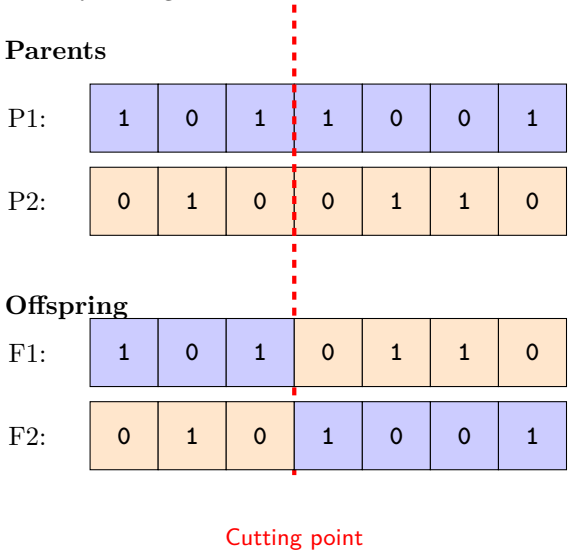
Logic:
Pick k individuals at random. The one with the highest fitness wins the tournament.

7.9 Crossover and mutation

Now we talk about the part about reproduction with variation. Selection depends only on phenotype while crossover and mutation only on genotype (DNA structure).

7.9.1 Crossover

We start by taking two solutions that are in P'.



This example is for 1-point crossover, but we can have multiple cutting points.

7.9.2 Mutation

For each character we change it with a given probability of mutation (another hyperparameter)

7.9.3 General function of genetic algorithms

Here we have a pseudocode on the general idea of genetic algorithms

1. Initialize a population P of N individuals (typically at random)
2. Repeat until termination condition is reached:
   (a) Calculate the fitness of all the individuals in the population P (can be avoided in some cases, like tournament selection). This step is computationally costly

(b) We create an empty population P'

(c) Repeat P' has N individuals

i. We choose a genetic operator (crossover with probability P_c or replication with probability $1-P_c$)

ii. Select 2 individuals from population P

iii. Apply the operator to the individuals

iv. Apply mutation to the individuals resulting from the previous step

v. Insert the individuals resulting from the previous point into P'

(d) Replace P with P'

3. Return the best individual in P in terms of fitness

Some points

- If N is odd:
  - Select 1 individual mutated and insert it into P'
  - One crossover event copies into P' only one offspring (typically the best in fitness)
- Elitism is the unchanged copy of the best K individuals in P into P' (bypassing the selection-crossover-mutation pipeline), typically K=1
- We have to do something like P →selection →P' →crossover + mutation →P" →P := P"
- Replication is the unchanged copy of the parents
- Termination condition is either
  - A good enough solution is found, and we return it
  - A maximum number of generation was performed

7.9.4 Hyperparameters of genetic algorithms

- String length →Given by the problem definition
- Population size (N) →The bigger, the better. We have an upper bound (true for every problem)
- Selection algorithm that we want to use →No free lunch holds, but tournament selection can be interesting to use (fast, and we can use the selection pressure). We can start with this and try the others. The tournament size should not be too big (2 can be good, and the minimum).
- Crossover probability (P_c) →Should be close to 1
- Mutation probability (P_m) →Should be close to 0
- Maximum number of generations →The bigger, the better but with an upper bound like for N
- Elitism on or off (and if on size of the elite and k) →Elitism should be used with K=1

7.10 Why genetic algorithms work

To answer we have to have another question →Will genetic algorithms lead to the discovery of a global optimum or an approximation of it?

To answer we have to have another question →How do genetic algorithms work?

To answer we have to have another question →What is the information that is maintained and processed in the population?

The hypothesis that we want to verify is that there are some syntactic features that give a boost to fitness. We would like to maintain these features through generations. To do this we would like to formalize a mechanism to identify structural similarities between individuals. We can call it **Schema** (set of individuals that have in common the fact of having the same characters in some positions).

A schema can be represented using strings composed by all admissible characters + 1 special character (don't care symbol, here is *). For example in the binary case, a schema is a string built using characters coming from the set {0, 1, *}.

Example
{*0000} represents both {00000} and {10000}.

Hypothesis →implicitly genetic algorithms work on schemata.

- Order of a schema →The number of characters different from *, o({011*1**}) = 4
- Length of a schema →The difference between the rightmost and leftmost positions that do not have an *, δ(011*1**) = 5 - 1 = 4

Now we want to know what schemata will survive in the population over the evolution of a genetic algorithm.

At generation t, m(H, t) is the number of individuals in the population that belong to the schema H. Given this we want to know what is m(H, t + 1). To solve this we need to know how selection, crossover and mutation change m(H, t).

7.10.1 Effect of selection on schemata

We will study the case of fitness proportionate selection.
$m(H, t+1) = m(H, t) n \frac{f(H)}{\sum_j f_j}$
Where f(H) is the average fitness of the individuals in the population that belong to schema H. In the denominator we have the average probability of selecting an individual belonging to schema H. We apply selection n times.

$$\text{AVG. FIT. OF INDIVS IN POP} = \tilde{f} = \frac{\sum_j f_j}{n}$$
$$\frac{1}{\tilde{f}} = \frac{n}{\sum_j f_j}$$
$$m(H, t+1) = m(H, t)\frac{f(H)}{\tilde{f}}$$

Schemata with an average fitness that is better than the average population fitness will increment the number of representatives and the others will decrement. At what speed?

$$\text{Assume } f(H) = \tilde{f} + c\tilde{f}$$
$$m(H, t+1) = m(H, t)\frac{\tilde{f} + c\tilde{f}}{\tilde{f}} = m(H, t)\frac{\tilde{f}(1+c)}{\tilde{f}}$$
$$= m(H, t)(1+c)$$

$$m(H, 1) = m(H, 0)(1+c)$$
$$m(H, 2) = m(H, 1)(1+c) = m(H, 0)(1+c)^2$$
$$m(H, 3) = m(H, 2)(1+c) = m(H, 0)(1+c)^3$$
$$\vdots$$
$$m(H, t) = m(H, 0)(1+c)^t$$

The growth of the number of representatives of schema H in the population is exponential with t.

7.10.2 Effect of crossover on schemata

EXAMPLE

$$A = 0\ 1\ 1 \mid 1\ 0\ 0\ 0$$
$$H_1 = *\ 1\ *\ \mid\ *\ *\ *\ 0$$
$$H_2 = *\ *\ *\ \mid 1\ 0\ *\ *$$

FOR ANY SCHEMA $P_s(H) = 1 - \frac{\delta(H)}{L-1}$

$$P_d(H_1) = \frac{5}{6} = \frac{\delta(H_1)}{L-1}$$
$$P_s(H_1) = 1 - \frac{5}{6} = \frac{1}{6} \implies 1 - \frac{\delta(H_1)}{L-1}$$

IN GENERAL

$$P_s(H) \geq 1 - p_c \frac{\delta(H)}{L-1}$$

7.10.3 Combined effect of selection and crossover

$$m(H, t+1) \geq m(H, t) \frac{f(H)}{\tilde{f}} \left[1 - p_c \frac{\delta(H)}{L-1}\right]$$

Schemata with:
– The average fitness is better the population average
– The small length will increment the number of representatives exponentially

7.10.4 Effect of mutation

The probability that 1 bit will not be changed = 1 - p_m

The probability of a schema surviving mutation =

$$\underbrace{(1-p_m)(1-p_m)(1-p_m)\dots(1-p_m)}_{o(H) \text{ times}}$$

$$P_s(H) = (1-p_m)^{o(H)} \qquad \begin{array}{l}\text{if } p_m \ll 1, \text{ then} \\ P_s(H) \approx 1 - o(H)p_m\end{array}$$

Schema theorem

$$m(H, t+1) \geq m(H, t) \frac{f(H)}{\tilde{f}} \left[1 - p_c \frac{\delta(H)}{L-1}\right] [1 - o(H)p_m]$$

We have schemata that:

- Have an average fitness better than population average
- Are short (small $\delta(H)$)
- Have a small order (small $o(H)$)

will increase the number of representatives exponentially

7.10.5 Building blocks' hypothesis

This hypothesis tells us that maintaining the building blocks in the population and making them increment exponentially will

- Maximize our chances of finding a global optimum
- Make the probability of finding a global optimum tend to 1 with time

7.10.6 K-armed bandit problem

This looks like something random but apparently is related to genetic algorithms.

Vanneschi just trashed Samuel in real time.

- Given N slot machines
- Every slot machine j has a number of arms K_j
- We assume that we already played t times and for each of this t games we know
  - The slot machine in which we played
  - The result of the game (win or lose)
- We want to find to which slot machine we should allocate our next attempts to maximize our probability of winning

Given the slot machine j, we call S_j the probability of winning using the slot machine j estimated using all the past t attempts. So $S_j = \frac{\text{Number of games where we used slot machine j and we won}}{\text{Total number of game}}$

Theorem time →If in the next game we allocate a number of attempts to slot j such that

- $S_j >$ average ($S_j > \frac{\sum_i S_i}{N}$)
- j has a small number of arms

Then

- We maximize our probability of winning
- The probability of winning tends to 1 with the number of attempts tending to infinity

The consequence is the theorem of asymptotic convergence of genetic algorithms.
Let i(t) be the best individual in the population at generation t and S_o be the best set of global optima. Then

$$\lim_{t\to\infty} P(i(t) \in S_o) = 1$$

7.11 Problems with genetic algorithms

The main drawbacks of standard genetic algorithms are

1. Premature convergence →also known as loss of diversity. For any possible problem our algorithm will produce individuals that are similar to each other. If they are not similar to the global optimum we might never reach it.
2. Position problem →typical of standard crossover. We risk splitting our good chromosomes if they are at different ends.
3. Unicity of the fitness

We can solve them with the following methods

7.11.1 Premature convergence

We have ways to measure the diversity of our population and to see if we have this problem

- Entropy →$\sum_{j=1}^{N} F_j \log(F_j)$
- Variance →$\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$

Both can be measured on genotypic (within the individual) and phenotypic (fitness) dimensions.

- Phenotypic entropy →N is the number of different fitness values in the population and F_j fraction of individuals with a specific fitness value in the population. For continuous fitness we usually define bins.

- Genotypic entropy →N is the number of different genotypes while (strings) and F_j the fraction of individuals with that genotype.
- Phenotypic variance →n is the number of individuals in the population, x_i is the fitness of individual i and $\bar{x}$ is the average fitness of the individuals in P.
- Genotypic variance →n is the number of individuals in the population, x_i (given a distance measure) is the distance of the individual i from the origin and $\bar{x}$ is the average distance to origin of the individuals in the population.

As origin, we can choose any individual (usually a string of zeroes). These methods are just monitoring systems (not a way to solve the problems).

How to maintain diversity?

We can do

1. Fitness sharing
2. Restricted mating

With fitness sharing we will give more fitness to individuals that are very diverse. Then similar individuals will be penalized. This approach doesn't change the representation of our individuals.

Restricted mating acts in the opposite way.

Fitness Sharing

For all individuals i in the population:

- Given a predefined distance measure calculate the distance from i to all the other individuals in the population
- Normalize all these distances in the interval [0, 1]
- Invert these distances (for instance calculating $S(i,j) = 1 - d(i,j)$)
- Calculate the sharing coefficient the sharing coefficient of i →$(S(i) = \sum_j S(i,j))$
- Redefine the fitness as $f_s(i) = \frac{f(i)}{S(i)}$

We can do this given:

1. A distance measure
2. A normalization method
3. A way to invert the distances

EXAMPLE

POPULATION

i_1 : 11111 $f(i_1) = 50$
i_2 : 01111 $f(i_2) = 40$
i_3 : 10111 $f(i_3) = 30$
i_4 : 00000 $f(i_4) = 10$

DISTANCE: Hamming Distance
NORMALIZATION: $d_N = \frac{d}{L}$
INVERSION: $1 - d_N$

MAXIMIZATION

New fitness i_4:

$$d(i_4, i_1) = 5 \longrightarrow d_N = 1 \longrightarrow S = 0$$
$$d(i_4, i_2) = 4 \longrightarrow d_N = \frac{4}{5} \longrightarrow S = \frac{1}{5}$$
$$d(i_4, i_3) = 4 \longrightarrow d_N = \frac{4}{5} \longrightarrow S = \frac{1}{5}$$

$$S(i_4) = 0 + \frac{1}{5} + \frac{1}{5} = \frac{2}{5}$$

The result is that we have implicit speciation.

Restricted mating

Starting from an arbitrary schema we divide it in two parts, template and functional. In the template we are basically coding what the individual will like from the mating point of view. We get that we can do cross over in a restricted way.

The model are

- Bi-directional match
- Uni-directional match
- Best partial match

RESTRICTED MATING

	⟨**TEMPLATE**⟩	⟨**FUNCTIONAL**⟩
i_1 :	$*\ 1\ 0\ *$	: 1 0 1 0
i_2 :	$*\ 0\ 1\ *$	: 1 1 0 1
i_3 :	$0\ *\ *\ 0$	: 0 0 0 0

7.11.2 Position problem

Crossover tends to destroy good individuals if the good genomes are on different sides after the cut. If we want to save those genomes we would like to reorder the indexes, but we will have problems in doing it. Our objective is to take two parents with indexes in different orders and create children that have all indexes (once). We can do something called **Cycle Crossover**

CYCLE CROSSOVER

PARENTS

	1	2	3	4	5	6	7	8	9	10
P_1 :	1	1	0	0	0	1	1	0	0	1
P_2 :	0	0	0	1	1	1	1	1	0	0

$\xrightarrow{\text{INVERSION}}$

Indices:	9	8	2	1	7	4	5	10	6	3
Values P_1:	0	0	1	1	1	0	0	1	1	0
Values P_2:	0	1	0	0	1	1	1	0	1	0

$\xleftarrow{\text{REORDER}}$

OFFSPRING

1 0 0 0 1 1 1 1 0 0
0 1 0 1 0 1 1 0 0 1

- **Step 1: Identifying a Cycle** Start with the first gene of Parent 1 (P_1) and look at the gene in the same position in Parent 2 (P_2). You then find where that value from P_2 is located in P_1[cite: 1]. Repeat this process until you return to the starting value, which completes one "cycle"[cite: 1].
- **Step 2: Mapping (Inversion)** In the example, the "Inversion" section shows the indices (1–10) rearranged to group them according to the cycles found[cite: 1]. For example, if indices 1, 4, and 5 form a cycle, they are analyzed together in this step[cite: 1].
- **Step 3: Selection and Swapping**

- For the first cycle, the offspring inherit the genes directly from their respective parents in those positions[cite: 1].
- For the next cycle, the genes are "swapped": Offspring 1 gets genes from Parent 2, and Offspring 2 gets genes from Parent 1[cite: 1].

- **Step 4: Reconstruction (Reorder)** Finally, the genes are moved back into their original numerical positions (1 through 10)[cite: 1]. The values highlighted in **red** in the image represent the specific genes that were either swapped or kept as part of a cycle to create the final *Offspring*[cite: 1].

Another way is to use **Partially matched crossover**

PARTIALLY MATCHED (MAPPED) CROSSOVER

PARENT 1

9	8	4	**5**	**6**	**7**	1	3	2	10
1	0	1	**1**	**0**	**1**	0	0	0	1

PARENT 2

8	7	1	**2**	**3**	**10**	9	5	4	6
0	0	1	**0**	**1**	**1**	1	1	0	0

OFFSPRING (Result)

9	8	4	**2**	**3**	**10**	1	6	5	7
1	0	1	**0**	**1**	**1**	0	0	1	1
8	10	1	**5**	**6**	**7**	9	2	4	3
0	1	1	**1**	**0**	**1**	1	0	0	1

Sometimes we have to use the repair algorithm if some problems are coming.
We can have a new kind of mutation where we swap the character's order inside the window.

Initial:	9	8	**4**	5	6	7	**1**	3	2	10	
							⇓				
mut1 (swap):	9	8	**1**	5	6	7	**4**	3	2	10	(swap)
mut2:	9	8	**1**	**7**	**6**	**5**	**4**	3	2	10	

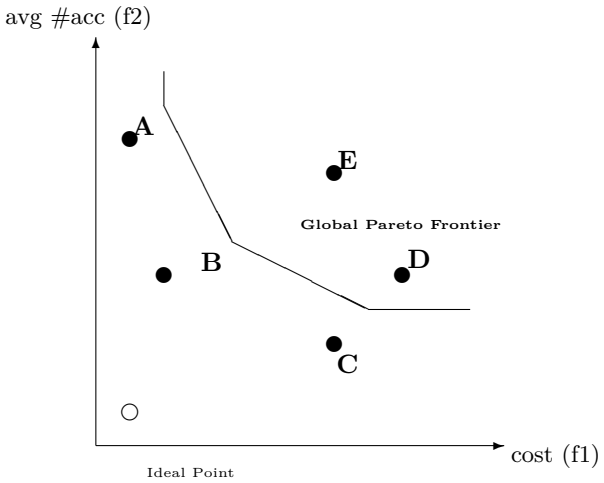
7.12 Multi-objective optimization

We might have more than one fitness to optimize, then we need a way to optimize for all of them.

MULTI-OBJECTIVE OPTIMIZATION (MULTI-FITNESS)

Fitness = (cost, avg. Numb. Of accidents)

Numeric Cases:	**Global Pareto Optimal Set** $= \{A, B, C\}$
$A = (2, 10)$	(Set of the optima)
$B = (4, 6)$	
$C = (8, 4)$	*A, B, C are NON-DOMINATED:*
$D = (9, 5)$	There is no other global solution in the
$E = (7, 8)$	search space better in all criteria.



If our solutions are not the best in all the criteria they are called non dominated. The set of non dominated solutions is called the Pareto optimal set. If all of our possible solutions are non dominated then all of them can be used, then we want to find the Pareto optimal set. But how can we choose the solution in our set?

We can plot a theoretical point to look for our preferred solution (Global optimum that doesn't exist). Since it doesn't exist then we look for the point that minimizes the distance from this preferred point. Then now we want to find an algorithm that find the closest point.

In genetic algorithms we can have multi optimization, changing selection as follows:

1. Copy all the individuals of the population into a set S
2. Find all non dominated individuals that are in S, assign a flag (1)
3. Remove these individuals from S
4. Find all non dominated individuals in S, assign a flag (2)
5. Remove them from the population and so on until S is empty
6. The flags now are proportional to the value of dominance/importance of the individuals
7. For each individual in the population the probability of selecting that individual is inversely proportional to its flag.

To implement a way to find the non dominated individuals we just look for the dominated ones. We can also have Pareto based optimization

7.13 Continuous optimization

In this setting in every position of our string we have continuous values and not integers. In this setting individuals are m-dimensional vectors of real numbers where each element is part of a given interval (optional). We can have many crossovers and mutations for this kind of algorithm. The main one are called geometric crossover and geometric mutation.

An individual can be considered as a point in an m-dimensional Cartesian space.

7.13.1 Geometric crossover

Given two parents $[x_1, x_2, ..., x_m]$ and $[y_1, y_2, ..., y_m]$ they will generate one child $[R_1x_1 + (1 - R_1)y_1, R_2x_2 + (1 - R_2y_2, ..., R_mx_m + (1 - R_m)y_m)]$ where $\forall i = 1, 2, ...m$ R_i is a random number $\in$ [0,1].

We call it geometric because if we plot the two parents we will notice that the kind lies in the middle of them (is a linear combination). The kid will not be better than the parents but for sure better than the worse one. But it can be better than both in certain situations.

7.13.2 Geometric mutation

Also known as box, or ball, mutation.

Given one individual $[x_1, x_2, ..., n_m]$ this mutation creates one individual $[x_1 + R_1, x_2 + R_2, ..., X_m + R_m]$ where $\forall i = 1, 2, ..., m$ R_i is a random number $\in [-m_s, m_s]$ where m_s is a parameter called mutation step. Geometrically we can get any element inside a given square or circle of a certain radius centered on our original point.

In the case that fitness is proportionate to a distance to a certain point (the global optimum) it is always possible to improve by creating a child that is closer to the global optimum while being inside the box. In this case we say that the fitness landscape is unimodal.

7.14 Particle Swarm Optimization

It's still a population based algorithm but is not inspired by the theory of evolution. It mimics as flocks of birds moves. It belongs to the field of swarm intelligence (when animals work together they are able to solve problems that they aren't able to if they are alone).

This algorithm is based on two laws of attraction on the single animal level

- The single animal has memory, it wants to go back to a point where it was comfortable
- There is a leader that will be followed by the rest of the animals

The individuals (particles) are m-dimensional vectors of real numbers while the population (swarm) is an n-dimensional vector of particles (a matrix).

We use the following notation

- $\vec{x_i}$ is the i^{th} individual (or position) in the swarm (i = 1, 2, ..., n)
- $\vec{v_i}$ is the velocity of particle i (m-dimensional vector of real numbers)
- $\vec{b_i}$ is the local best position of particle i (i $\in \mathbb{R}^m$)
- $\vec{g}$ is the global best position of any particle in the swarm (best so far)

The algorithm follows the following pipeline

1. For all i = 1, 2, ..., n: we initialize $\vec{x_i}$and$\vec{v_i}$
2. For all i = 1, 2, ..., n: we initialize $\vec{b_i} = \vec{x_i}$
3. $\vec{g} = \text{argmin}_{\vec{x_i}} f(\vec{x_i})$ (for minimization problems)
4. While not termination condition, for all i = 1, 2, ..., n
   (a) $\vec{x_i} = \vec{x_i} + \vec{v_i}$
   (b) Update $\vec{v_i}$
   (c) If $f(\vec{x_i})$ better than $f(\vec{b_i})$ then $\vec{b_i} = \vec{x_i}$
   (d) If $f(\vec{x_i})$ better than $f(\vec{g_i})$ then $\vec{g_i} = \vec{x_i}$
5. Return $\vec{g}$

The update of the velocity is done using the following formula:

$$\vec{v_i} = w\vec{v_i} + C_1\vec{R_1}(\vec{b_i} - \vec{x_i}) + C_2\vec{R_2}(\vec{g} - \vec{x_i})$$

Where

- w is the inertia constant, how a movement is similar to a previous one
- C_1 is the cognitive component
- C_2 is the social component
- $\vec{R_1}$ and $\vec{R_2}$ are vectors of random numbers $\in$ [0, 1]

8 Big data analytics

8.1 Introduction

8.1.1 What is Big data

Big data is a buzzword used to characterize certain kind of data. Those data were originally characterized by their volume, velocity and variety, but now we have more dimensions. Big data processing life cycle follows this path

1. Acquisition
2. Preprocessing
3. Storage and management
4. Privacy and security
5. Analyzing
6. Visualization

Usually these datasets are very huge and traditional tools can't process them efficiently. This part involves not just size but also variety, speed and complexity.

Traditional data

- Relational
- Structured
- Manageable in SQL systems

Big data

- Distributed across systems
- Structured, semi-structured or unstructured
- Requires new technology

With traditional data the best way to manage them is by vertically scaling the requirements, with big data we have to grow horizontally. Basically from using big CPUs to many small ones.

8.1.2 Data sources

The majority of big data come from the web revolution with drivers that are everyday more common

- Internet
- Smartphones
- Social media
- Sensors

Big data matters because we can discover patterns and insights and so supporting decision-making and training for machine learning models.

The data can be considered by their origins

- Human generated
- Machine generated
- Business and institutional
- Open/public

8.1.3 The Vs of Big data

The 3 Vs

1. Volume
2. Velocity
3. Variety

5 additional Vs

1. Veracity
2. Value
3. Variability
4. Validity
5. Visualization

The most common 5 Vs are Volume, Velocity, Veracity, Value and Variety.

8.1.4 The Big data ecosystem

The place where we process these data, composed of

- Storage systems
- Process engines
- Management and governance
- Analytics and visualization tools
- People and skills

The ecosystem is characterized by 4 layers

1. Storage layer
2. Processing layer
3. Management layer
4. Analytics layer

We use many tools to visualise these data (Tableau, Power BI, QlikView or Python based ones) and 3 main kinds of people act in the ecosystem

1. Data engineers
2. Data scientists
3. Business analysts

8.1.5 Applications of Big data

Big data analytics as many possible fields of application

1. Business and retail
2. Social media communication
3. Healthcare and life sciences
4. Finance and economy
5. Government and public sector
6. Smart cities
7. Science and research

8.2 Distributed storage, ecosystems and programming paradigms

8.2.1 Distributed storage and file systems

To save our data we could think about saving flat files on local disks, but this comes with problems of redundancy, slow search and retrieval and poor security. Then we can use file systems that define how files are named, stored and retrieved. Those systems use directories and store the data in blocks of given size.

As said before we need distribution to save our data in a good way, then we use many servers. To avoid problems given by common node failures we can use replication and sequential writing. The concept of connecting many independent computers to work as one big system is called **cluster computing**. Each computer runs its own operating system but when combined they form a unified resource. **Sharding** is another way where we split our big datasets into smaller pieces called shards which are then distributed across multiple machines.

Splitting our data let us ensure availability and reliability, usually we have a replication factor of 3. One of the main systems is HDFS (Hadoop Distributed File System). In HDFS we have name nodes, data nodes and replication pipelines among the data nodes.

HDFS strengths

- Efficient for very large files
- Streaming data access
- Good on low-cost pieces of hardware

HDFS cons

- Low latency access
- Lots of small files
- Multiple writers/edits

Another way to store data is using object storage (Cloud). The data are stored as objects in a bucket. Good for elastic scaling, but we have higher latency and lower throughput. Some examples are AWS and Azure Blob. This is a cost-effective way to handle many files. Replication is used as always. The advantages of this approach are

- Can deal with many files
- Small files are used
- We can scale elastically up and down
- Cheap

We also have cons

- Atomic replacement of full objects
- Lower throughput
- High latency
- Lack of query engine
- Off premises

8.2.2 Distributed computing models

Divide and conquer approach where we process data where they are stored. We scale horizontally across nodes.

8.2.3 The big data ecosystem

Formed by a web of interconnected tools for storage, processing, analytics and visualization. They will lower costs and time of deployment.

Hadoop ecosystem

- HDFS for storage
- YARN to manage resources
- MapReduce for batch processing
- Hive, Pig, Sqoop, Flume for analytics

Apache Spark

- Unifier engine for multiple tools
- Faster than MapReduce
- Multiple programming languages

The databases are usually NoSQL and can be

- Key-Value
- Document
- Columnar
- Graph

For unstructured and structured data we use data lakes, warehouses of lakehoues

8.2.4 Programming paradigms

As said we need to parallelize and resist faulting when working with big data. We can have imperative (step by step), functional (we define our own functions) and declarative (what to compute, not how) programming.

The most common approach is functional programming where computation is considered as evaluation of functions. We use things like map, reduce, filter and fold.

MapReduce paradigm

- Apply function to data blocks (Map step)
- Group by Key (Shuffle step)
- Aggregate results (Reduce step)

8.3 Distributed parallel processing and the map reduce paradigm

8.3.1 Distributed parallel processing

Using big data we deal with terabytes of data, so we can't put them on a single server. The execution will be too slow, then we will split the workload into smaller tasks that will run simultaneously and possibly in parallel.

- **Parallel** →multiple tasks executed at the same time
- **Concurrent** →multiple tasks that progress together but not necessarily at the same time

The traditional way of analyzing data is to move all the data to one machine and process them. This is inefficient, to save network bandwidth we can do a distributed approach where we send the computations where the data already is. The architecture is cluster like and made by nodes organized in racks.

8.3.2 The MapReduce paradigm

Paradigm where we try to reduce complexity. We can use Hadoop to resolve logistic problems.

Work flow

1. **Map** →Process input split and emit key-value pairs
2. **Shuffle/Sort** →group intermediate pairs by key
3. **Reduce** →Aggregate values for each key to get a final output

```
def map(key, value):
    for word in value.split():
    emit(word, 1)

def reduce(key, values):
    emit(key, sum(values))
```

8.3.3 Case study

- Temperature records
- Retail analytics

Strengths of MapReduce

- Scales to thousands of nodes
- Fault-tolerant
- Simplifies distributed programming

Limitations

- Batch only, causes latency
- Heavy disk
- Iterative and ML inefficient

8.4 The Hadoop ecosystem

8.4.1 Hadoop

Hadoop is an open source framework for distributed storage and processing of large datasets. It's designed to scale horizontally across clusters.

Key features are fault tolerance (data replication), high throughput (parallel processing) and scalability (from few nodes to thousands). Hadoop has two main pillars

1. HDFS →Hadoop distributed file system (stores data across multiple machines and replicates blocks for reliability)
2. MapReduce →processing engine (splits computation between map and reduce, extended with YARN to manage cluster resources)

Before Hadoop, we had to fit our data into one machine (limited scalability). Hadoop allowed big data storage and parallel computation.

8.4.2 Filesystems and HDFS basics

Since large datasets can't fit on one machine we have to distribute it across nodes and to make computation where data resides.

Terms of Filesystems

- **Block** →fixed-sixe unit of storage
- **Split** →logical division of input for MapReduce
- **Shard/Partition** terms for splitting datasets

Filesystems can be local (one disk, not fault-tolerant) or distributed (replicates blocks across many nodes). HDFS stores large files and is optimized for throughput (not latency), write once and read many times.

HDFS components

- NameNode →Manages metadata and block locations
- Secondary NameNode →assist with checkpoints
- DataNodes →store actual blocks

To ensure fault tolerance we have block replication, one block is stored in multiple nodes or racks. Large block size reduces metadata overhead, and we have data locality for the computation.

8.4.3 Hadoop's execution model

Hadoop originally used MapReduce v1 both as computation engine and resource manager, but there were some problems like single job tracker, limited scalability and rigidity. YARN was introduced to fix this in Hadoop 2.0.

V1

- We have a JobTracker that assigns and manages tasks.
- The TaskTracker runs the map and reduce tasks and report the status to the JobTracker.
- If JobTracker is overloaded it will break.
- As the clusters grow we have a bottleneck that will easily break the system.

YARN

- Separates resource management from job execution.
- The key roles are ResourceManager, NodeManager and ApplicationMaster
- It supports multiple processing frameworks
- Allows for better scalability, reliability and flexibility

YARN Architecture Components

- **ResourceManager (RM):** The global authority of the cluster. It handles resource scheduling (via the *Scheduler*) and job submissions (via the *ApplicationManager*). It tracks availability of CPU, memory, and I/O.
- **NodeManager (NM):** Runs on each worker node. It manages local resources, launches tasks inside **Containers**, and reports health and usage back to the RM.
- **ApplicationMaster (AM):** One instance per job. It negotiates resources with the RM and manages the job lifecycle (requesting containers, handling failures) without micromanaging every task.

Containers and Locality

- **Containers:** A logical bundle of resources (CPU + Memory + I/O). Every task runs in its own isolated container to prevent conflicts.
- **Locality Awareness:** To minimize network costs, YARN prefers **Node-local** execution (data on the same node), then **Rack-local**, and finally **Off-switch** as a last resort.

Scheduling and Benefits

- **Schedulers:** Supports *FIFO* (simple order), *Capacity* (guaranteed queues), and *Fair* (equal sharing/preemption).
- **Advantages:** Enables multitenancy, scales to tens of thousands of nodes, and supports multiple engines like Spark, Tez, and Flink beyond just MapReduce.

8.4.4 Optimization and refinements

From Splits to Reducers

- **Input Splits & Blocks:** Large files are broken into HDFS blocks (default 128 MB). Each split is assigned to a map task; parallelism depends on the number of blocks.
- **Shuffle & Sort:** Mappers produce (key, value) pairs. The system groups all values with the same key across the cluster. This step is **network-intensive** and often the bottleneck of MapReduce.
- **Reducers:** Each reducer receives one or more key groups, applies the function (sum, max, etc.), and writes the final output back to HDFS.

Optimizations: Combiners & Partitioners

- **Combiners:** Optional "mini-reducers" on mapper output. They perform local aggregation before the shuffle phase to reduce network traffic (e.g., local word count sums).
- **Partitioners:** Decide which reducer processes a specific key (default: $hash(key) \pmod{numReducers}$). Used for load balancing or specific workflows.

Handling Stragglers & Joins

- **Speculative Execution:** To handle slow nodes ("stragglers"), Hadoop launches duplicate copies of slow tasks on other nodes. The first copy to finish is kept, reducing job completion time.
- **Reduce-side Joins:** Used for joining datasets on a key (e.g., $R(A, B) \bowtie S(B, C)$). Mappers emit (B, row) from both datasets, and the reducer performs the join. Flexible but expensive due to high shuffle overhead.

Hadoop Strengths & Limitations

- **Strengths:** Handles petabytes of data, fault-tolerant (automatic replication/retries), and supported by a large ecosystem (Hive, HBase, Sqoop).
- **Limitations:** **Batch-oriented** (not suitable for real-time), high latency due to heavy disk I/O, and inefficient for iterative jobs (ML, graph algorithms).

8.4.5 YARN and scheduling

Resource Management in YARN

- **Resource Types:** YARN manages four key resources across the cluster: CPU (cores), **Memory** (RAM), **Disk I/O** (read/write), and **Network I/O** (bandwidth).
- **Container Allocation:** Resources are bundled into **Containers**. Each container represents a specific amount of CPU and Memory, ensuring jobs get exactly what they requested.
- **Benefits:** This provides strict **isolation** between jobs and increases efficiency by sizing containers to actual application needs.

YARN Scheduling Policies

- **FIFO Scheduler:** Jobs run in the strict order of submission. While simple, it causes head-of-line blocking where small jobs must wait for large ones to finish.
- **Capacity Scheduler:** The cluster is divided into **queues** with guaranteed resources (e.g., Department A gets 50% quota). This ensures multi-tenant access, though unused capacity can be "borrowed" by other queues.
- **Fair Scheduler:** Resources are dynamically shared so that all jobs get a fair share over time. Small interactive jobs can start immediately even if a large job is already running.

Scheduling Scenarios in Practice

- **Scenario 1 (FIFO):** A massive data-cleaning job starts first; a subsequent small query is blocked until the first job completes, leading to poor responsiveness.
- **Scenario 2 (Capacity):** A department is guaranteed 50% of the cluster. Even if other departments are busy, the quota ensures they always have access to resources.
- **Scenario 3 (Fair):** When a small job arrives during a large job's execution, the scheduler reassigns some containers, so both can progress simultaneously.

8.4.6 Hive, Pig and Sqoop

Apache Hive: Data Warehouse on Hadoop

- **Overview:** A data warehouse infrastructure built on top of Hadoop. It provides **HiveQL** (a SQL-like language), making Hadoop accessible to analysts without Java/MapReduce knowledge.
- **Schema-on-Read:** Unlike traditional databases, Hive loads data into HDFS "as is" and applies the schema only when the data is read. This offers great flexibility for semi-structured data.
- **Architecture:**
  - **Metastore:** Stores table metadata and schemas.
  - **Compiler & Optimizer:** Parses HiveQL and improves the execution plan (e.g., predicate push-down).
  - **Execution Engine:** Converts the plan into MapReduce, Tez, or Spark jobs.

Hive Optimization & File Formats

- **Partitioning:** Splits tables into HDFS directories based on a column (e.g., *year=2024*). It improves performance via **partition pruning**.
- **Bucketing:** Further divides data based on a hash function to ensure balanced distribution for joins.
- **Storage Formats:**
  - **ORC / Parquet:** Columnar formats, highly compressed, best for analytical queries.
  - **Avro:** Row-based format, excellent for schema evolution and streaming.

Apache Pig: Scripting for ETL

- **Overview:** A high-level platform for analyzing large data sets using a procedural language called **Pig Latin**.
- **Use Case:** Designed for complex data pipelines and ETL (Extract, Transform, Load) workflows. It is more "step-by-step" compared to Hive's declarative SQL approach.
- **Example Flow:** `LOAD` (bring data) → `GROUP` (by key) → `FOREACH` (apply aggregation).

Apache Sqoop: SQL-to-Hadoop

- **Overview:** A tool designed for efficiently transferring bulk data between **Relational Databases** (RDBMS) and the Hadoop ecosystem (HDFS, Hive, or HBase).
- **Functionality:** It uses Map-only jobs to import/export data in parallel via JDBC.
- **Example Command:**

```
sqoop import --connect jdbc:mysql://localhost/db --table orders
```

- **Pros/Cons:** Simple and supports incremental loads, but it is better suited for bulk transfers rather than real-time streaming.

8.4.7 Summary

- **Execution model** →How Hadoop runs MapReduce jobs
- **Refinements** →optimizations via combiners, partitioners, speculative executions
- **Ecosystem** →Hive, Pig and Sqoop

Hadoop it's still foundational but has been partially replaced by Spark for faster analytics

8.5 Introduction to Spark

Spark is a unified engine for big data processing. Some advantages of Spark are:

1. We can process in parallel large datasets across a cluster
2. We can perform ad hoc interactive queries and visualize datasets. We can also do machine learning models using MLlib
3. We can implement end-to-end data pipelines from different streams
4. We can analyze graph data sets and social networks

The success of Spark comes from the fact that is open source, fast and provides parallel and distributed computing. The interface for Python is called PySpark.
Some improvements of Spark with respect to Hadoop

- MapReduce on Hadoop is slow
- Spark is in-memory so faster and more flexible
- Spark is a unified engine for batch, SQL etc.
- Spark decouples computation from storage. Spark runs independently of HDFS and keeps data in memory between stages, avoiding repeated disk reads/writes
- Spark enables interactive queries and iterative algorithms that are difficult in MapReduce

Spark in the big data landscape complements Hadoop, works with different kind of datasets, it has cloud-native integrations, bridges between different data platforms and evolved after Hadoop improving it.

Spark supports multiple languages like

- Scala
- Python
- Java
- R
- More via 3rd party APIs

All languages run on the same Spark core engine ensuring consistency and scalability. The choice on what to use depends on skills, integration needs and project goals.

Spark provides two levels of APIs for working with data

- Low-level (RDDs) →fine-grained control, defines transformations and actions
- High-level (DataFrame and Dataset) →structured, optimized queries using Catalyst optimizer and Tungsten engine

RDDs are ideal for unstructured or experimental workloads while DF and DS are better for performance and interoperability. Users can mix both depending on their data workflow.
Some useful Spark applications are Batch ETL, interactive SQL queries, streaming analytics, ML at scale and graph processing.

The main Spark architecture is made of the followings

- Driver program →orchestrates the operations (runs SparkContext, defined RDD transformations and actions and collects results)
- Cluster manager →YARN, Kubernetes and Mesos
- Executors →run tasks and store data (worker processes on cluster nodes, executes tasks in parallel and cache data in memory for reuse)

SparkContext is an old entry point for Spark application, it creates RDDs and manages jobs. It was introduced in early Spark, and it is the core connection to the cluster. This by communicating with the cluster manager to allocate resources. The RDD API depends on the SparkContext for all operations.

SparkSession is a higher-level entry point that unifies SQL and RDDs APIs.

In Spark, we have two main operations

1. Transformations →define a new RDD based on the current one(s). The data inside are never changed, but we want to modify them as needed. These transformations are lazy (they won't be executed until action). Some common transformations are map, filter and flatMap.
2. Actions →return values to the driver or save to storage. Some actions are count, take, collect and SaveAsTextFile

The evaluation is lazy since transformations build a plan while actions trigger the execution.

RDD →Resilient distributed dataset, it's an immutable, distributed collection of objects built via transformations. RDD properties are immutability, lazy evaluation, fault-tolerance and partitions across cluster. We can create RDDs from collections and data in memory, from external storage and from transforming existing RDDs.

Spark has two main level of execution

1. Logical →what operations to perform, built when RDD operations are defined, represented as a DAG of transformations
2. Physical →how Spark executes the plan, it splits the DAG into stages and tasks, the DAG scheduler and Task scheduler optimize and distribute the work

Lazy evaluation follows this procedure

- Spark builds DAG of operations
- Optimizes before execution
- Saves redundant computation

Creations and transformations do nothing on their own. They are applied when an action is launched.

Transformations on RDD can be by Type

- Unary →operate on one RDD and obtain one new RDD, usually for element-wise operations like filter, map and flatMap
- Binary →combine two RDDs, like union, intersection and subtraction
- Pair →work on (key, value) pairs, like groupbykey, reducebykey and sortbykey

We can also define narrow and wide transformations based of the execution implications

- Narrow →each output partition depends on one input partition
- Wide →the output depends on multiple input partitions, they trigger shuffles (expensive operations)

For ML algorithms some important elements are persist() and cache() because they store RDD in memory.

To ensure fault tolerance RDDs follow a lineage graph, so if the partition is lost we can recompute from lineage. It's a built-in recovery mechanism that uses a DAG that records how an RDD was created. We don't store data but we recompute them.

Different from Spark (in-memory, DAG scheduler and faster) MapReduce is disk-based, simpler and robust.

The current system supports several cluster managers like Standalone mode, YARN and Kubernetes.

RDDs are limited by some elements

- Verbose API
- No schema
- Less optimized than DataFrames

8.6 DataFrames in Spark and Spark-Hadoop relationship

8.6.1 Recap

In the previous section we saw the concept of RDDs that are distributed and fault-tolerant collections. But they are also limited by the fact that they have no schema (so they are inefficient for queries) and that the API is verbose. Our next step will be to review DataFrames since they are structured and optimized (SQL-like).

8.6.2 DataFrames

A dataframe is a distributed table-like structure with rows and named columns. Pandas DF and SQL tables inspired them. Each column has a defined type. These DataFrames are built on top of RDDs, but they are faster (in-memory) and easier (table-like).

With an RDD we tell Spark how to aggregate, while with a dataframe we communicate what we want and Spark will optimize it.

On top of that DataFrames have the following benefits

- Easy to use
- Performance are good
- Interoperability
- Integration

To create a DataFrame we can start from

- Structured files
- RDDs with schema
- External database

The schema can be inferred by Spark or defined by the user.

We recall transformations (return a new dataframe) vs actions (trigger execution). They follow the schema Transformations →DAG →Actions.

Some very useful functions are called Window functions →They enable analytics like top-N per group.

The queries are optimized by Catalyst in the following stages

1. Analysis
2. Logical optimization
3. Physical planning

4. Code generation

After a query is written Spark builds a logical plan that describes what operations need to be done. We then get the unresolved logical plan that includes references Spark has not yet verified. Catalyst then analyzes the plan using the schema and the metadata creating an analyzed logical plan. This last plan is independent of any physical execution strategy.

The optimization phase follows these steps

1. The Catalyst Optimizer transforms the analyzed logical plan into an optimized logical plan
2. It applies both rule-based and cost-based optimization
3. Predicate pushdown (move filters close to the data source)
4. Projection pruning (remove unnecessary columns)
5. Constant folding (simplify expressions)
6. Null elimination and common subexpression removal
7. The result is an optimized logical plan that preserves semantics but reduces computation

The physical plan defines how Spark will perform the computation

- Catalyst's planner takes the optimized logical plan and selects the most efficient execution strategy (SortMergeJoin, BroadcastHashJoin, ShuffleHashJoin, whole-stage code generation, task parallelization and stage pipelining)
- We can have multiple candidate physical plans and Spark will choose the best one
- Tungsten engine receives the plan, and it will execute it managing the memory

Tungsten executes the physical plan using whole-stage code generation and off-heap memory management

- Memory + CPU optimizations
- Off-heap memory management
- Whole-stage code generation
- Vectorized processing

Catalyst →handles query analysis and optimization ensuring that Spark finds efficient execution strategies automatically

Tungsten →handles execution and performance optimizing CPU and memory usage for physical plan execution.

These two together form the Spark SQL engine. This engine makes Spark efficient, declarative and adaptable. Spark can be integrated with SQL providing the ability of working with different types of data and formats.

In this setting the DataFrames are the API and Spark SQL is the interface.

Performance tips

- Use Parquet/ORC instead of CSV
- Partition data by common query keys
- Cache DataFrames when used
- Broadcast joins for small tables

From this we can have the takeaway that DataFrames are preferable to RDDs unless RDD operations are needed.

Use RDDs if

- You need low level transformations

- You work with unstructured or complex data types
- You require fine-grained control over partitions or persistence

Use DataFrames when

- Data is structured
- You want automatic optimization
- Concise SQL-style queries are preferable

For Python users DataFrames are better. They can resort to RDDs if they need more control.

8.6.3 Datasets

Spark has also the option of using datasets, a unified API that combines RDDs and DataFrames. Some benefits are that datasets are type-safe, support both functional and relational operations, available in Scala and Java (not Python) and we can have optimization via Catalyst and Tungsten.

8.6.4 Spark-Hadoop relationship

Spark integrates with Hadoop but doesn't depend on them. It can also run standalone or in cloud environments. The read and write come from HDFS, YARN is for resource management, Hive Metastore for table metadata and SQL queries, MapReduce is replaced for in-memory executions.

The integration is done

- On YARN
- With Hive
- With HDFS

8.7 Spark in Cloud-based computational environments

The main use of Cloud computing in the context of big data is given by the fact that is elastic and can be used with on-demand options. It's the ideal way to work with distributed data processing. It also removes hardware management overload and some models support pay-as-you-go models.

Running Spark in the cloud is also useful because pre-existent clusters can be costly, too rigid, and they require skilled admins. With cloud based nodes on the other hand with more elastic and scalable options. Basically we don't need to manage our clusters manually, we can scale easily, the integration with cloud-native storage is easier, and we can access prebuilt Spark distributions.

Some benefits of this approach are

- The deployment is easy and so the management
- We can integrate everything with cloud-native tools
- Team collaboration is optimized

Some cloud deployment options are

- Managed Spark services
- Spark on Kubernetes or standalone clusters
- Interactive notebooks

Spark can be managed in two main ways: Managed and Self-Managed

Managed Spark

- Automatically handles setup, scaling and monitoring
- Best for production and teaching

Self-Managed Spark

- Full control over configuration and cost
- Best for research and experimentation

8.7.1 Databricks

Databricks is a unified platform for data engineering, analytics and AI. It was created by John Spark (the inventor of Spark). This platform is able to fully manage Spark clusters with a notebook based environment. Databricks is formed by the following components:

- Workspace →collaborative notebook environment
- Clusters →auto-scaling Spark clusters
- Jobs →automated workflows
- DBFS →Databricks File System (built on cloud storage)

Databricks has the following advantages

- One-click cluster creation
- Integrated visualization and SQL interface
- Optimized Spark runtime
- Supports all major clouds

Databricks follows this workflow

1. Upload data to the cloud
2. Create the Databricks notebook
3. Initialize Spark session
4. Read, transform and visualise data
5. Export the results or train models

Simplifying to **Raw →Clean →Analytics**.

Databricks can be used with its free edition (not for production tho).

8.7.2 AWS Elastic MapReduce (EMR)

AWS EMR is an Amazon-managed Hadoop and Spark platform that uses EC2 instances and S3 for storage. These two provide resizable virtual services and object storage services.

EMR clusters are made of

- Master node →job coordination
- Core nodes →run HDFS and Spark executors
- Task nodes →compute-only

On EMR Spark environments are pre-configured, and they can be easily integrated with S3. EMRFS allows seamless S3 access.

The workflow is the following

1. Store data in Amazon S3
2. Launch EMR with Spark application
3. Process the data and write the results back to S3 or RedShift (data warehouse)

S3 →EMR →S3 →RedShift

EMR has the following advantages

- Flexible cluster customization
- Cost-control via auto-termination
- Deep integration with AWS analytics tools like Athena and Glue

8.7.3 Google Cloud Dataproc

Dataproc is a tool that help us manage Hadoop and Spark services on Google Cloud. It can be integrated with services like GCS (data lake), Google BigQuery (Analytics) and Vertex AI (unified ML platforms).

Dataproc is convenient since it help us for fast cluster startup, pay-per-second billing is enabled and some tools like Jupyter, Spark and Hadoop are already installed.

Dataproc uses compute engine VMs for nodes, the storage is done via GCS (not HDFS) and Job orchestration is done via Cloud Composer (Airflow).

8.7.4 Azure HDInsight

HDInsight is a Microsoft's managed big data service. It supports Hadoop, Spark, Hive, Kafka and HBase and has a tight integration with Azure Data Lake. The key features of HDInsight are its good security, the fact that can be easily scaled through Azure and that it can be integrated with Power BI and Synapse.

A typical use include the following steps

1. Data from Azure Data Lake
2. Spark job on HDInsight
3. Visualization in Power BI

HDInsight has some pros like a very good Microsoft integration (how is this a pro?) and the cluster management. On the other hand the startup is slow and Databricks is cheaper.

Some replacements that we can find in Azure are Databricks, Fabric, Azure Synapse Analytics, Azure Data Factory, HDInsight on AKS, Azure Stream Analytics and Azure Data Explorer (Kusto).

8.7.5 Kubernetes

Since Spark can run natively on K8s clusters we have some advantages if we want to use Kubernetes. Some of them are that we don't care about which cloud service we use, we have fine-graned container scheduling, and we use easily hybrids of cloud services. But we require some DevOps skills, and we have to manually tuning our resources.

8.7.6 Google Colab

Colab is a free, simple and web-based environment that is very useful for learning and experimenting. For storage, it can be connected to Google Drive. Spark can be installed easily on Colab.

Some problems with Colab

- Not suitable for large datasets
- No persistent cluster
- Good for demonstrations or teaching only
- For production we need dedicated or enterprise servers.

8.7.7 Lightning AI

Lightning AI is a cloud platform for distributed AI and data workflows. It can be integrated with Spark for large-scale preprocessing. It's focussed both on ML pipelines and not just ETL. In this integration Spark handles data ingestion and transformation while Lighting handles model training and orchestration. It's very suitable for AI labs and teaching advanced pipelines.

Load data from S3 or GCS →Use Spark to preprocess →Lightning AI trains model on cleaned data

Lightning AI supports GPU acceleration, auto-scaling and monitoring and Jupyter-like notebooks

8.8 Performance and data engineering in Spark

8.8.1 Why Spark jobs are slow

Different ways of running a query can have different results in terms of execution cost. Useless aggregations can result in worse performance, the use of filters can be advisable. In Spark moving data is more expensive than computing on it and by this Shuffle is the main bottleneck in terms of performance.

The golden rule in Spark to improve efficiency is to reduce data and movement, this can be achieved by

- Filtering early
- Select only needed columns
- Avoid unnecessary shuffle

8.8.2 How Spark optimizes queries

Spark does not optimize the queries as written instead it analyzes the query, it rewrites and chooses the most efficient execution. Every Spark job follows the pipeline

1. Query
2. Plan
3. Optimize
4. Execute

Usually Spark tries to reduce rows early, reduce columns, minimize shuffle with the goal of reducing data movement. For the Join Spark chooses Broadcast Join or Shuffle Join.

8.8.3 Data engineering for performance

Good performance is obtained by following these principles

1. Reduce data size
2. Avoid shuffle
3. Use efficient formats
4. Reuse computation
5. Use correct join strategy

One good format is Parquet, different from CSV because is columnar, compressed and fast. Data can be organized by key and by partition pruning.

Another useful element is caching, we save some results in the memory to reuse them. We should avoid caching if we are in a one-time use setting, data are very large and the memory is limited.

Joins are expensive operations, and we can have two types of them

- Broadcast →Small table sent to all nodes, large table stays distributed
- Shuffle →Large tables require redistribution, expensive because data move across nodes

If we apply repartition we have shuffling, with coalesce we don't.

Uneven data distribution can also be problematic since if one node is overloaded we can have a bottleneck. To handle this case

- Repartition
- Filtering early
- Key salting

The size of the files has an impact too, too many small files means overhead while too large files means poor parallelism.

The ETL pipeline

1. Raw
2. Clean
3. Transform
4. Store
5. Query

Real pipeline example

1. S3
2. Spark
3. Parquet
4. BI

The processing of the data can be periodic (Batch) or in real-time (streaming).

Performance checklist

- Use Parquet
- Filter early
- Select needed columns
- Avoid shuffle
- Use caching wisely

Common mistakes

- SELECT *
- Too many joins
- No partitioning
- Ignoring skew

8.9 Big data storage technologies and relation with Spark

8.9.1 Intro to big data storage

Given the scale and diversity of big data we can't use traditional relational databases. The problems come from the volume, the velocity and the variety of this kind of data. We need a scalable, distributed and flexible way to store our data. Most modern solutions employ horizontal scalability, fault tolerance, flexible schema and integration with different engines like Spark and Hadoop.

Recall that HDFS splits the data in block of 128 MB across nodes, and it's ideal for large sequential reads (batch analysis) but not for small and real-time queries. NoSQL technologies arose to handle these new workloads, we have the main 4 families:

1. Key-value Stores
2. Document Stores
3. Column-family store
4. Graph databases

Recall the CAP theorem, BASE and ACID principles from storing and retrieving.

Since no single database fits every use case modern architectures combine multiple storage types, this approach is called polyglot persistence.

8.9.2 Key-value DB

This is the simplest form of a NoSQL database. The data are stored as a pair key:value. It's optimized for constant-time lookups, it's schema-free and extremely fast. Some common systems are Redis, Amazon DynamoDB and Riak. They key must be unique, and the values can be of different types. Redis keeps the data in memory, so the operations will be very fast.

Some operations are

- SET key value →store data
- GET key →retrieve
- DEL key →delete
- INCR key →increment counters
- EXPIRE key seconds →set time-to-live

Strengths

- Extremely fast reads/writes
- Simple data access model
- Scales horizontally

Limitations

- No complex queries and joins
- Limited analytical capability

The best use cases are caching, gaming and high-traffic web apps. This approach generalizes the concept of Associative Keys to databases.

8.9.3 Document DB - MongoDB focus

This kind of databases store data as JSON-like documents. The schema is flexible, and we can easily query using fields and indexes. Some examples are MongoDB and Couchbase. It's ideal for semi-structured data such as catalogs, users and logs. MongoDB is the most popular document database. It's open-source, widely adopted and stores data and BSON (binary JSON). It balances performance, flexibility and scalability. Used by organizations like eBay, Forbes and CERN.

The hierarchy is the following

1. Database
2. Collections
3. Documents

Even though the schema is not fixed we can still validate it. Every MongoDB document needs to have a '_id'.

The usual operations are CRUD (create, read, update and delete)

1. Insert
2. Read
3. Update
4. Delete

Using indexes we can improve the query performance. Indexes can be

- Single-field
- Compound
- Text and geo-indexes

The aggregation in MongoDB works like a pipeline with some common stages like $match, $group, $sort and $project.

Replication in MongoDB ensures data redundancy and fault tolerance by copying data across multiple servers, called replica sets. A replica is made of primary node (handles all write operations) and secondary node (replicate data from the primary asynchronously). If the primary fails, one secondary becomes the new primary (automatic failover) and the clients continue reading from any node. The benefits of this approach are high availability, fault tolerance, data redundancy and read scalability.

Sharding is another tool where data are distributed across multiple servers (shards) so that no single node has to handle all the load or store all the data.

1. Data is split into chunks based on a shard key
2. Each chunk is stored on a different shard
3. A config server keeps track of where each chunk resides
4. A mongo's router direct client queries to the appropriate shard

The benefits are horizontal scalability, balance of read and write load across nodes and geographical distribution is supported.

In production, we usually combine replication ad sharding to make global scalability possible. Each shard is a replica set, this provides both scalability and fault tolerance.

MongoDB has some strengths (flexible schema, JSON integration, great for rapid prototyping and rich aggregation features) and limitations (joins are limited, ACID are weaker, and we require index management)

The main ecosystems related to MongoDB are

- MongoDB Atlas
- Compass
- BI Connector
- Spark Connector

8.9.4 Wide column DB

This typology of database store data in rows and dynamic columns grouped into column families. It's inspired by Google's Bigtable. Some examples are Apache Cassandra and Apache HBase. The first one doesn't rely on Hadoop while the second one does. Wide columns DB are best for high-write, time-series and IoT data.

The table structure is like relational databases but with more flexibility and scalability. Each row will have a different set of columns, and columns are grouped into column families rather than fixed tables (like a nested key-value store). Data is stored by column rather than by row. This structure is optimized for aggregations on specific columns. The usual Cassandra operations are Create, Insert, Query and Delete.

As always we have clusters (here peer-to-peer with no master node), data is replicated across nodes for fault tolerance, and we have tunable consistency (one, quorum and all). We can see Wide columns DBs as a collection of spreadsheets where each row have its own set of columns depending on what data it has.

Some design guidelines are

- Denormalize instead of join
- Use both column names and column values to store data
- Model an entity with a single row

8.9.5 Graph DB

Graph databases store data as nodes, edges and properties in order to traverse relationships easily. Some examples are Neo4j, JanusGraph and TigerGraph. With this structure queries can be fast and intuitive. Some easy common use cases are social networks, fraud detection, knowledge graphs and recommendation systems.

8.9.6 Other DB types

We also have additional kinds of databases like

- Search databases →Optimized for text search and analytics. Some examples are Elasticsearch and Apache Solr. Inverted indexes are used for fast keywords and full-text queries. These are great for log analytics, monitoring and search engines.
- Time-series databases →Specialized for sequential data. Some examples are InfluxDB, TimescaleDB and Prometheus. This structure is optimized for regular insertions and aggregations over time windows
- Vector databases →New generation databases for AI and similarity search. Some examples are Pinecone, Weaviate, Milvus and FAISS. The database stores embeddings of our numerical vectors and can be used for semantic search, recommendation and image/text similarity.

8.10 Machine Learning in Spark

8.10.1 Introduction

As we know machine learning is the process of extracting pattern from data using statistics, linear algebra and numerical optimization. It can be applied to many problems. We have the following types of machine learning

- Supervised
- Unsupervised
- Reinforcement learning
- Semi-supervised
- Self-supervised

Here we will see mainly supervised and unsupervised. Then we will talk about classification, regression, clustering, dimensionality reduction and association rules. Some algorithms in Spark MLlib for unsupervised learning are

- K-means
- Latent Dirichlet Allocation

- Gaussian mixture models

We want to use machine learning in Spark because our big data are too large for single machine machine learning. Spark MLlib is a scalable ML library built on top of Spark. Some key features are

- It works on distributed DataFrames
- Supports both supervised and unsupervised machine learning
- Integrates with pipelines

Originally it worked with RDDs, now is dataframe based. This library provides

- Transformers
- Estimators
- Pipelines

The advantages of MLlib are

- Distributed training on clusters
- Compatible with Spark SQL and DataFrames
- Integration with streaming and GraphX
- Used in production

8.10.2 Spark MLlib concepts

Transformers
These are functions that transform DataFrames using fixed rules. They do not learn from the data and implement the transform(). We get input→output columns. These functions are usually used in preprocessing. Some examples are Tokenizer, VectorAssembler, HashingTF, StandardScalerModel.

Estimators
These are algorithms that fit models and learn from the data. Implement the fit() method that in return gives us a model. Some examples are LogisticRegression, Kmeans and LinearRegression.

A special case is the one of estimators that learn transformations. They basically learn on how to transform data for feature engineering. They implement with fit() and produce a Transformer. Some examples are StringIndexer, OneHotEncoder and StandardScaler.

Pipelines
Pipelines are chains of transformations and estimators used to standardize the ML workflows. For example preprocessing →model training →evaluation.

Feature engineering
In Spark MLlib we use feature engineering to transform raw data into meaningful numerical features via indexing, encoding, scaling and assembling in order to improve our model performance in distributed ML pipelines. Some common transformers are

- StringIndexer →Categorical encoding
- VectorAssembler →Combine features
- StandardScaler →Normalize data

8.10.3 Algorithms

For classification

- Logistic regression
- Decision trees
- Random forests

For regression

- Linear regression
- Gradient boosted trees

For clustering

- KMeans
- Gaussian mixture

Spark MLlib includes ALS (Alternating Least Squares)

8.10.4 Evaluation

In MLlib we have the following metrics

- Accuracy
- Precision
- Recall
- RMSE
- R^2
- Silhouette score

Recall that we have to split our dataset into train, test and validation and evaluate the model with the right transformations avoiding leakage. We can apply cross-validation.

8.10.5 Advanced Topics

Spark MLlib is mainly for classical machine learning, to perform deep learning tasks we can integrate TensorFlow and PyTorch using

- BigDL →run DL models directly in Spark clusters
- Horovod →enables distributed training of TensorFlow and PyTorch models using Spark for data preprocessing and coordination.

We can apply these models on real-time data to work with streaming.

MLlib has some limitations on the side of deep learning support, the algorithms are not flexible and debugging on distributed nodes can be tricky.

8.11 Pipelines in Spark

8.11.1 Introduction

As we know the ML workflows involve multiple steps like cleaning, engineering training and evaluation. Without pipelines the code can be messy, with them, we can standardize it by using modular workflows. The key idea is that a pipeline is a sequence of stages, mainly transformers (to process data) and estimators (to learn from these data). The idea is similar to the one present is Scikit-learn but applied to big data.

8.11.2 Spark pipeline API

The transformers stage include taking a dataframe as input and to output a transformed version of it. We want to apply fixed rules to implement to transform function. We are not learning from the data.

The estimators phase on the other hand is made of algorithms that learn from data. They implement the fit function (it produces a model). We also have some special cases of estimators that learn transformations, their fitted models are the transformers used in pipelines. These estimators will learn rules to do feature transformation but not to do predictive tasks. We must fit them before the use.

As we know a pipeline is an ordered list of stages where the data flows through each one in order. Pipelines are good because they are

- Modular and reusable
- Automates repetitive steps
- Scales across distributed Spark clusters
- Encourage clean ML engineering practices

8.11.3 Feature engineering pipelines

Some useful pipelines can be

1. **StringIndexer + OneHotEncoder** →To convert categorical values to numbers or to binary encode them
2. **VectorAssembler** →Combines multiple feature columns into a single one
3. **StandardScaler** →Standardizes the feature into an N(0, 1) fashion to help our algorithms converging faster

8.11.4 Model training and evaluation

To train a pipeline we use the fit method, it will create a pipeline. To apply it we use the transform method. The evaluators provided by Spark MLlib are for classification, regression and clustering. We can evaluate a model using the usual metrics.

8.11.5 Hyperparameter tuning

To improve our models we can use some built-in functions such as grid search, cross validation and the split in validation and training.

8.11.6 Advanced Pipelines

Some advanced concepts can be persistence and deployment of pipelines. With MLflow we can track experiments, manage models and deploy pipelines efficiently.

This can be used to enable reproducibility and deployment. We can also integrate Spark SQL and MLlib or to real time streaming (structured streaming).

8.12 Graph Analytics

8.12.1 Introduction

As we know many real-world problems involve networks and traditional databases are not optimized for this kind of relationships. Using graph analytics we can get insights from nodes and edges. The basic terms of graphs are the following:

- Node (or vertex) →entity
- Edge →relationship between nodes
- Directed/Undirected, Weighted/Unweighted

We can apply graph analytics to do tasks like

- Community detection
- Suspicious pattern detection
- Link prediction
- Path optimization
- Interaction network

8.12.2 Spark GraphX

One native Spark graph analytics library is GraphX. It's built on top of RDDs, and it combines both ETL and graph computation.

We consider a graph as a pair of RDDs and with this library we can use graph operators and algorithms. Some common operations are

- mapVertices/mapEdges →transform attributes
- joinVertices →enrich data
- subgraph →filter
- triplets →combine vertices + edges

In GraphX we have some built-in algorithms that can be used, like:

- PageRank
- Connected Components
- Triangle counting
- The Shortest Paths
- Label propagation (communities)

8.12.3 GraphFrames

Since we can use GraphX only with Scala and since it's mainly RDD based we would like to have a framework to work with DataFrames and Python. Luckily we have GraphFrames. It also integrates well ML and SQL.

We can explore our graph using graph queries. If we want to do some pattern matching operation we can use something called motif finding. Additional operations are for computation of degrees (inDegrees, outDegrees and degrees), aggregations, page rank (node importance), connected components (find group of nodes), label propagation (find communities) and breadth first search (the shortest path, reachability and relationship discovery).

8.12.4 Advanced topics

Graph features can be used inside ML pipelines for different tasks. These graphs can be used for streaming of data since they can evolve dynamically.

The storage options are usually:

- HDFS
- Cassandra
- Neo4j

With these structures we can encounter problems if our graphs are huge since we can fall inside problems with our partitions. Also, the algorithms might not parallelize well. Some useful ways to solve the problems are partitioning, caching, check pointing and query optimization.

8.12.5 Algorithms in details

Here we can see how our algorithms can work

- PageRank →We consider a node important if many important nodes link to it. We iteratively distribute rank across the graph to find which is the most influential node.
- Connected components →We group vertices connected by paths
- The Shortest path →We want to find the minimum distance path between nodes by applying algorithms like Dijkstra or BFS.
- Triangle counting →We have a triangle when 3 nodes are all connected, this can tell us the clustering coefficient (how tight a network is).

8.13 Streaming

8.13.1 Introduction

Traditional batch processing is done on bins of hours or days, but many modern apps require insights done in real time. In this setting streaming analytics allow us to process events as they arrive.

Before Spark most engines used a record-at-a-time model, one by one. This approach is fast but difficult to operate at scale. Spark then introduced a new model to simplify scaling and reliability. This approach is based on micro-batch processing where data is grouped into small batches. Each batch is processed using deterministic and parallel tasks. The key advantages are that we have a strong fault tolerance, exactly once semantics and deterministic compute are ensured and operations are simplified. Latency is a bit higher but still suitable.

8.13.2 Structured streaming overview

Structured streaming is built on Spark SQL and DataFrame API. The key idea is that we consider streaming as incremental batch, so developers write queries as if static while Spark run them continuously.

Structured streaming treats a data stream as an unbounded table (a new event is a new row appended to this table). The developer will write a normal DataFrame or SQL query on that table (as for batch) while Spark incrementalizes the query, each micro-batch updates the result table. The triggers decide when to update the result and sinks decide where it is written.

8.13.3 Streaming concepts

Some useful concepts about streaming

- **Micro-Batch model** →Spark processes small batches of data, we want a balance between throughput and latency. This is transparent to developers
- **Event time vs Processing time** →Event (when the event occurs) or processing (when Spark processes it). Structured Streaming handles late data with watermarks
- **Windowing** →Data are grouped into time windows
- **Exactly-one semantics** →Spark guarantees no duplicate processing even when failures happen, this is important for finance and transactions. We have it when we have replayable source, deterministic transformations, idempotent or transactional sink and a stable checkpoint directory
- **Checkpointing and fault tolerance** →States are stored in checkpoints, this allows recovery after failure. Key for production streaming jobs

8.13.4 Spark structured streaming API

Usual input sources are Kafka, Socket, files and cloud storage. Output sinks usually are console, files, Kafka and databases.

Kafka

Kafka is a distributed publishing/subscribe platform designed for horizontal scalability. It's often used as a scalable buffer for streaming data into and out of Hadoop storage systems. Acts as middleware that guarantees durable ingestion from diverse data sources. Distributed sequential logs are used to store messages allowing clients or consumer groups to read from any position via simple numerical offsets. Output modes can be append, complete and update.

As we know structured streaming keeps a result table that is updated every trigger. The output mode controls which rows from that table get written to the sink.

Append

- Writes only new rows to the result table since last trigger
- It requires that those rows will never change later
- Typical queries are stateless transforms and event-time window aggregations with watermark
- Good for append-only sinks like files or data lakes

Update

- Writes only rows whose values changed since the last trigger
- Can output the same key multiple times as aggregates evolve
- Typical queries are stateful ones
- It's good for sinks that can handle upserts and overwrites

Complete

- It writes the entire result table every trigger
- Only practical when the resultant table is small
- Typically used for debugging, dashboards or in-memory tables where a full refresh is cheap
- Not ideal for file sinks

With this information we should use append for immutable and append only results (files), update for changing aggregates with updatable sinks and complete only when the result is small, and we want a full refresh each time.

A structured streaming recipe follows this pipeline

1. Define the source
2. Transform the DataFrame
3. Choose sink + output mode
4. Configure trigger + checkpoint
5. Start and monitor the query

8.13.5 Advanced streaming

Some ways to improve our streaming can be using triggers (control how often the streaming query processes new data), use stateless (no stored history) or stateful (keeps running state) transformations, use the right aggregations and joins (stream-to-stream or stream-to-batch), use ML models and graph analytics. For performance and scaling we can tune batch interval, use state TTLs, prefer append mode, compress checkpoints and monitor latency.

8.13.6 Challenges and best practices

Some challenges in Streaming can be

- Out-of-order events
- Late arrivals
- Fault tolerance at scale
- Cost of always-on clusters

Best practices

- Use watermarks for late data
- Keep stateless when possible
- Monitor with Spark UI and logs
- Test with small replayable streams

To monitor our streaming apps we can use metrics embedded in Spark UI, do some latency monitoring and integrate with Prometheus or Grafana

8.13.7 Deep dives

In more details

- Windowed aggregation →We can have a fixed window, a sliding window or a session window
- Watermarks for late data →To handle delayed events, if an event is later than the selected time frame it will be ignored.
- Stream joins →Can be stream-to-stream or stream-to-batch

8.14 The future of Big Data

8.14.1 Intro

As Big Data tech evolves rapidly it will happen that tools that work today like Hadoop and Spark they won't do it in the future.

8.14.2 Current limitations of Big Data

As we know in our setting we can have problematic limitations. Hadoop and Spark for example by having a batch-first architecture will have high latency. Also, they entail a heavy ecosystem, they are heavy on the disk and the support for current tasks is limited.

From the point of view of cost and complexity we can say that maintaining on-prem clusters is expensive, data duplication can happen, and we need to comply with different regulations. Also, the shift is happening on the professional level.

8.14.3 Emerging trends

Some emerging trends are

- Shift from on-prem Hadoop to cloud
- Data lakehouse are being used much more (scalability of lakes + governance of warehouse)
- Shift from batch to real time insights
- Lower complexity for unified batch and streaming
- The need for serverless Big Data

8.14.4 AI and Big Data integration

ML can be used for automatic indexing and so for query optimization, this can reduce manual tuning overhead. We can also integrate LLMs to automate different steps or applying natural language querying. More human tasks like report writing, dashboard and explanations can be automated.

8.14.5 Privacy, ethics and regulation

Regulations keep being written, the main problems are related to data residency and sovereignty in global cloud. A must-have is the application of encryption and differential privacy.

The main ethical concerns (if did not have ethics then you won't understand) are:

- Bias in AI pipelines
- Surveillance risks
- The need for explainability

In the future we will have metadata driven platforms automated lineage tracking and the embedding of transparency and auditability. Datacenters will be required to use renewable energy and carbon-aware scheduling.

8.14.6 Semantic data and interoperability

Future Big Data will not only store data but also understand it. Some ways will be the use of RDF, OWL and knowledge graphs. Metadata catalogs and semantic layers link raw data to business meaning.

8.14.7 MLOps, DataOps, and AIOps convergence

In the future we will have much more automation across the lifecycle of our analytics

- DataOps →Automates data ingestion, quality and lineage
- MLOps →Automates model training, validation and deployment
- AIOps →Uses AI to optimize and monitor data and ML pipelines

8.14.8 Research and open challenges

We still need to solve some problems like:

- Scalability challenges
- Processing at edge
- Quantum computing application

9 Neural and evolutionary learning

9.1 Machine learning review

The traditional computational method involves the following pipeline: Problem →Algorithm →Program. When we encounter a problem for which it's difficult to come up with an algorithm we can go towards machine learning based approaches. In these approaches we try to give the computer the ability to learn how to solve those problems.

A good way of learning is improving thanks to our own experience, a cycle where we do mistakes, we try to understand what was the mistake, why we did that mistake, and we try again while getting a reward whenever we do something good. Our systems will basically do this. In the context of machine learning we always try to learn a function.

The problems always involve a set of data and a function that tries to be as similar as possible the one that generated the initial set of data D.

$$\forall i \in \{1, 2, \ldots, N\}, \quad \Phi(x_{i1}, x_{i2}, \ldots, x_{iM}) = y_i$$

Where

1. Φ →is our target function
2. **D** →is our dataset
3. We call learn the process that allows us to find a function **f** that approximates Φ
4. **f** →is the function returned by the learning process and is called data model
5. $y_1, y_2, ..., y_M$ →are called target values

If the target values are known we talk about supervised learning, if they are unknown we are talking about unsupervised learning. Our function f should have good generalization ability (it behaves like Φ also with data that do not belong to D). This is a crucial concept in machine learning.
If this generalization ability is not good we say that f overfits the data, and this issue is known as overfitting. Generally we assume that our data are generated following a certain process and that we have knowledge hidden in our data. We want to learn this knowledge and build follow-up knowledge that can be used with data that are outside our training set.

The process of learning can be applied to two main tasks:

- Classification →The co-domain of the target values is a discrete set of limited dimensions called classes. We want to partition our data into these classes
- Regression →We try to find a function that receives the data and tries to output a continuous value

When we receive new data in a supervised setting we apply the function to the new data, and we accept the result (prediction). In the case of unsupervised we insert our new datum inside the class that is most similar to it.

9.1.1 Performance measures for regression

- **Mean Absolute Error (MAE)**

$$MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|$$

  where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $|y_i - \hat{y}_i|$ = absolute error between actual and predicted value

- **Mean Squared Error (MSE)**

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $(y_i - \hat{y}_i)^2$ = squared error between actual and predicted value

- **Root Mean Squared Error (RMSE)**

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

where:

  - N = total number of observations
  - y_i = actual (true) value of the i-th observation
  - $\hat{y}_i$ = predicted value of the i-th observation
  - $(y_i - \hat{y}_i)^2$ = squared error
  - $\sqrt{\cdot}$ = square root operation

- **Pearson Correlation Coefficient**

$$r = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{N}(y_i - \bar{y})^2}}$$

where:

  - N = total number of paired observations
  - x_i = value of the first variable for the i-th observation
  - y_i = value of the second variable for the i-th observation
  - $\bar{x}$ = mean value of variable x
  - $\bar{y}$ = mean value of variable y
  - r = Pearson correlation coefficient

MAE is not very sensitive to outliers in comparison to MSE since it doesn't punish huge errors. MSE is more accurate than MAE since it highlights large errors over small ones. MSE is also differentiable, so it can help to find maximums and minimums more effectively but by squaring all the values some errors are extremized. We can use RMSE to reduce the impact of this last point.

- MAE →when outliers are low
- MSE →when we have many outliers but not a lot of noise
- RMSE →if our large errors affect a lot the model's performance

9.1.2 Performance measures for classification

- **Accuracy**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

  - TP = true positives
  - TN = true negatives

  - FP = false positives
  - FN = false negatives
  - $TP + TN + FP + FN$ = total number of predictions

- **Precision**

$$Precision = \frac{TP}{TP + FP}$$

  where:

  - TP = true positives
  - FP = false positives
  - $TP + FP$ = total predicted positive instances

- **Recall**

$$Recall = \frac{TP}{TP + FN}$$

  where:

  - TP = true positives
  - FN = false negatives
  - $TP + FN$ = total actual positive instances

- **F1-Measure**

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

  where:

  - $Precision$ = proportion of correctly predicted positive instances
  - $Recall$ = proportion of correctly identified actual positive instances
  - $F1$ = harmonic mean of Precision and Recall

- **Cohen's Kappa Measure**

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

  where:

  - κ = Cohen's Kappa coefficient
  - p_o = observed agreement between predictions and true labels
  - p_e = expected agreement by chance

- **ROC Curve**

$$TPR = \frac{TP}{TP + FN} \qquad FPR = \frac{FP}{FP + TN}$$

  where:

  - TPR = true positive rate (Recall)
  - FPR = false positive rate
  - TP = true positives
  - FP = false positives
  - TN = true negatives
  - FN = false negatives
  - ROC curve = graphical plot of TPR against FPR

- **Area Under the Curve (AUC)**

$$AUC = \int_0^1 TPR(FPR)\, d(FPR)$$

  where:

  - AUC = area under the ROC curve
  - TPR = true positive rate
  - FPR = false positive rate
  - higher AUC indicates better classifier performance

9.1.3 Features

Our data are characterized by their meaning, in this case we call those meanings features. Features can be useful if they help us to make a difference in our decision. An appropriate choice of the features is crucial for our models. Redundancy can slow down our models, so we want to cut the number of features as much as possible while retaining the biggest amount of information. We have many ways to do feature selection, and we usually do it in the pre-processing phase.

9.1.4 Experimental ML study

Since we can't know a priori the best model possible for our task we have to do some trial and error experiments. We should check as many algorithms as possible to be sure that our search space was covered enough.

The general pipeline of a machine learning study follows these steps

- Data preprocessing
- Choice of the method and the parameters
- Estimating the predictive error
- Inducing the final model
- Testing the final model

When we split the data we need to remember that we shouldn't do anything on the test set if we learn information from it (data leakage). If we don't have a naturally given test set we should get it from the learning one before doing any transformation.

The typical transformations include normalization, errors cleaning, outliers removal, missing values imputation and feature engineering. Remember to base the transformations to the knowledge present in the training data only. Normalization of the test set for example should be done only with parameters learned the training set. Anyway we should know the expected error of our model.

We can also get the validation set to find the best possible parameters and to assess overfitting. One way to have a more general partition of our data is obtained by using the so-called cross-validation. We can have different kinds of cross-validation like simple k-fold, nested and Monte Carlo.

9.2 Genetic programming

Genetic algorithms are inspired from natures, they are not the only ones since we also have ANNs, particle swarm intelligence, fuzzy systems and evolutionary algorithms.

9.2.1 Evolutionary algorithms

We have two big hypotheses

- We have a problem that has a huge amount of possible solutions, among which we want to find the best one
- For each existing solution we are able to quantify its quality by means of a function called fitness (usually we associate a real-valued number to each solution)

The evolutionary process starts from a population to which we select some parents (ability of adapting, competition). These parents will then reproduce with some sort of variation (given by our genetic operators, usually crossover and mutation are applied). After this we will have our offspring population that will go into our original population (survival selection). At the reaching of a certain stopping condition we will select our final solution, the one with the highest fitness.

The two most common forms of evolutionary algorithms are

- Genetic algorithms →Our individuals are strings of constant length, and we usually try to use them for optimization
- Genetic programming →Individuals are computer programs, we can use this approach for machine learning.

The selection mechanism is similar since for both cases is done using fitness values. So we have things like fitness proportionate selection, ranking selection and tournament selection. The simplest one is fitness proportionate selection that takes the name of roulette wheel.

In genetic programming our representation is done by using trees. We can express a mathematical expression with this data-structure. We can define a tree using internal (F) and terminal nodes (T). We have many more ways.

Our fitness is a measure of the deviation between our expected target and the calculated output. So for regression we have RMSE, MAE or correlation while for classification we can have accuracy or F1.

With trees crossover can be done by moving around branches into the other tree. Mutation can be done by adding a branch randomly.

Genetic programming pseudocode

Algorithm 1 Genetic Programming Algorithm

1: 1. Initialize a population P of N programs (individuals) (typically at random)
2: 2. Repeat until termination condition:
3: 2.1 Execute each program in P and assign it a fitness according to how well it solves the problem
4: 2.2 Create an empty population P'
5: 2.3 Repeat until P' contains N individuals:
6: 2.3.1 Choose a genetic operator among crossover (with probability p_c), mutation (with probability p_m) and replication (with probability p_r). Remark that $p_c + p_m + p_r = 1$
7: 2.3.2 If the operator chosen in 2.3.1 was either mutation or replication, then select one individual from P (using one of the selection algorithms), apply the operator and insert the resulting individual in P'
8: 2.3.3 If the operator chosen in 2.3.1 was crossover, then select two individuals from P (using one of the selection algorithms), apply the operator and insert the resulting individuals in P'
9: 2.4 Selection of survivors: Extract one new current population P from P and P' (typically $P := P'$)
10: 3. Return the best individual in P

9.2.2 Symbolic regression

Symbolic regression is a task for which given a set of input-output pairs I_i, O_i we want to find (or approximate) a function that maps those points ($f(I_i) = O_i$) for any i.

Given a matrix with 2 input columns and 1 output column we have a regression in 2 variables. We will apply the following operators $+, -, *, //$ randomly in a tree and run our data. By calculating our fitness we have our first idea of how good our solution is. After building our population we will find the fitness of each point, we will select the parents with the roulette wheel, crossover and mutation will be applied, and our new population is there. We will re-run the algorithm until our stopping condition is met.

9.2.3 Why do evolutionary algorithms work?

Let

- s_t be the best individual in the population at generation t
- S_O be the set of all globally optimal solutions in the search space

$$\lim_{t \to \infty} P(s_t \in S_O) = 1$$

9.2.4 Peculiarities of genetic programming

The initialization of the population is the first step of the evolution process. It consists in the creation of the program structures that will later be evolved. The most common initialization methods in tree-based GP are

- Grow
- Full
- Ramped half-and-half

Grow When the "Grow" method is employed each tree of the initial population is built using the following algorithm

- A random symbol is selected with uniform probability from $\mathcal{F}$ to be the root of the tree.
- Let n be the arity of the selected function symbol. Then n nodes are selected with uniform probability from the set $\mathcal{F} \cup \mathcal{T}$ to be its sons
- For each function symbol between these n nodes, the method is recursively applied (its sons are selected from the set $\mathcal{F} \cup \mathcal{T}$), unless this symbol has a depth equal to d - 1. In the latter case its sons are selected from $\mathcal{T}$

Basically we avoid having single node trees at the starting point. Nodes with depths between 1 and d - 1 are selected with uniform probability from $\mathcal{F} \cup \mathcal{T}$, but once a branch contains a terminal node, that branch has ended, even if the maximum depth d has not been reached. Finally, nodes at depth d are chosen with uniform probability from $\mathcal{T}$. Since the incidence of choosing terminals from $\mathcal{F} \cup \mathcal{T}$ is random throughout initialization, trees initialized using this method are likely to have irregular shape (branches of various different shape).

Full Initialization

Instead of selecting nodes from $\mathcal{F} \cup \mathcal{T}$, the full method chooses only function symbols until a node is at the maximum depth. Then it chooses only terminals. The resulting tree has maximum depth for every branch.

Ramped Half-and-Half

Populations initialized with the above two methods might be built with trees that are too similar between them. In genetic programming diversity is really important, then to enhance the diversity of the population we can use this other technique. We start from d (maximum depth parameter) and the population is divided equally among individuals to be initialized with trees having depths equal to 1, 2, ..., d - 1, d. For each depth group, half of the trees are initialized with the full technique and half with the grow one.

Parameters

Once the set of functions and terminals are set, and we have the exact implementation for the genetic operators we have to select the parameters to use. Some are:

- Population size
- Stopping criterion
- Initialization technique
- Selection algorithm
- Crossover type and rate
- Mutation type and rate
- Maximum tree depth
- Presence or absence of steady state
- Presence or absence of ADFs or other modular structures
- Presence of absence of elitism

9.2.5 Weak points of genetic programming

We still have some problems that might arise

- Bloat →solvable with dynamic limits, tarpeian method or operator equalization
- Premature convergence (loss of diversity) →solvable with early stop/restart or with parallel and distributed genetic programming
- Overfitting →solvable with early stop/restart or multi-objective optimization

We can incorporate genetic programming into machine learning pipelines for our algorithms.

9.3 Geometric semantic genetic programming

Like Pokémon, we have evolutions with new operators. New crossovers and mutations that supposedly are better.

9.3.1 Optimization problems

Informally solving an optimization problem means to find the best solution(s) in a typically huge set of other candidate solutions. Formally a pair (S, f) where S is the set of all possible solutions (search space) and f is a function such that f = S $\rightarrow\mathbb{R}$. This function f quantifies the quality of the solutions in S and is called cost or fitness function.

An optimization problem is a minimization problem if it consists in looking for a solution o $\in$ S such that $f(o) \leq f(i), \forall i \in S$.
An optimization problem is a maximization one if it consists in looking for a solution o $\in$ S such that $f(o) \geq f(i), \forall i \in S$.
Such a solution is called global optimum (minimum or maximum).

The most natural and immediate method to solve an optimization problem is Hill climbing. It consists in trying to improve fitness step by step (stepwise improvement) by means of the concept of neighborhood. A solution $j \in S$ is called local optimum if

- for minimization problems $f(j) \leq f(i) \forall i \in N_j$
- for maximization problems $f(j) \geq f(i) \forall i \in N_j$

With hill climbing we will always have a local optimum that is not necessarily the global one.

9.3.2 Fitness Landscapes

If we want to represent the evolution of our fitness with respect to our solutions we can plot the fitness landscape. Formally defined as

- The fitness landscape can be represented as (S, V, f)
- S: set of potential solutions
- V: S $\rightarrow 2^S$, neighborhood function
- f: S $\rightarrow\mathbb{R}$, fitness function

Given our neighborhood function $\forall x \in S$

- $V(x) = \{y \in S | y = op(x)\}$
- $V(x) = \{y \in S | d(x, y) \leq 1\}$

The fitness landscape will give us a visual intuition of the facility or difficulty of a search agent to find the global optimum. A smooth landscape will imply an easy problem, while a rugged one will have many local optima and so a harder problem. In reality is impossible to draw a fitness landscape since our search spaces and neighborhoods will be huge.

The operator that is related to the neighborhood structure is called ball mutation. If we have a continuous fitness with a known optimum we have a CONO (continuous optimization with notorious optimum).

9.3.3 Genetic algorithms

Genetic algorithms are related to solutions where our individuals are strings of prefixed length. We can apply the usual crossover and mutation. We are not obliged to work with bits, but we can also use vectors of continuous values. We have many operators like Geometric ones. Geometric crossover for example differs from the normal one. The coordinates of the offspring are the weighted average of the corresponding coordinates of the parents. In this case the offspring can't be worse than the parents.

Mutation is implemented by applying ball mutation, we move our points inside a radius.

9.3.4 Genetic programming

If in our evolutionary algorithms the individuals are computer programs we entered the setting of genetic programming. One example is Symbolic Regression where given a matrix we try to approximate the function that built it.

Genetic programming can be used as a machine learning method.

- Known: the correct output for a fixed given set of inputs $\{I_i, O_i\}$.
- Sought: a function belonging to a certain class that interpolates those points, so $f(I_i) = O_i \forall i$
- Output vector: the vector of the outputs of f is f(I) = f(I_i)
- Fitness: a measure of the error on the training set. Basically the distance between the output vectors of f and the target output vector $F(f) = D(f(I), O)$

We can apply this in a context of supervised learning.

9.3.5 Implementation of geometric semantic operators

We can save genetic information by applying the concept of semantics, two programs can have a very different structure but if the produced results are always the same they share the semantics.

Let $X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]$ be the set of input data (fitness cases) of a supervised learning problem and $\mathbf{t} = [t_1, t_2, \ldots, t_n]$ the vector of the respective expected output or target values.

A GP individual (or program) P can be seen as a function that, for each input vector $\mathbf{x}_i$, returns the scalar value $P(\mathbf{x}_i)$.

We define the semantics of an individual P as the vector

$$\mathbf{s}_P = [P(\mathbf{x}_1), P(\mathbf{x}_2), \ldots, P(\mathbf{x}_n)]$$

This is a point in an n-dimensional space.

We can map our programs into a semantic space (output) and visualize how far from the target they are. Traditional genetic operators will produce offspring by blindly manipulating the parents without caring about the semantics. We want to preserve this syntactic genetic material and define transformations that make sense from a semantic point of view.

If we are able to find a transformation on the syntax of the individual whose effect is ball mutation on the semantic space we induce an unimodal fitness landscape on every problem consisting in matching input data into known targets. Genetic programming should have a good evolvability on those problems.

9.3.6 Discussion

Ongoing research is being done to create the aforementioned operators

- Geometric semantic crossover →The only offspring generated by this crossover has a semantic vector that is a linear combination of the semantics of the parents with random coefficients included in [0, 1] and whose sum is equal to 1.
- Geometric semantic mutation →Each element of the semantic vector of the offspring is a weak perturbation of the corresponding element in the parent's semantic. We can tune its importance. It's basically a ball mutation in the semantic space, so we induce an unimodal fitness landscape.

These semantic operators can have some drawbacks, for example at each generation the offspring will be larger than the parents causing a fast growth in sizes. This makes the basically useless. One solution can be simplifying these individuals during the evolution.

9.3.7 Open issues

We still have some issues, one of them is that reconstructing the best individual can be impossible after too many generations. Also, these models are basically black boxes.

9.4 Semantic Learning algorithm based on inflate and deflate mutations

We left the last class with one problem, the genetic operators always create offspring bigger than their parents. Then our final model is huge and very difficult to interpret. We have now SLIM-GSGP (Semantic Learning algorithm based on Inflate and deflate Mutations).

We redefine Geometric Semantic Mutation as →given a parent function $T : R^n \to R$, GSM with mutation step ms returns the function $GSM(T) = T + \Delta T$. Where

- $\Delta T = ms(T_{R1} - T_{R2})$
- T_{R1} and T_{R2} are random functions.
- $\Delta T \in$[-ms, ms]

Note that the expression $T_{R1} - T_{R2}$ is used as a zero-centered random expression. T_{R1} and T_{R2} are normalized in the range [0, 1] applying a sigmoid. Then no crossover is needed in GSGP.

SLIM-GSGP follows two main steps

- If in the definition of GSM ΔT was subtracted, instead of added, the resulting operator would be equivalent to GSM.
- Given that the GSM uses + and the one that uses - are equivalent, we can alternate them without modifying the GSGP dynamics

The last step is called deflated mutation if the offspring is smaller than the parent. This is opposed to traditional GSM where we have inflated mutation. We can implement this by removing a term that was previously added.
Notice that

- Deflate mutation is a geometric semantic mutation
- Deflate mutation is not a backtracking operator

We need then to define a function with values in [-ms, ms] and one to obtain ball mutation on the semantic space.

Let $GSM(T) = T + \Delta T$, and S the sigmoid, then we can have the following ways of defining that ΔT. How to build a function in [-ms, ms]

1. $SIG2 = ms(S(T_{R1}) - S(T_{R2}))$
2. $ABS = ms(1 - \frac{2}{1+|T_R|})$
3. $SIG1 = ms(2S(T_R) - 1)$

How to get a ball mutation on semantic space

1. Summing a quantity close to 0, as in traditional GSM
2. Multiplying by a quantity close to 1, this can be done with
    - 1 + SIG2
    - 1 + ABS
    - 1 + SIG1

Then we have the following six variants of SLIM-GSGP

- SLIM + SIG2
- SLIM + ABS
- SLIM + SIG1
- SLIM * SIG2
- SLIM * ABS

- SLIM * SIG1

One very easy implementation of SLIM-GSGP can be done using a linked list.

The crossover can be implemented in different ways

- Swap crossover →swaps items of the linked list between parents, allowing recombination of genetic material while keeping offspring the same size as the parents
- Donor crossover →one parent donates a block to the other parent

These crossovers sometimes can enhance the predictive power of SLIM.

9.5 Neural networks

9.5.1 Introduction

ANNs are computational techniques that belong to the field of Machine Learning. The aim of these architectures is to mimic in a very simplified way the human brain. As we know the brain is composed by a set of simpler elements called neurons. The power of the brain is obtained by how these neurons interact and communicate between themselves.

Every neuron can do very simple computations but by combining a lot of them we can emulate more complex functions. The base structure in the field of neural networks is the so-called Perceptron

9.5.2 Perceptron

A Perceptron ca be considered the most basic neural network possible. By putting more of them together we can get more layer and so a multilayer Perceptron. One limitation of the perceptron is that the flow of information inside it is uni-directional (feed-forward neural network). The Perceptron is composed by one or more output neurons, each of which is connected to several input units by means of connections that are characterized by weights. In analogy with biology, these connections are called synapsis.

As we said the simplest structure is composed by just one neuron. The output of a perceptron can be calculated as

$$y = f(\sum_{i=1}^{n} w_i x_i + \theta)$$

The function f is called activation function. One of the most simple activations function is called Threshold function

$$f(s) \begin{cases} 1 & \text{if s} > 0 \\ -1 & \text{else} \end{cases}$$

This activation function is useful for classification problems. With this setup we can easily classify between 2 classes by projecting a line on a plane (binary classification). The Perceptron will classify one or the class based on the output that the function gives.

We should define two phases for our neural networks

- Learning →The network modifies the weights
- Generalization →We can use the network on new data for our purposes

During the learning phase the network receives the training data and with the use of certain algorithms it will change the weights to adjust the classification outputs. If we know our target we are doing obviously supervised learning.

The algorithm used by the perceptron to modify the weights is the following

1. Initialize the connections with a set of weights generated at random.
2. Select an input vector $\bar{x}$ from the training set. Let y be the output value returned by Perceptron for this input value and d the correct answer (target), that is associated to $\bar{x}$ in the dataset.
3. If $y \neq d$ (the calculated output is wrong), then the weights (w_i)of all connections are modified as follows

- if y > d $w_i := w_i - \eta x_i$
- if y < d $w_i := w_i + \eta x_i$

4. Back to 2

The algorithm terminates when all vectors in the training set are classified in a correct way or after a certain prefixed number of iterations. Remember that also the value of the threshold θ needs to be changed (but the input will always be 1). The parameter η is called learning rate and indicates by how much we need to adjust our changes, it can be fixed or variable (but always positive).

As we said we want to find a line that separates the 2 classes. The right term that will allow the generalization entering the realm of networks is hyperplane. With 2 inputs we have

$$w_1 x_1 + w_2 x_2 + \theta = 0 x_2 = -\frac{w_1}{w_2} x_1 - \frac{\theta}{w_2}$$

The straight line that we get allows us to graphically separate the two classes.
This straight line has the following properties

- All the points that belong to class 1 are "above" the line and all the ones of class 2 are "below"
- The weights vector is perpendicular to the separating line
- θ allows us to calculate the distance of the straight line to the origin

If a weight vector that allows us to obtain y = d for all training instances exists than the Perceptron will converge to a solution in a finite number of steps without caring about the initial choice of the weights. And if this vector exists then the classes are linearly separable.

If our data can be partitioned in more than two classes we are in the realm of multiclass classification. Given a problem in which data must be classified into more than two classes, more than two neurons are needed. We simply connect our input layer to an output layer made of a number of neurons equal to the number of classes. The Perceptron is able to separate data into 2^m classes depending on the binary output values of its output.

During training each single neuron is trained independently with the Perceptron learning rule. At the end then we will get a weight vector for each neuron. Using this vector we can define the needed hyperplane.

The class of problems that we saw is the one of linearly solvable ones. But if we have a XOR problem we can't really use a linear function since no set of straight line will be able to separate the two classes. Then for this particular problem we have to add a new layer that will correct the predictions. Our new network will have a hidden neuron between the input and the output layer (hidden layer). In order to solve non-linear problems we must use neural networks with at least one hidden neuron.

So far we only looked at neurons that use the threshold (step) function as activation. But we can have many more

- Logistic or sigmoid $f(x) = \frac{1}{1+e^{-ax}}$
- Hyperbolic tangent $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- Gaussian $f(x) = e^{-x^2}$

9.5.3 ADALINE

An evolution based on the normal perceptron is ADALINE, the difference is that the learning rule is called Delta Rule and can be seen as a generalization of the Perceptron learning rule. The difference is given on how the output is managed. The Perceptron uses as output the result of an activation function to learn while the Delta Rule uses the result of the summation to which it can, or can not, be applied an activation function, so:

$$y = \sum_{i=1}^{m} w_i x_i + \theta$$

or

$$y = f\left(\sum_{i=1}^{m} w_i x_i + \theta\right)$$

The main difference between Perceptron and Delta Rule is that the latter compares this calculated output with the desired output by using an error. The error function can be defined either as mean square error $E = \frac{1}{n}\sum_{i=1}^{n}(d_i - y_i)^2$ or as absolute error $E = \sum_{i=1}^{n}|d_i - y_i|$. The Delta Rule looks for the vector of weights that minimizes the error function using the method of gradient descent (then we arrive at an optimization problem). We can visualize this error as a surface where we look for the point that minimizes our function looking for the minimum.

If the problem is linearly separable then our surface is unimodal, and so we have only one global minimum and no local minimum. Delta Rule will try to minimize at each step the error. If the problem is not unimodal then ADALINE will probably fall into a local minimum.

To decrease the error E at each step the weights are modified by a quantity Δw which is proportional to the derivative of the function E $\rightarrow \Delta w = -\eta\frac{\partial E}{\partial w}$. The weights are modified in the negative direction of the gradient of E. At the minimum the derivative is 0 and so the algorithm stops.

- Initialize all connections with random weights;
- Repeat until (termination condition):
  - Select a training observation j
  - Calculate the output of the neuron for this observation:

$$y_j = f\left(\sum_{i=0}^{m} w_i x_{ji}\right)$$

  - If y_j approximates d_j in a satisfactory way, then do nothing, else, for each connection $i = 1, 2, \ldots, m$:
    - if $y_j < d_j$ then $w_i = w_i + \eta y_j(1 - y_j)x_{ji}$
    - else $w_i = w_i - \eta y_j(1 - y_j)x_{ji}$
- end repeat

As said we stop when we converge, but we can also decide a maximum number of iterations. Here we have an example of the derivative of the sigmoid

$$f(x) = \frac{1}{1 + \exp(-x)}$$

Its derivative $f'(x)$ can be calculated as follows:

$$\begin{aligned}
f'(x) &= \frac{\partial}{\partial x}\big(1 + \exp(-x)\big)^{-1} \\
&= -1 \cdot \big(1 + \exp(-x)\big)^{-2} \cdot \big(-\exp(-x)\big) \\
&= \frac{\exp(-x)}{\big(1 + \exp(-x)\big)^2} \\
&= \frac{1}{1 + \exp(-x)} \cdot \frac{\exp(-x)}{1 + \exp(-x)} \\
&= \frac{1}{1 + \exp(-x)} \cdot \left(\frac{1 + \exp(-x) - 1}{1 + \exp(-x)}\right) \\
&= \frac{1}{1 + \exp(-x)} \cdot \left(1 - \frac{1}{1 + \exp(-x)}\right) \\
&= f(x)\big(1 - f(x)\big)
\end{aligned}$$

Therefore, the derivative of the sigmoid can be expressed as a function of the sigmoid itself

9.5.4 Non-linearly separable problems

In general, non-linearly separable problems can be solved using multi-layer Neural Networks, i.e. Neural Networks in which one or more neurons are not directly linked to the output of the network (we call these neurons hidden neurons, and given that they are generally organized into layers, we call these layers hidden layers).

9.5.5 Backpropagation

Backpropagation is an algorithm that can be applied to a multi-layer perceptron in order to change its weights and optimize them. The main characteristics of a feed-forward neural network are:

- The neurons in a given layer receive as input the outputs of the neurons of the previous level
- There are no inter-level connections
- All the possible intra-level connections exist between two any subsequent layers

The Backpropagation is the most common learning rule for multi-layer Neural Networks (sometimes also called multi-layer Perceptron or Feed-Forward Neural Networks). The Backpropagation can be seen as a generalization of the Delta-Rule to multi-layer Neural Networks. The Delta-Rule modifies the weights of each neuron on the basis of the error committed by that neuron. To be able to extend the Delta-Rule to multi-layer networks is how to modify the weights of the neurons that belong to the hidden layers. In fact, the correct output for these neurons is not known, and thus we cannot directly calculate the error.

The basic idea of the Backpropagation is the following: the errors of the output neurons are propagated backwards to the hidden neurons. In other words: the error of a hidden neuron is defined as the sum of all the errors of all the output neurons to which it is directly connected.

The Backpropagation can be applied to networks with any number of layers, but the following theorem holds: Universal Approximation Theorem (Hartman, Keeler, Kowalski, 1990) Only one level of hidden neurons is sufficient to approximate any function (with a finite number of discontinuities) with arbitrary precision, provided that the activation functions of the hidden neurons are non-linear. This is one of the reasons why one of the most used configurations of a Feed-Forward multi-layer Neural Network is with only one layer of hidden units that use a sigmoidal activation function

The Backpropagation is composed by one forward step and one backward step. In the forward step, the weights of the connections remain unchanged and the outputs of the network are calculated propagating the inputs through all the neurons of the network. At this point, the error of each output neuron is calculated. The backward step consists in the modification of the weights of each connection. This calculation is made in the following way:

- For the output neurons, the modification is done by means of the Delta Rule
- for the hidden neurons, the modification is done by propagating backwards the error of the output neurons: the error of each hidden neuron is considered as being equal to the sum of all the errors of the neurons of the subsequent layer.

The weights of the connection that enter each neuron are updated using the formula:

$$w_{ij} = w_{ij} + \Delta w_{ij}$$

where w_{ij} is the weight of the connection between neuron i and neuron j, and:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Now the problem is how to calculate Δw_{ij} without having to calculate every time the derivative of the error function E.

1. Initialize all the weights in the network with random values.
2. Select a vector $\mathbf{x}$ of the training set (let $\mathbf{d}$ be the correct output – or target – that can be found in the dataset, corresponding to vector $\mathbf{x}$). Calculate the output of the network $\mathbf{y}$ for input $\mathbf{x}$.
3. If $\mathbf{y}$ is satisfactory (i.e., $\mathbf{y}$ approximates $\mathbf{d}$ in a sufficient way), then **go to** step 2. Else:

   3.1. For each output neuron j, calculate:

   $$\delta_j = (d_j - y_j) \cdot y_j \cdot (1 - y_j)$$

   and modify the weight of the connections that enter into neuron j as follows:

   $$w_{ij} = w_{ij} + \eta \delta_j y_i$$

3.2. For each hidden neuron j, calculate:

$$\delta_j = y_j(1 - y_j) \sum_k \delta_k w_{jk}$$

(where the sum is calculated over all output neurons k), and modify the weight of the connections that enter into neuron j as follows:

$$w_{ij} = w_{ij} + \eta \delta_j y_i$$

(In both cases, y_i is the value entering in neuron j from source i)

4. **go to** step 2.

The algorithm terminates when one of the following conditions is satisfied:

- The correct outputs **d** are approximated from the outputs **y** of the network in a "satisfactory" way for every vector **x** of the training set.
- A prefixed number of maximum iterations has been executed.

As for other algorithms we can improve Backpropagation by selecting the right kind of parameters, the main ones are

- Learning rate η →if is too small the learning will be slow, if is too big the network will be unstable
- Momentum α →we can use it to reduce the risk of staying too much inside a local minimum
- Number of hidden neurons →Usually based on experience, we risk to fall inside a local minimum

To have a good result for our network we have to avoid both underfitting and overfitting. We can find the sweet spot by selecting an appropriate size for our network.

9.5.6 Cyclic (or Recursive) Neural networks

An improvement on normal feed-forward networks can be obtained by using cyclic neural networks. Some example can be Jordan, Elman, Hopfield and Bolzman machines. In Jordan networks we have a state unit that helps the learning phase by combining input and state units. Backpropagation is the learning rule.

9.6 Neuroevolution and NEAT

We call Neuroevolution the artificial evolution of neural networks using evolutionary algorithms.

In traditional Neuroevolution we decide the topology structure for the evolving network before the experiment begins. Usually the starting topology is a single, hidden and fully connected layer of neurons. The evolution process will look for the weights to optimize the problem. Since we can set the weights as vectors we can apply things such as genetic algorithms and PSO.

Some variants can be obtained by:

- Having a dynamic size for our strings
- Use different ways to represent our chromosomes
- Use genetic programming to evolve the network (as topology, weights and activation functions)

One possible algorithm based on PSO is Greedy Iterative Layered PSO.

We can represent the connection matrix of a network with a bit string and the relative weight can be obtained via means of Backpropagation. The topology can be evolved using genetic algorithms and the fitness can be considered as the final performance of our network.

Even though is a very simple approach we also have some drawbacks like:

- The size of the connectivity matrix is huge
- The maximum number of nodes must be decided by a human
- A linear string of bits makes crossover difficult to be applied to get good results

It's known that modifying both topology and weights of a network is beneficial, then we want to use an evolutionary algorithm to do it. Even though we can approximate any function using one hidden layer we would like to minimize the size of our network as much possible. Another aspect is that if we minimize topologies while growing our structures incrementally we can speed up our learning.

9.6.1 NEAT

The main algorithm for this is called NeuroEvolution of Augmenting Topologies (NEAT). Everything is good, but we have some additional challenges to tackle:

- How to represent crossover in a good way
- How to protect a topological innovation
- How to optimize the size without resorting to a specific fitness

The basic idea of NEAT is to represent entire networks (topology and weights) by means of a linear representation. We also have ad hoc crossover and mutations over these linear structures (we recombine and mutate the networks). The basic idea of NEAT is that we start with an initial population of small individuals, and we let them grow during the evolution.

Since new structures are introduced incrementally we only save structures that are useful for our problem. Also since our initial population is minimal we have a minimal search space. Then we have a performance advantage over other approaches.

One of the main problem that we have is the one of Competing Conventions or Permutation problem, where the same network can be represented with a permutated string while being considered a new one. The consequence is that our crossover may produce damaged offspring with the loss of information as consequence.

The solution is gene alignment before sexual reproduction: the genes of the parents are aligned if they share the same information (homologous genes). This is done by using a unique ID called Global Innovation Number. With this ID we can also track how the gene was created. The ID is incremental.

9.6.2 Mutations

Mutations in NEAT are done on the weights (perturbation) and on the structure (add connection or add node).

- Add connection →a single new connection with a random weight is added to connect two previously unconnected nodes
- Add node →an existing connection is split, and a new node is placed inside it. The old connection is then disabled.

The innovation ID is inherited, and the new connections receive a new ID, so that we can track the changes. By keeping track of the innovations occurred in a certain generation we can give be sure that every innovation number is unique, and so we won't have any explosion of IDs.

9.6.3 Crossover

When we apply crossover we have already our genes lined up (matching genes). Genes that do not match are either disjoint or excess. In composing a new offspring, genes are randomly chosen from either parents at matching genes while all excess or disjoint are taken from the most fit parent.

In NEAT the crossover is asymmetric and directed, and we have a dominant parent. We always get one child from the crossover. This is for two main reasons:

- Fitness Bias →emphasizing the fitness parent promotes steady evolutionary progress
- Structural compatibility and diversity →producing two children can cause too similar structures

Even though NEAT tries to build small children the application of crossover will augment the size. Necessarily our starting individuals are really similar between themselves, but NEAT tries to diversify them as the evolution goes on. The idea is to protect the diversity.

9.6.4 Maintaining diversity in the population

One problem that can arise from NEAT is that smaller structures will be optimized faster than bigger ones, then the second will probably be removed easily from the population. To solve this we should protect the innovation by speciating the population.

This is done to allow our organisms to compete within their own niches and so the topological innovations are done inside a niche (implying that similar topologies are considered as a sort of similar species).

The number of excess and disjoint genes between a pair of genomes is a measure of their compatibility distance. The more different two genomes are the less compatible they will be. We can calculate this distance using the number of excess E, disjoint D and the average weight differences of matching genes $\bar{W}$, including the disabled genes, with the following formula

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \bar{W}$$

Where the coefficients $c_1, c_2 and c_3$ allow us to adjust the importance of the three factors. N is the number of genes in the larger genome.

With δ we can speciate using a compatibility threshold δ_t. An ordered list of species is maintained. In each generation genomes are sequentially placed into species. Each existing species is represented by a random genome inside the species from the previous generation. A given genome g in the current generation is placed in the first species in which g is compatible with the representative genome of that species. In this way species do not overlap. If g is not compatible with any existing species, a new species is created with g as its representative

9.6.5 NEAT Speciation

NEAT also uses fitness sharing. This mechanism imposes that each individual has to share the fitness with the other individuals in the same niche. So, niches cannot become too big (i.e., containing too many individuals).

Fitness is calculated as:

$$f'_i = \frac{f_i}{\sum_{j=1}^{n} \mathrm{sh}(\delta(i,j))} \tag{40}$$

Where f'_i is the adjusted fitness of individual i. It is calculated according to its distance δ to every other individual j in the population. The sharing function sh is set to 0 when the distance $\delta(i,j)$ is above the threshold δ_t; otherwise $\mathrm{sh}(\delta(i,j))$ is set to 1.

In practice, since species are already clustered by compatibility using the threshold δ_t, the denominator of the previous equation is equal to the number of individuals of the same species as i.

So, for each individual in a given niche, its fitness is divided by the number of individuals that belong to the same niche.

The smaller the niche (i.e., the smaller the number of individuals it contains), the better the fitness of its individuals.

So, niches cannot get too big, otherwise its individuals will have a bad fitness.

10 Text mining

10.1 Introduction to Text mining

10.1.1 Introduction to NLP

Natural language processing is an area of computer science and artificial intelligence that is concerned with the interaction between computers and humans in natural language. The ultimate goal of NLP is to enable computers to understand language as well as we do.

Some applications of NLP can be machine translation, dialog systems, search engines and predictive keyboards with autocorrectors.

10.1.2 Challenges of natural language

Natural language is made of two main elements:

1. Grammatical system, with its own rules, used by people to communicate
2. Natural evolution due to people communication (like new words or regularization of the language)

Some problems in its analysis can arise from the variability of it. Different sentences can have the same meaning (paraphrases) or a single sentence can have multiple meanings (ambiguity). NLP systems should also be able to generalize as much as possible (work in different languages), to achieve this we train it on a corpus. Since the system can encounter inputs that it never encountered (Out-of-Domain or Out-of-Vocabulary) we want to extend its knowledge to these new words or meanings.

10.1.3 The NLP pipeline

A typical NLP pipeline follows these steps: Corpus →Feature engineering →Task.

- **Corpus** →collection of text organized into datasets. It can be made up of everything that is written in natural language. A collection of more than one corpus is called corpora. First we split the corpus into train, validation and test set (or we can use k-fold cross-validation)
- **Feature Engineering** →we need a way to represent text in a way that a computer can understand it. The most basic way is the Bag-of-Words representation (glorified 1-Hot-encoding), it gives us a sparse representation of our documents.
- **Task** →we can have multiple tasks like keyword extraction, text similarity, sentiment analysis and many more. Given two documents we can transform them into sparse vectors and compare them with simple distance metrics. If they share a lot of words they will be close to each other. This is the basic for multiple tasks. We can use euclidean distance ($D(a,b) = \sqrt{\sum_{i=1}^{n}(b_i - a_i)^2}$) to calculate the distance matrix and apply KNN to label our documents.

10.1.4 Text preprocessing methods

Before applying BoW we need to define what's a word, this process is called **Tokenization**. We want to identify tokens (features in text), but it's not a trivial task. Some problems can arise from composed words and punctuation. It's also clear that with this kind of sparse representation we can encounter problems like the usual curse of dimensionality and the fact that we lose semantic relationships.

- **Curse of dimensionality** →The total dimension of the BoW is the vocabulary size. Our model can easily overfit. We can use dimensionality reduction techniques.
- **No semantic relationships** →BoW doesn't consider the semantic relationships between words since we discard the order and similar words can be considered as different features.

Following Zipf's law we know that some words have not a very big importance, but they are really common (the so-called stop words). Stop words presence means that we have more rules and more processing to apply.

We should also lowercase everything to reduce the vocabulary size (but take care of names) and normalize dates, numbers and names by either using regular expressions or applying a default token in their place.

We can also transform words in a way that if they represent the same meaning they are captured by the same token. This can reduce language inflection. Some ways are

- Stemming →Remove the last characters to obtain a shorter form even if it has no meaning.
- Lemmatization →Turns the words into their root word

We can also just consider certain type of words like verbs, nouns, adjectives and adverbs.

10.2 Word representations

10.2.1 Classification with BoW

We just throw our binary vectors inside a neural network.

10.2.2 Word representations

BoW is the baseline for every text mining task since it has a lot of limitations like curse of dimensionality, no semantic relationship, all words will have the same importance and the context is ignored even though it changes the meaning.

To solve these problems we want to use vectors to represent our words. From words to tokens to embeddings. Using embeddings for words is better than embed full documents since we can have a more granular and precise representation. One of the breakpoints was the creation of Word2Vec in 2013.

We want that our word representations will be able to capture semantic relationships in a mathematical way. Some ways to generate our word vectors are

- Term-Term matrix
- Point-Wise mutual information (PMI)
- Skip-gram(Word2Vec)

In more details

Term-Term matrix

Term-term matrix is of dimensionality |V|x|V| and each cell records the number of times the row word (target) and the column word (context) co-occur in the training corpus

	data	model	learning	algorithm
data	25	12	9	7
model	12	30	15	18
learning	9	15	28	14
algorithm	7	18	14	26

PMI

Point-wise Mutual Information (PMI) is a measure of how often two events x and y occur, compared with what we would expect if they were independent:

$$I(x,y) = \log_2 \frac{P(x,y)}{P(x)P(y)}$$

The point-wise mutual information between a target word w and a context word c is then defined as

$$PMI(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$$

It is common to use Positive PMI (called PPMI) which replaces all negative PMI values with zero

$$PPMI(w,c) = max(\log_2 \frac{P(w,c)}{P(w),P(c)}, 0)$$

To build our PMI between information and data

- Calculate co-occurence matrix
- Calculate word probability

- Calculate context probability
- Calculate co-occurence probability
- Create probabilities matrix
- Calculate PPMI

With this approach we solve the problems of semantic relationships and the one of the words having the same importance. We still have the curse of dimensionality and the fact that context is not accounted for.

10.2.3 Vector embeddings (Word2Vec)

We can use another strategy to solve also the curse of dimensionality problem. Using an approach called Skip-gram (or Word2Vec) we can create short and dense word vectors. This approach works in the following way

1. We start from the idea that similar words will tend to appear together
2. We train a classifier, a simple neural network with one hidden layer, to perform a binary prediction of the kind →Is word w likely to show up near c?
3. We learn the weights of the hidden layer, aka the word vectors that we are trying to learn

More in details

1. Define word and context, for each word in our corpus select the context window
2. Add negative samples for words that do not appear in the same context window
3. Define the task, usually if word w and c appeared in the corpus. Use the weights of our first model to represent the words
4. Train the model to get our embedding size and get the probabilities for each possible word
5. Retrieve the embeddings

After this pipeline we have the possibility to extract semantic relationships between words

10.2.4 Classification with Word2Vec

To make predictions using our embeddings we need to find a way to give the vectors as input. Some ideas can be to use the mean of the vector or apply an RNN (or a LSTM) since we consider the input as a varying one.

10.3 Sequential models

10.3.1 Review of word embeddings

From the previous class we know that some problems are present with Bow representation. To solve them we can use Word2Vec, but we still have the problem of having to input a sequence inside our neural network. We can average our vectors into one scalar, but we will lose a lot of semantic information. To solve this problem we can use recurrent neural networks (RNNs).

10.3.2 RNNs

To process sequential input we can use some models like RNNs and LSTMs. Some common tasks can be next word prediction, sequence labeling, sequence classification and machine translation.

A recurrent neural network is a network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input.
In a standard RNN architecture, three main matrices are involved:

- U: the input-to-hidden weight matrix, responsible for processing the current input.
- W: the hidden-to-hidden recurrent weight matrix, which propagates information from the previous hidden state to the current one.
- V: the hidden-to-output weight matrix, used to generate the network output.

These matrices are shared across all time steps, meaning that the same parameters are reused throughout the sequence. This parameter sharing allows the network to generalize across sequences of different lengths and reduces the total number of trainable parameters.

At each time step t, the network computes:

$$h_t = f(Ux_t + Wh_{t-1})$$

where:

- x_t is the input vector at time step t,
- h_{t-1} is the hidden state from the previous time step,
- h_t is the updated hidden state,
- $f(\cdot)$ is a nonlinear activation function such as tanh or ReLU.

The output of the network is then computed as:

$$y_t = g(Vh_t)$$

where:

- y_t is the output at time step t,
- $g(\cdot)$ is typically a softmax or another activation function depending on the task.

Because the hidden state carries information across time, RNNs are particularly useful for sequential learning problems where context and order are important.

For a sentence/document, each word (embedding) is processed through the network one step at the time. In parallel, the hidden vector of the previous word, moves in the network and is used to calculate the hidden vector of the next word. The main problem with RNNs is related to long dependencies, The final hidden state reflects more information about the end of the sentence than its beginning.

Another problem is the one of vanishing gradient, during the backward pass of training, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence. A frequent result of this process is that the gradients are eventually driven to zero, which means that the initial weights cannot be updated properly.

The long dependencies' problem can be solved by using bidirectional models.

10.3.3 LSTMs

As said we can use bidirectional models to solve some problems related to RNNs. LSTMs models are explicitly designed to avoid the long term dependency problems. The context management is divided into two subproblems:

- Removing information no longer needed from the context
- Adding information likely to be needed for later decision-making

To accomplish this LSTMs use:

- Context layer (cell state)
- A specialized neural unit that work as gates to control the flow of information into and out of the units that comprise the network layers.

The cell state is the horizontal line that runs through the top of the diagram. It acts as a transport highway that transfers relative information all the way down the sequence chain. Is like the memory of the network.

The main gates in LSTMs are

- **Forget gate** →This gate decides what information should be thrown away or kept in the cell state. Information from the previous hidden state and information from the current input is passed through the sigmoid function. Values come out between 0 and 1. The closer to 0 means to forget, and the closer to 1 means to keep.

- **Input gate** →This gate decides what information should be used to update the cell state: first, the previous hidden state and the current input are passed through a sigmoid function, which produces values between 0 and 1 to determine which information is important to keep; simultaneously, the same inputs are passed through a tanh activation function to squash values between -1 and 1, helping regulate the network; finally, the outputs of the sigmoid and tanh functions are multiplied together so that the sigmoid gate controls which information from the tanh output should be retained. Next, the cell state is pointwise multiplied by the forget vector, allowing the network to discard information associated with values close to 0; afterward, the output of the input gate is added pointwise to the cell state, updating it with new information considered relevant by the neural network.
- **Output gate** →This gate determines the next hidden state or, at the final time step, the prediction of the network: first, the previous hidden state and the current input are passed through a sigmoid activation function; then, the updated cell state is passed through a tanh function; afterward, the outputs of the sigmoid and tanh functions are multiplied together to determine which information should be carried forward in the hidden state; the resulting vector becomes the new hidden state, while both the updated cell state and hidden state are propagated to the next time step or forwarded to an output activation layer to produce the final prediction.

10.3.4 Seq-to-seq models

We already cited the task of machine translation, but we also have another one that enters the realm of sequence to sequence, chatbots.

Sequence-to-sequence (Seq2Seq) models are a class of neural network architectures designed to transform one sequence into another sequence. They are widely used in tasks such as machine translation, text summarization, speech recognition, and question answering.

In Seq2Seq models, both the input and the output are sequences of variable length. For example, in a machine translation task, the input sequence may be a sentence in English, while the output sequence is the translated sentence in another language.

A Seq2Seq architecture is generally composed of two main components:

- An **encoder**,
- A **decoder**.

The encoder processes the input sequence and converts it into a fixed-length vector representation commonly called the **context vector**. This vector summarizes the semantic information contained in the entire input sequence.

The encoder is typically implemented using a recurrent neural network (RNN), such as a Long Short-Term Memory (LSTM) network or a Gated Recurrent Unit (GRU). At each time step, the encoder receives an input token and updates its hidden state according to the sequential information processed so far.

Given an input sequence:

$$x_1, x_2, x_3, \ldots, x_T$$

the encoder updates its hidden state recursively as:

$$h_t = f(x_t, h_{t-1})$$

where:

- x_t is the input token at time step t,
- h_{t-1} is the previous hidden state,
- h_t is the current hidden state.

After the final input token has been processed, the last hidden state of the encoder is used as the context vector:

$$c = h_T$$

This context vector is then passed to the decoder as its initial hidden state.

The decoder is another recurrent neural network responsible for generating the output sequence one element at a time. At each time step, the decoder receives:

- The previous output token,
- The previous hidden state,
- The context vector.

The decoder hidden state is computed as:

$$s_t = f(y_{t-1}, s_{t-1}, c)$$

where:

- y_{t-1} is the previously generated output token,
- s_{t-1} is the previous decoder hidden state,
- c is the context vector generated by the encoder.

The decoder output is obtained through a linear transformation followed by a softmax activation function:

$$y_t = \text{softmax}(W s_t)$$

Where W is the output weight matrix.

The softmax function converts the decoder output into a probability distribution over the vocabulary. The token with the highest probability is selected as the generated word for the current time step.

During training, the decoder often receives the true previous target word as input, a technique known as **teacher forcing**. During inference, instead, the decoder uses the word generated at the previous time step.

As an example, consider the sentence:

"Great movie"

which must be translated into Portuguese:

"Ótimo filme"

The encoder first converts the input words into embedding vectors:

$$\text{great} \rightarrow [0.03,\ 0.98]$$

$$\text{movie} \rightarrow [0.76,\ 0.85]$$

The encoder processes these embeddings sequentially and generates the context vector. The decoder then uses this context vector to produce the translated sequence token by token:

$$< s > \rightarrow \text{ótimo} \rightarrow \text{filme} \rightarrow < /s >$$

At every decoding step, the softmax layer produces a probability distribution over all words in the vocabulary, allowing the model to determine the most likely next word.

Seq2Seq architectures represented one of the first successful approaches for neural machine translation and sequence generation tasks. However, traditional Seq2Seq models may struggle with long sequences because the entire input information must be compressed into a single context vector. This limitation motivated the development of attention mechanisms and Transformer architectures.

10.4 Attention and transformers

10.4.1 Attention mechanisms

In the previous class we saw how RNNs work but with modern LLMs we need to add some elements. For example, we need to talk about attention mechanisms. As we know classical encoder-decoder methods have to encode all the information about a sentence inside a vector while the decoder must create the translation using only that vector.

The final hidden state is like a bottleneck where all the information is represented in a smaller representation with respect to the original. The decoder if trained well should be able to recreate the original input using only the context vector. One problem is that languages behave differently from the syntactic point of view, so the model should remember the semantic information as much as possible.

A way to solve this problem is using the mechanism of attention. We allow the decoder to get information from all the hidden states of the encoder, not just the last one. The first idea of attention is to create a fixed-length vector by taking a weighted sum of all the encoder hidden states. At each step, the decoder does an extra step before producing it's output. The pipeline is the following:

- Look at the sets of hidden states, each encoder hidden state is most associated with a certain word in the input sentence
- Give to each hidden state a score
- Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

In more details

- Compute how much to focus on each encoder state. Relevancy is captured by computing for each state a score measure. This score is called dot product attention $\text{score}(h^d_{i-1}, h^e_j) = h^d_{i-1} \cdot h^e_j$
- The score that we get is a scalar that reflects the degree of similarity between two vectors. To make use of these scores we have to normalize them with a softmax to create a vector of weights. This vector will tell us the proportional relevance of each encoder hidden state to the prior hidden decoder state $\alpha_{ij} = \text{softmax}(\text{score}(h^d_{i-1}, h^e_j) \forall j \in e)$
- Finally we compute a fixed length context vector for the current decoder state by taking a weighted average over all the encoder hidden states $c_i = \sum_j \alpha_{ij} h^e_j$

10.4.2 Transformer architecture

Some stories on GPU and why generating cat videos is making a shitty 12mb GPU cost like 9000 dollars. Anyway we can't use GPUs for RNNs because parallelization is basically impossible. To solve this problem Self-attention was created.

Self-attention allows each token to directly attend to all other tokens in the sequence at the same time, capturing dependencies regardless of distance. Here we have the first important building block of transformers.

The standard architecture for building modern large language models is the Transformer. Like LSTMs, Transformers can capture long-distance relationships between words in a sequence. However, unlike LSTMs, Transformers do not rely on recurrent connections that process tokens step by step. Instead, Transformers use self-attention, a mechanism that allows each token to directly attend to all other tokens in the sequence. This makes it possible to compute token representations in parallel, improving training efficiency and making much better use of GPUs.

Transformers are made of two main types of blocks: encoders and decoders. The Encoder reads the input sequence and transforms each token into a contextual representation. The Decoder uses these representations to generate the output sequence, one token at a time. In the original Transformer architecture, the encoder and decoder work together: the encoder represents the input, and the decoder produces the output based on both the previous generated tokens and the encoder's representations.

Each transformer will have many encoders and many decoders Inside each Encoder, we will find a layer of self-attention and a standard Neural Network - information moves forward through the encoders Decoders receive the output from the Encoder and move them through two attention layers (self-attention and encoder-decoder attention) and a (feed-forward) neural network. The last decoder layer produces an output.

10.4.3 Transformer encoders

Some Transformers use only the encoder, because the goal is to understand text rather than generate new text. Encoder-only models read the full sentence at once, capturing context from both previous and following words. They produce rich, context-aware word and sentence representations, usually stronger than static embeddings like Word2Vec or GloVe. These representations can be reused in downstream tasks such as classification, sentiment analysis, NER, similarity, clustering, and retrieval (Transfer Learning).

BERT (Bidirectional Encoder Representations from Transformers), developed by Google, is probably the most famous Encoder-only Transformer model. BERT (BERTLarge) model uses 24 encoder layers, referred to in the paper as Transformer blocks. BERT uses 16 self-attention heads, allowing the model to capture multiple contextual relationships in parallel. It has a hidden size of 1024 units, making its internal representations larger

and more expressive. This means that each token is represented by a 1024- dimensional contextual embedding produced by the final encoder layer. The representations generated by BERT can then be fed to another model as word embeddings

Transfer learning is the process of reusing knowledge learned from one task to improve performance on another task. In NLP, models such as BERT are first pre-trained on massive text corpora to learn general language patterns. They are called Pre-trained Language Models (PLMs) This models can be called Foundation models - very large pre-trained models trained on broad and diverse datasets at massive scale. The pre-trained model is then adapted (fine-tuned) to a specific downstream task using a smaller labeled dataset
BERT was pre-trained on large unlabeled datasets, including English Wikipedia and the Bookworm's collection of books. The model learns language patterns using self-supervised tasks such as Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

- In MLM, during training, random words are masked, and BERT learns to predict them using context from both directions of the sentence.
- In NSP, BERT receives two sentences as input and predicts whether the second sentence logically follows the first.

The original BERT Large model contains approximately 340 million parameters, with 24 encoder layers, 16 attention heads, and 1024 hidden dimensions. BERT training is a two-step process: first pre-training a model (which is already done when we use it) and then fine-tuning it for a particular task. Finally, we can fine-tune it for tasks like sentiment analysis or predicting movie reviews.

Word and Sentence Embeddings

Since BERT produces multiple representations for the same word across layers, we must choose how to extract the final embedding. To extract embeddings for each word instead of a sentence level one, we can aggregate information from all token embeddings using pooling strategies. Common approaches include:

- Last Layer: use the final hidden representation of each token.
- Sum all layers: sum the token representations from all Transformer layers.
- Sum last four: sum the token representations from the final four layers.
- Concat last four: join the final four token representations into one larger embedding vector.

BERT adds a special token called [CLS] at the beginning of every input sequence. Across multiple Transformer layers, the [CLS] embedding gradually becomes a contextual summary of the entire sequence. During pre-training objectives such as Next Sentence Prediction, but also during fine-tuning, we can explicitly force [CLS] to capture sentence-level information. After the final encoder layer, BERT outputs one contextual embedding per token. For sequence classification tasks, the final embedding corresponding to [CLS] is usually selected as the sentence representation. This vector is then fed into a simple classifier, typically a linear layer followed by Soft max or a Logistic Regression.

Classification and Variations

We can perform classification directly with a task-specific model or indirectly with general-purpose embeddings. A task-specific model is a representation model, such as BERT, trained for a specific task like sentiment analysis. Alternatively, BERT can be used to generate general-purpose context embeddings that can be used for a variety of tasks not limited to classification, like semantic search. Over the years, many variations of BERT have been developed, including:

- RoBERTa
- DistilBERT
- ALBERT
- DeBERTa

10.5 LLMs

10.5.1 Classification with encoders

To extract embeddings for each word instead of a sentence level one we can aggregate information from all token embeddings using pooling strategies like:

- Last layer
- Sum all layers
- Sum last four
- Concat last four

BERT adds a special token called [CLS] at the beginning of every input sequence. This token during self-attention can attend to all words in the sentence to help the making the context. After it BERT will output one contextual embedding per token. This will then be sent into a simple classifier.

We can perform classification directly with a task specific model or indirectly with a general-purpose one. In this setting a task-specific model is one model that is built to support one specific work, like sentiment analysis. The embeddings can be used also for semantic search.

To use BERT we have to

- Tokenize the input sequence using the tokenizer of the transformer model
- Embed our sentences
- Use the embeddings as features
- Train a classification model

10.5.2 Self-attention

Recall that like LSTMs, transformers can capture long-distance relationships between words in a sequence. However, the work for the seconds use self-attention and not recurrent connections. This mechanism will allow each token to directly attend to all other tokens in the sequence.

Self-attention is made of two steps

- Relevance scoring for each position
- Combine the information based on those scores

In the encoder we have to generate as always a hidden state, using self attention we have to consider 3 main elements

- Query →current focus of attention
- Key →context input being compared to the current focus
- Value →used to compute the output for the current focus of attention

The three of them have different roles, to capture them we use weight matrices to project each input vector x_i into a representation of its role as a key (K), query(Q) or a value (V).

- $q_i = W^Q x_i$
- $k_i = W^K x_i$
- $v_i = W^V x_i$

Self attention is carried out by the interaction of these matrices, that are produced by multiplying the input layer with the projection matrices. The scoring of the relevance is done by multiplying the query associated with the current position with the key matrix. Self attention combines the relevant information of previous positions by multiplying their relevance scores by their respective value vectors. With this step we get and improved and contextualized embedding representation of the current word.

Since each output y_i is computed independently this entire process can be parallelized by packing the input embeddings of the N tokens of the input sequence into a single matrix $X \in \mathbb{R}^{Nxd}$. Each row of X is the embedding of one token of the input. The next step is to multiply X by the key, query and value matrices

- $Q = XW^Q$
- $K = XW^K$
- $V = XW^V$

We can also get query and key together by multiplying Q and K^t. After we can scale these scores, take the softmax and then multiply the result by V. We can reduce self-attention as

$$\text{SelfAttention(Q, K, V)} = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

10.5.3 Deeper into the encoder block

The self-Attention calculation lies at the core of what's called a transformer block which includes also additional feedforward layers, residual connections and normalizing layers. The transformers as we have described them don't have any notion about the positions of the tokens of the input, this means that scrambling the order of the inputs will give the same answer. One solution is to modify the input embeddings by combining them with Positional Embeddings specific to each position in an input sequence.

We can get better LLMs by doing attention multiple times in parallel, increasing the model's capacity to attend different types of information. Hence, we have many "self-attention heads". Each head in a self-attention layer is provided with its own set of key, query and value matrices. These are used to project the inputs into separate key, value and query embeddings separately for each head, with the rest of the self-attention computation remaining unchanged.

$$\text{head}_i = \text{SelfAttention}(Q, K, V)$$

The outputs from each head are concatenated and then using yet another linear projection to reduce it to the original output dimension for each token.

Residual connections in transformers are implemented by adding a layer's input vector to its output vector before passing it forward. Residual connections pass information from a lower layer to a higher one without going through an intermediate. This allows information from the activation going forward and the gradient going backwards to skip a layer. This allows us to improve learning and gives higher level layers direct access from lower layers.

These summed vectors are then normalized using layer normalization using a variant of the standard score by applying it to a single hidden layer. In the standard implementation we introduce two parameters called γ and β representing gain and offset values.

The representations generated by the encoder are

- Contextualized →Each word is represented based on its contextual meaning, allowing the same word to have different representations depending on surrounding words
- Bidirectional →Representations are influenced by both left and right context
- Position-aware →Using positional embedding the model retains information about word order

10.5.4 The architecture of decoders

In the decoder we can also find a layer of encoder-decoder attention (where the decoder attends to the outputs of the encoder). The decoder attention is called masked attention because it can only attend to previous tokens because it can't look into the future. The encoder starts by processing the input sequence. The output of the top encoder is then transformed into a set of attention vectors K and V. These are to be used by each decoder in its "encoder-decoder attention" layer which helps the decoder focus on appropriate places in the input sequence.

The decoder has an additional attention layer that attends to the output of the encoder. At a high level of abstraction, Transformer LLMs take a text prompt and output generated text. The decoder decides which of the tokens to output by sampling based on the probabilities. The tokens with the highest probability after the model's forward pass are chosen. Transformer LLMs generate one token at a time, not the entire text at once. An output token is appended to the prompt, then this new text is presented to the model again for another forward pass to generate the next token.

The final linear layer (that is followed by a softmax one) turns the output into a word. This linear layer is a simple neural network that projects the vector produced by the stack of decoders into a bigger vector called logits vector. The logits is then sent into the softmax layer that will turn the scores into probabilities and produce the most probable output. The previous process is repeated until a special symbol is reached indicating that the transformer decoder has completed its output. The predicted word is fed to the bottom decoder in the next time step, and the decoders send up their decoding results. Just like in the encoder inputs we embed and add positional encoding to those decoder inputs to indicate the position of each word.

In summary

1. Sample a word in the output to get our first input
2. Use the word embedding for that first word as the input to the network at the next time step
3. Continue generating until the end marker is reached

10.5.5 Generative AI models

Two main kinds of LLMs exist

- Proprietary models →Also known as closed source, they can be accessed by an API, as a result the details of their code and architecture are not shred with the user
- Open models →they are directly managed by the user and so people can see code and architecture

As we know these kinds of models are huge and to reduce their weight and size we can try different strategies

- Quantization →reduces model size and memory usage by reducing the number of bits needed to represent the parameters of an LLM. It enables them to run faster and with less powerful hardware requirements
- GGUF →GPT-Generated Unified Format, a file format used to represent a compressed version of the quantized LLM

10.5.6 Model Configuration

LLMs can generate different outputs even if they are presented with the same input. We can control the kind of output that we receive modifying some parameters

- Temperature →the randomness of the generated text. How easy is to select tokens that are less probable
- Top_p →nucleus sampling, control which subset of tokens the LLM can consider
- Top_k →how many tokens the LLM can consider

Another way to shape our output is to use some prompt engineering. The two most basic components are the instruction and the data it refers to. If we extend the prompt with an output indicator that allows for some specific outputs we can transform our LLM into a classifier.

If we provide our LLM with examples of exactly the thing that we want to achieve we enter the realm of incontext learning. Zero-shot prompting does not use examples, one-shot prompts give one example and few-shot prompts use multiple examples.

We can also define the system prompt as the set of hidden instructions that an LLM has to follow before it interacts with a user. The maximum number of tokens an LLM can process simultaneously is called context window. This can be considered as a sort of memory of the model. One of the most intuitive forms of giving LLMs memory is by using Conversation Buffer Memory (we append a past conversation in our prompt). If we reach the end of our context window we can apply Conversation Summary Memory where we first ask another LLM to summarize our conversation.

10.5.7 Retrieval Augmented Generation (RAG)

LLMs are trained on massive amount of data, but they cannot know some private information. Also, they are able to hallucinate or generate wrong answers if the information that we want to know is missing. RAG is an AI framework that grounds LLMs with specific and authoritative data outside their original training data. A basic RAG pipeline is made up of a search step followed by a grounded generation step where the LLM is prompted with the question and the information retrieved from the search step.

The first step is to chunk our documents and save them in a vectorial database. We can then query this database for information about the knowledge base. To improve our search we can split documents into smaller chunks at different levels (sentence, paragraph or overlapping window). Once the query is embedded we need to find the nearest vector to it from our text archive. We can compare embeddings to find the most similar documents to a query. This is called semantic search.

Semantic search can be done with different kinds of strategies

- Dense search →uses embedding to capture meaning and intent
- Keyword search →matches exact terms in documents while ignoring semantics
- Hybrid search →run keyword search and semantic one while combining the scores of both methods using a weighted formula

We can find the most relevant information to an input prompt by comparing the similarities between embeddings. The most relevant information is added to the prompt before giving it to the LLM.

10.5.8 Agentic AI

On its own a standalone LLM is primarily performing next-token prediction. Hence, LLMs can struggle with some tasks like basic arithmetic. LLM agents are advanced systems that use LLMs as their brain to understand problems, break them into steps and use tools to complete tasks.

In agentic AI the LLM is the cognitive core, augmented by 3 primary components:

- Memory →short term and long term
- Tools →APIs that enable real-time interaction with dynamic environments
- Planning/reasoning →The core generative capability of the LLM, which forms plans and generates responses

With memory, tools and planning our agent can do ReAct (Reason + Act). If our problem is too complex for the capabilities of a single agent we might need to use Multi-Agent Systems (MAS). These agents can collaborate with agent orchestration using a supervisor model